QUANTUM RESISTANT CRYPTOGRAPHIC SOLUTIONS

QSC

Cryptographic Library Comprehensive Engineering Qualification and Security Assessment

TECHNICAL QUALIFICATION AND DUE-DILIGENCE DOSSIER

Source-controlled cryptographic evidence, architecture, interoperability, negative testing, native platform qualification and technical due diligence.

15 / 15	2 / 2	354
Native CI configurations	Consolidated evidence gates	Archived source files

ISSUED 06 NOVEMBER 2026 • EXTERNAL DISTRIBUTION EDITION 1.1

Controlled implementation baseline: QSC 2026-11-06-B. Qualification evidence: QSC-ARM-Tests, 15 native test configurations, 2 consolidated matrices.

PUBLICATION RECORD / EXTERNAL TECHNICAL REVIEW

Document Control and Evidentiary Status

Attribute	Recorded value
Subject	QSC cryptographic library, November 6 implementation
Issuing organization	Quantum Resistant Cryptographic Solutions Corporation (QRCS)
Evidence method	Controlled source inventory, cryptographic test records, consolidated hosted CI evidence
Repository	QRCS-CORP/QSC-ARM-Tests
Qualification branch	qualification/acvp-qsc-20261106
Qualification commit	07f32d8a18a91f7a9035994adcbecd234a09b5ac
Source archive	QSC 2026-11-06-B.zip
Archive SHA-256	2e64df12c4956c4f58cac1aa468a0765f21a05b23dc4f86a257d48ed15dbf45c
Recorded engineering status	Offline qualification: 15/15 configurations PASS; consolidated status PASS_WITH_COVERAGE_GAPS
Independent validation status	See Qualification Scope and Accreditation Status below

Qualification Scope and Accreditation Status

This is a QRCS-issued engineering qualification and technical due-diligence report, based on controlled QRCS source and recorded test evidence. It is not an NVLAP-accredited laboratory report and does not confer NIST ACVP algorithm certificates, CMVP validation or FIPS 140-3 module certification. Official validation, if pursued, is a separate controlled evaluation. Within the stated offline test scope, all fifteen native configurations passed and both consolidated evidence gates succeeded; the original harness retained its PASS_WITH_COVERAGE_GAPS classification.

Evidence acceptance policy

Primary findings are attributed to the controlled implementation archive and the two original consolidated CI evidence records. The earlier local Linux compiler, interoperability and sanitizer campaign is preserved as a distinct source-baseline study. Source identifiers, compiler configurations and test-report boundaries are disclosed so the conclusions can be independently reconciled.

INVESTOR AND PROCUREMENT SUMMARY

Executive Abstract

The QSC cryptographic library is a substantial native C software asset that combines conventional symmetric and public-key algorithms, standardized post-quantum cryptography, TLS and X.509 integration, and interfaces for hardware-backed key services. The reviewed November 6 source archive contains 354 files: 176 C implementations, 164 headers and 14 supporting build/project entries. A lexical inventory of public export markers identifies 1,846 distinct QSC-prefixed function declarations across 134 headers. These are verifiable codebase attributes rather than measures of cryptographic security.

The principal qualification outcome is a complete fifteen-configuration offline cryptographic test matrix. The evidence covers Windows Server 2022 x64 and Ubuntu 24.04 x64, plus Linux ARM64 under GCC and Clang and macOS ARM64 under Apple Clang. Each environment exercised QSC security classes 1, 3 and 5. All fifteen runners passed the defined qualification gates. Two independently generated consolidated records report success, consistent source revisions and no missing, failed or unexpected runner artifacts.

Each native result includes passing legacy AES, SHA-2/HMAC, SHA-3/SHAKE, ML-KEM, ML-DSA and SLH-DSA suites, as well as passing higher-level adapter/interoperability predicates. The legacy test parser recorded a total of 315 successful marker occurrences across the fifteen configurations. This quantity must not be interpreted as 315 independent algorithm certificates or an exhaustive test of every QSC public API.

The development program also documented an earlier five-profile native Linux x86-64 campaign comprising 42,475 successful functional assertion executions, covering streaming hashes, authenticated encryption, boundary conditions, ECC negative inputs, post-quantum interoperability and a sanitizer build. Its historical source baseline is different from the final common-source qualification, and the two result sets are not merged into one unsupported performance or security claim.

The controlled implementation also includes a configurable nine-profile CMake build system and policy-aware hardware-key provider interfaces that support HSM, TPM, secure-element and platform key-service integrations. Together with the qualification evidence, these capabilities form a concrete basis for integration planning, customer technical review and diligence on enterprise or embedded deployments. The precise external description is: source-pinned offline cryptographic qualification completed across fifteen native configurations, with the coverage exclusions recorded in the original evidence.

DOCUMENT NAVIGATION

Contents

01 Executive Findings and Qualification Position 7
02 Assessment Basis, Independence and Use of Evidence 9
03 Commercial and Technical Decision Summary 11
04 Report Architecture and Reading Guide 13
05 Source Snapshot, File Composition and Engineering Footprint 15
06 Public API Surface and Integration Contracts 17
07 CMake Deployment Profiles, Source Selection and CPU Optimization 19
08 System Support: Memory, Integer, Concurrency and Networking 22
09 Advanced Encryption Standard: Modes and Operational Safety 24
10 SHA-2 and HMAC: Digest and Authentication Foundations 26
11 SHA-3, SHAKE and Keccak State Discipline 27
12 KMAC, cSHAKE, HKDF, PBKDF and KDF Interfaces 28
13 ChaCha20, Poly1305 and Stream-Authenticated Processing 29
14 QRCS-Specific RCS, CSX and Related Constructions 30
15 Elliptic-Curve Cryptography: P-256, P-384 and P-521 31
16 Edwards-Curve Signatures and Diffie-Hellman 32
17 ML-KEM and FIPS 203 Parameter-Set Coverage 33
18 ML-DSA and FIPS 204 Signature Assurance 34
19 SLH-DSA SHAKE Variants and FIPS 205 35
20 Additional Post-Quantum Interfaces: HQC and Classic McEliece 36
21 TLS 1.3, Key Schedule and Cryptographic Binding 38
22 X.509, ASN.1, Certificate Lifecycle and PKI Components 39
23 Hardware Key Services, Provider Contracts and Platform Integration 40
24 Entropy, Randomness and Sensitive-Key Lifecycle 43
25 Windows, Linux and macOS Portability 44
26 Methodology: Positive, Negative and Differential Tests 47
27 Historical Native Linux x86-64 Differential Campaign 49
28 ACVP Harness Architecture and Controlled Source Revisions 51
29 Stage 1: Build Completeness and Immutable Source Evidence 53
30 Stage 2: Strict Qualification and Evidence Integrity 54
31 Stage 3: Registered-Capability and Parameter Accounting 55
32 Final Fifteen-Configuration Hosted Matrix 56
33 Success Markers, Suite Inventory and Stage Evidence 58
34 Qualification Control Verification and Rerun Evidence 60
35 Cross-Compiler and Cross-Architecture Consistency 62
36 Memory Safety, Sanitizer Results and Static Analysis 63
37 Constant-Time Behavior, Fault Handling and Side-Channel Claims 64

38 Self-Tests, Module Error States and FIPS-Readiness 65
39 Threat Model, Misuse Resistance and Negative Inputs 66
40 Technology Portfolio and Differentiation 68
41 Customer Integration and Procurement Considerations 69
42 Intellectual Property, Licensing and Asset Transfer 70
43 Engineering Governance and Reproducibility 71
44 Due-Diligence Assessment and Evidence-Weighted Conclusions 72
45 Final Technical Conclusion 73
Appendix A Runner-by-Runner Qualification Evidence 75
Appendix B Legacy Suite and Stage Evidence 78
Appendix C Complete Source-File Inventory 79
Appendix D Public Exported API Inventory 87
Appendix E Source Provenance and Chain of Custody 120
Appendix F Historical Local Test Ledger 121
Appendix G Standards and Reference Sources 122
Appendix H Terminology and Evidence Definitions 123
Appendix I Reporting and Reproduction Protocol 124

PART I

EXECUTIVE ASSESSMENT

The evidence position, intended use of the publication and technical-commercial conclusions before the detailed examination.

EVIDENCE-LED TECHNICAL EXAMINATION

CHAPTER 01 | I. EXECUTIVE ASSESSMENT

01 Executive Findings and Qualification Position

QSC is a substantial native C cryptographic and systems library whose source-defined breadth is supported by a completed, source-pinned, fifteen-configuration offline qualification campaign. The principal findings cover cross-platform execution, cryptographic algorithm-family coverage, evidence integrity, hardware-provider architecture and configurable deployment profiles.

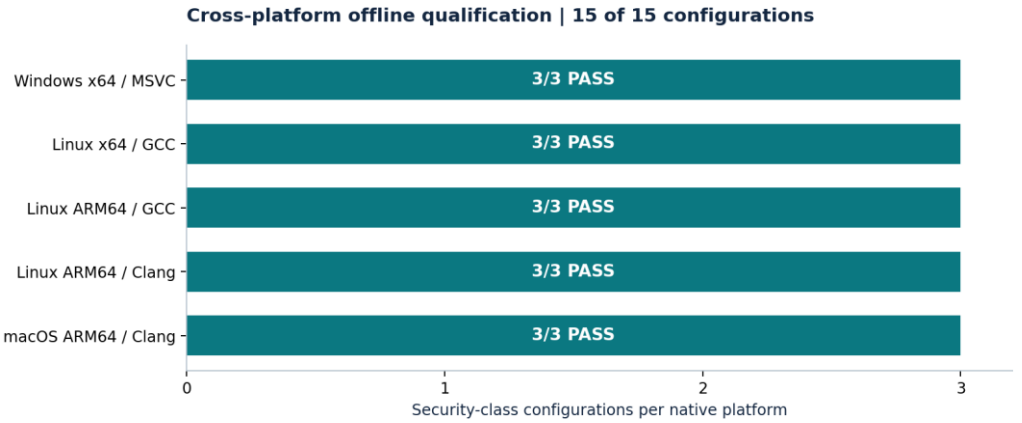


Figure 1. Native platform and security-class results for the fifteen-runner qualification campaign.

1.1 The outcome that can be demonstrated

The two archived consolidated GitHub evidence packages each report success, uniform source revision, no missing runner results, no failed runner results, and no unexpected artifacts. Six jobs executed on Windows x64 and Linux x64; nine on Linux ARM64 and macOS ARM64. Three QSC security-class settings were exercised per platform/compiler family. These results are unusually useful because they describe execution across dissimilar operating-system and instruction-set environments, rather than one green build on a developer machine.

1.2 Qualification result and scope

The fifteen successful configurations cover the defined offline harness and its declared algorithm families. Both consolidated evidence gates passed and retained the explicit result status PASS_WITH_COVERAGE_GAPS. The qualification boundaries and accreditation status are defined once in the Document Control section; further chapters describe test coverage only where directly relevant to an individual technical claim.

1.3 Why the assurance is commercially relevant

Enterprise customers evaluate more than mathematical primitives. Portability affects deployment schedules; deterministic evidence affects procurement and legal diligence; visible regression gates affect maintenance cost; and FIPS readiness affects access to controlled procurement. The archived evidence supports a demonstrable engineering base from which regulated validation work can proceed, while leaving that additional validation as a separate decision and budget item.

1.4 Measured implementation inventory

The controlled November 6 source baseline contains 354 files: 176 C implementation files, 164 headers and 14 supporting build/project files. A lexical scan of QSC_EXPORT_API declarations identified 1,846 distinct QSC-prefixed function names across 134 headers. The full source and interface inventories appear in the technical appendices.

Chapter conclusion

The demonstrated outcome is a successful, source-consistent offline qualification across fifteen native configurations, supported by an extensive and traceable implementation inventory. The findings provide a substantive technical foundation for enterprise and acquisition due diligence.

CHAPTER 02 | I. EXECUTIVE ASSESSMENT

02 Assessment Basis, Independence and Use of Evidence

This assessment is organized as a source-specific engineering qualification review. Primary evidence comprises the controlled implementation archive and original consolidated GitHub Actions records, supplemented by separately identified historical local engineering tests. The report applies consistent provenance and acceptance rules to every technical finding.

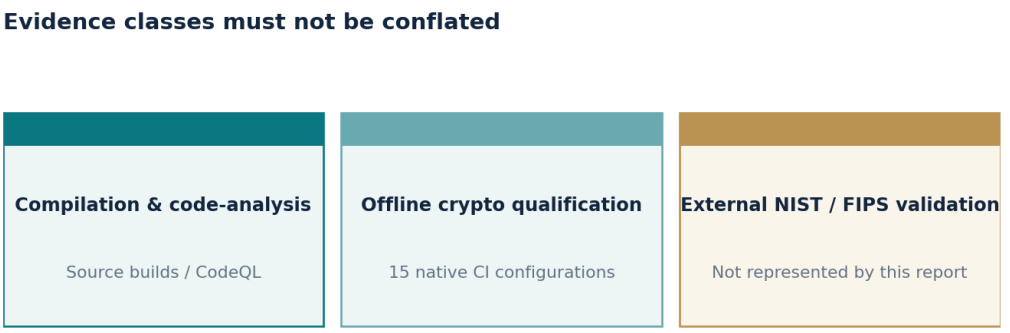


Figure 2. Evidence-assurance hierarchy distinguishing implementation, native tests and evaluation scope.

2.1 Primary evidence hierarchy

Highest weight is given to the original GitHub-generated consolidated JSON and Markdown records, which specify the job identity, source hashes, security class, passing suites and gate status. A second layer consists of the controlled QSC source archive, whose SHA-256 can be independently recomputed. Earlier local compiler and interoperability summaries are included as historical engineering evidence but are not silently presented as a fresh re-execution under this report.

2.2 Source immutability

The qualification branch commit is 07f32d8a18a91f7a9035994adcbecd234a09b5ac. The source archive digest is 2e64df12c4956c4f58cac1aa468a0765f21a05b23dc4f86a257d48ed15dbf45c. Every archived runner result repeats both values, and the consolidated reports mark source_uniform true. A reported common source_digest supplies a further cross-run content-consistency signal.

2.3 Ownership of the evidence

The examined code, automated test systems and primary qualification records are attributable to QRCS. The document is issued as a QRCS technical engineering assessment, with reproducible source identifiers and links that permit independent examination of the published evidence. It contains no claimed third-party opinion or laboratory signature beyond the underlying records.

2.4 Interpretive standard

A successful workflow is treated as evidence of the jobs and gates it actually implemented. A documented API is treated as an available interface, not a finding that the function is secure. An

observed failure-injection regression is treated as evidence of that gate's behavior, not as a universal proof of fail-closed behavior under all fault models. These distinctions prevent the common error of enlarging the scope of a result as it moves from engineering to marketing.

Chapter conclusion

Reproducible source identity, an explicit evidence hierarchy and consistent interpretation provide the basis for technical review, acquisition diligence and regulated procurement evaluation.

03 Commercial and Technical Decision Summary

A buyer evaluating the QSC library should distinguish intrinsic cryptographic functionality from integration infrastructure and from verification maturity. The source archive contains a broad set of primitives, TLS/X.509 components, utility subsystems, and hardware-provider interfaces. The verified test program demonstrates repeatability across platform families; the remaining certification pathway is a separate contractual and regulatory undertaking.

3.1 Breadth as an integration attribute

The same source base includes standardized symmetric algorithms, hash and XOF primitives, conventional and post-quantum public-key algorithms, TLS and certificate management, source-level hardware-provider interfaces, threading, networking and serialization infrastructure. A customer may not need all these modules, but the common library boundary can reduce the number of independent third-party interfaces it must maintain.

3.2 Evidence that reduces technical uncertainty

Passing native and cross-platform tests reduce the specific concern that an implementation works only for the development machine, only under one compiler, or only for one security-class build. The previously recorded local Linux campaign adds algorithm-focused differential, boundary and negative tests. The strongest claim is that the defined harness and source revision performed as recorded in these environments.

3.3 Hardware integration and optimized deployment as differentiators

The controlled source baseline implements a versioned hardware-key provider facade, backend adapters and TLS signing bridges, alongside a configurable CMake system with nine composition profiles and optional hardware-provider selection. These features address distinct integration requirements: externally held keys, cryptographic-service abstraction, reduced build composition, architecture-targeted compilation and installable package manifests. They represent concrete source functionality that can simplify product integration and technical evaluation. Physical device acceptance and profile-specific code-size/performance measurements remain separate qualification activities and are not assigned numerical results in this study.

3.4 Areas that cannot be priced as completed certification

Neither the official ACVP algorithm process nor the FIPS 140-3 module process is established by the materials reviewed. Physical HSM qualification, operational-key custody, all deployment-mode entropy provenance, and full third-party timing/side-channel evaluation are distinct evidence domains. A transaction should not book those outcomes as complete assets simply because the code exposes APIs for them.

3.5 Due-diligence transaction package

A defensible technical data room would include the pinned source archive, repository commit, consolidated JSON records, complete per-job evidence, source-license terms, an SBOM, test-harness

reproduction instructions, development controls, a vulnerability remediation ledger and a regulator-facing boundary diagram. The current package provides several important pieces and identifies the others explicitly.

Chapter conclusion

QSC demonstrates a substantial technical asset with an organized qualification discipline. Valuation and indemnity terms must rely on independently reviewed intellectual property, licensing and assurance evidence, not test pass counts alone.

04 Report Architecture and Reading Guide

This study is organized so that non-specialists can read the executive findings and decision analysis while cryptographic engineers can inspect methods, per-family evidence, platform variation, interfaces and the underlying source inventory. Every technical chapter uses the same conclusion discipline: what was examined, what was observed, and what follows from that observation.

4.1 Reading the technical chapters

The primitive chapters distinguish algorithm architecture from verified test coverage. For example, a header inventory can establish the presence of APIs for several hash families, while the offline harness can establish that selected SHA-2/HMAC and SHA-3/SHAKE legacy tests passed. Neither fact alone establishes every advertised construction or parameter mode as validated.

4.2 Understanding the matrices

A green platform cell means the corresponding job concluded successfully under the pinned source and security-class setting. The number of cells is a configuration count, not a count of unique algorithms or a performance metric. The laboratory-style evidence diagrams explain the acceptance chain and why source revision and missing-artifact gates matter.

4.3 Using the appendices

The final appendices contain a full lexical inventory of exported QSC-prefixed functions identified in the provided headers, a source-file catalog with line counts, the fifteen-runner manifest, legacy-suite success-marker accounting, definitions, and permanent source links. Names are included for technical traceability, not to imply that each was separately exercised in the CI program.

4.4 Scope of conclusions

The report remains a technical study of the supplied November 6 source and the archived qualification work. Subsequent repository changes are not automatically covered. References to NIST publications establish the relevant normative algorithm definitions and validation framework, a technical description of the applicable algorithms.

Chapter conclusion

The structure is designed to support a practical document review: a decision maker can assess the evidence position quickly, and an engineer can trace the underlying detail without depending on unverifiable narrative claims.

PART II

SOURCE AND ARCHITECTURE

Reproducible source inventory, exported interface topology, support components and portability assumptions.

EVIDENCE-LED TECHNICAL EXAMINATION

CHAPTER 05 | II. SOURCE AND ARCHITECTURE

05 Source Snapshot, File Composition and Engineering Footprint

The first reproducibility question is what body of code was actually examined. The supplied source snapshot contains 354 files totaling approximately 13.2 MB uncompressed. Its principal engineering substance resides in 176 C source files and 164 headers. The remaining entries include build configuration and associated project metadata.

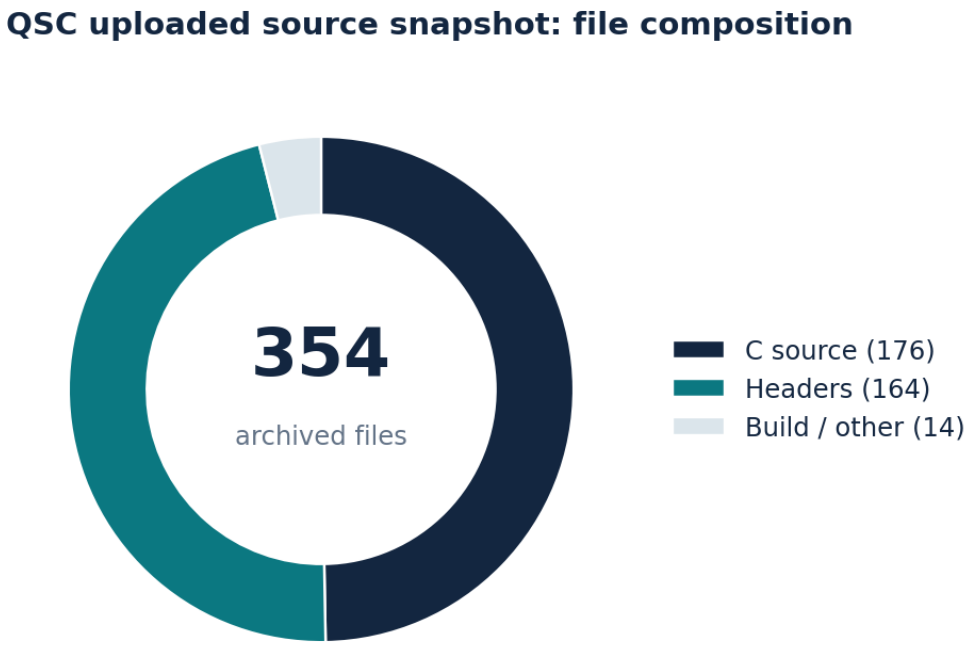


Figure 3. Composition of the controlled QSC implementation by source-file category.

5.1 Source inventory method

The file inventory was generated directly from the uploaded ZIP rather than inferred from repository navigation. The source archive was read without modifying its contents, each member name was recorded, and its type and line count were measured programmatically. The archive digest independently identifies the source used by the hosted qualification runners.

5.2 Size and interpretation

The extracted C and header files contain approximately 257,815 text lines, including licensing notices, documentation, tables and generated or specialized code. This is a raw textual inventory, not a claim about hand-written effective lines of code, development cost, algorithmic complexity or independently verified originality.

5.3 Component concentration

The larger export groups include X.509 wrapper functions, TLS sockets, socket base utilities, integer conversion, Keccak/SHA-3, Classical McEliece interfaces and SLH-DSA entry points. This distribution reflects both cryptographic algorithms and the operational infrastructure around them. It helps explain why library qualification cannot be reduced to a single symmetric benchmark.

5.4 Source provenance as a controlled variable

The cross-platform matrix uses one source archive digest and one qualification commit. That eliminates a major ambiguity in interpreting a mixed platform result: if the source files differed, a green Windows result and a green ARM result might not describe the same implementation. The source-identity gate is therefore a substantive assurance control, not administrative decoration.

Chapter conclusion

The evidence supports an identifiable and materially broad native-code implementation with reproducible provenance. The source size and API inventory are objective codebase attributes; they are not substitutes for independent security review.

CHAPTER 06 | II. SOURCE AND ARCHITECTURE

06 Public API Surface and Integration Contracts

A cryptographic library's operational risk is often concentrated at its public boundaries: buffer lengths, ownership rules, error handling, state initialization, callback lifetimes and algorithm selection. The reviewed headers expose a large API surface. This chapter evaluates its observable organization and the implications for integrators rather than claiming to have exhaustively audited all individual signatures.

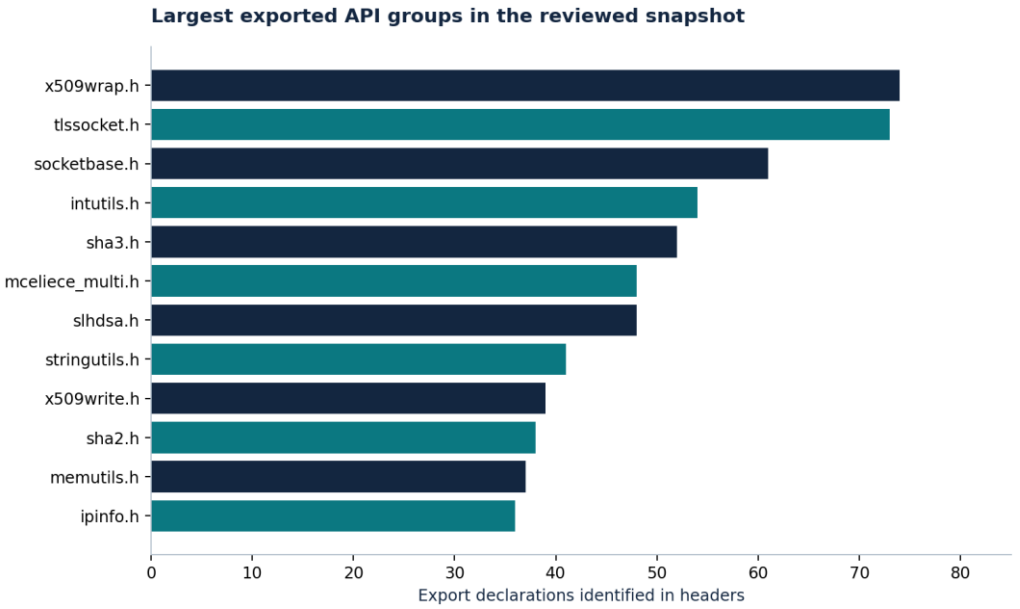


Figure 4. Exported QSC API declarations by major module group.

6.1 Measured public declarations

An automated scan of the `QSC_EXPORT_API` marker in 164 headers yields 1,846 distinct QSC-prefixed function names, across 134 headers containing at least one identified declaration. This is a lexical extraction, which may include function-like declarations not meant for a given binary configuration. It does not determine ABI compatibility across releases or establish that every declaration is linked into every profile.

6.2 Functional organization

Interfaces are visibly grouped by algorithm and service, such as AES, Keccak/SHA, ML-KEM parameter sets, ML-DSA, SLH-DSA, ECC, TLS, X.509, PKCS#11 and hardware providers. Separate headers for parameterized implementations facilitate targeted use; higher-level wrappers may simplify application integration. The existence of both layers means engineers should agree on which facade is supported for stable commercial use.

6.3 Integrating safely

A customer-facing API contract should explicitly define accepted key sizes, nonce uniqueness and tag lengths, input/output overlap behavior, failure-state transitions, buffer ownership and zeroization

responsibility. The header names alone do not establish those semantics. The most valuable diligence follow-up is a function-by-function mapping from public header to unit tests, not simply a count of exports.

6.4 Compatibility governance

The breadth of the API argues for exported-symbol checking, versioned interface reviews and a deprecation policy. For an acquisition, a clean inventory enables product teams to quantify integration reach and concentrate review on high-traffic code paths. For a regulated deployment, the approved-service list should be substantially narrower than the full development library.

Chapter conclusion

The public surface is broad and strongly organized into recognizable domains. Its commercial benefit is capability breadth; its engineering obligation is disciplined API stability and precise cryptographic usage contracts.

07 CMake Deployment Profiles, Source Selection and CPU Optimization

The controlled QSC implementation includes a dedicated CMake composition system for selecting cryptographic function families and hardware-provider adapters without editing existing cryptographic arithmetic. The nine implemented compilation profiles, explicit source inventory, CPU-target controls and installed capabilities manifest provide a concrete foundation for reproducible classical, post-quantum, hybrid and platform-specific deployment configurations.

7.1 The nine implemented composition profiles

QSCProfiles.cmake defines FULL, BOOT_VERIFY, DEVICE_IDENTITY, CRYPTO_CLASSICAL, CRYPTO_PQC3, CRYPTO_PQC5, CRYPTO_HYBRID, CLOUD_HSM and EDGE_GATEWAY. These are source-composition recipes: FULL includes the complete QSC source inventory, whereas compact families assemble explicitly listed support and algorithm translation units. CLOUD_HSM and EDGE_GATEWAY deliberately retain the complete TLS/X.509 dependency graph, so they should be understood as deployment-oriented full-crypto recipes rather than compact TLS implementations.

7.2 Compact classical and post-quantum compositions

BOOT_VERIFY selects SHA-2, SHA-3, core support code, ECDSA P-256 and ML-DSA-65 translation units for boot-image verification use cases. CRYPTO_CLASSICAL selects classical ECDSA P-256 and X25519 dependencies with hashing and AES; CRYPTO_PQC3 combines ML-KEM-768 with ML-DSA-65, and CRYPTO_PQC5 combines ML-KEM-1024 with ML-DSA-87. DEVICE_IDENTITY and CRYPTO_HYBRID include both classical and level-three post-quantum primitive groups. These profiles control which source files are linked; they do not alter the underlying implementations or promise a verification-only ECDSA arithmetic core.

7.3 Source inventory controls and reproducibility

QSCSources.cmake maintains an explicit list of implementation translation units and checks that every source on disk is classified. It also checks Visual Studio project source entries when the project file is present. This deliberate inventory prevents newly added implementation files from silently entering an optimized build. The compile-time selections can be captured with profile identity, toolchain revision, output digest and installed capabilities manifest.

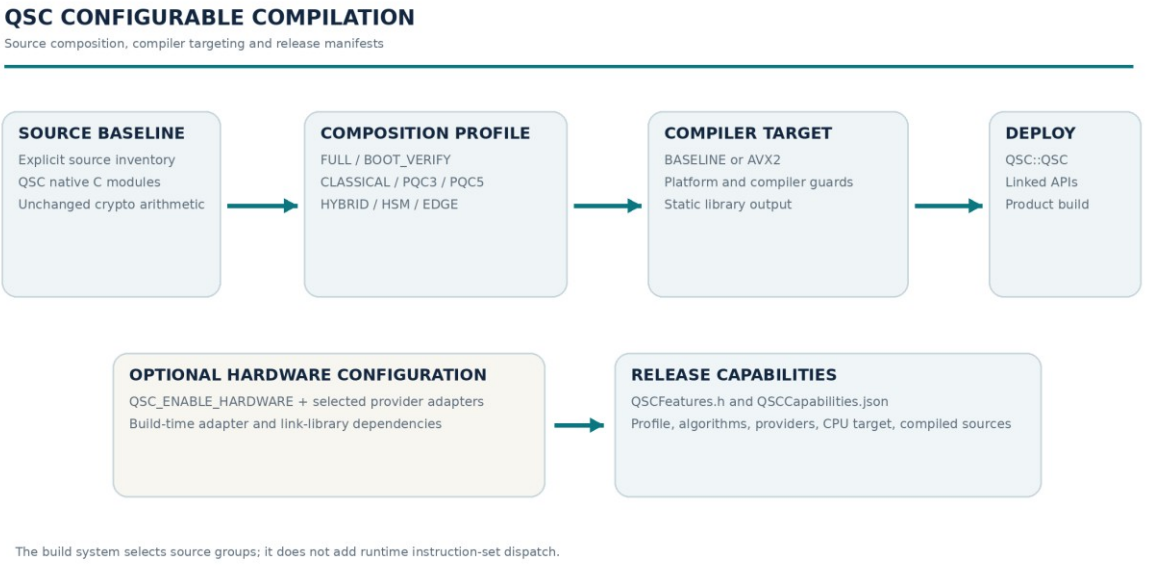


Figure 5. Controlled source selection, CPU targeting and recorded deployment capabilities in the QSC CMake system.

7.4 Architecture-specific CPU targeting

QSCCompiler.cmake exposes QSC_CPU_TARGET with BASELINE and AVX2 choices. The AVX2 setting adds /arch:AVX2 under MSVC or -mavx2 -maes -mpclmul for supported GCC/Clang x86 targets. The QSC source retains compile-time instruction-set guards; the CMake system does not perform runtime CPU dispatch. Consequently, an AVX2-targeted binary must be deployed only where the required CPU features are available. VAES and VPCLMULQDQ are separate capabilities and cannot be assumed merely from AVX2 or AES-NI support.

7.5 Build artifacts and consumer contracts

The supplied implementation produces static libraries and provides an installable CMake package through QSC::QSC. Generated QSCFeatures.h and QSCCapabilities.json artifacts record the selected profile, algorithm groups, hardware switch, provider names, CPU target and compiled source count. The unchanged public headers are installed for all profiles; consumers must use the capabilities manifest because a compact library may intentionally omit symbols declared in the full header set.

Table 7.1 Implemented source-composition profiles

QSC_PROFILE	Source group / intended role
FULL	Complete implementation inventory, including TLS and X.509 dependencies
BOOT_VERIFY	ECDSA P-256 and ML-DSA-65 verification-oriented composition
DEVICE_IDENTITY	Classical identity and PQC level-three primitive groups
CRYPTO_CLASSICAL	Classical X25519, ECDSA P-256, AES and hash dependencies
CRYPTO_PQC3	ML-KEM-768, ML-DSA-65, AES and SHA-3 dependencies
CRYPTO_PQC5	ML-KEM-1024, ML-DSA-87, AES and SHA-3 dependencies
CRYPTO_HYBRID	Combined classical and level-three PQC source groups
CLOUD_HSM	Full TLS/X.509 composition with optional hardware adapters
EDGE_GATEWAY	Full TLS/X.509 composition for gateway deployments

7.6 Optional hardware backends at build time

QSC_ENABLE_HARDWARE enables the backend-independent provider core, while QSC_HARDWARE_PROVIDERS selects PKCS11, NCRYPT, TPM, ATECC, SE050, PSA, PSA_NATIVE or PLATFORM adapters. The CMake logic validates platform restrictions and required SDKs: for example, NCRYPT requires Windows, TPM2-TSS integration requires Linux and tss2-esys/tctildr, and PSA_NATIVE requires the configured PSA SDK library and include directory. Hardware-enabled code selection is independent of whether a suitable device is attached at runtime.

7.7 Reproducible configuration examples

A full release library can be configured using QSC_PROFILE=FULL and CMAKE_BUILD_TYPE=Release. A focused post-quantum build uses QSC_PROFILE=CRYPTO_PQC3 or CRYPTO_PQC5. A hardware-enabled enterprise build can select QSC_PROFILE=CLOUD_HSM, QSC_ENABLE_HARDWARE=ON and QSC_HARDWARE_PROVIDERS=PKCS11. For every intended product configuration, the profile, CPU target and provider dependencies should be recorded alongside the resulting binary SHA-256, and the application should validate that its required exported symbols are present.

7.8 Qualification significance and actual evidence

The final fifteen-runner offline ACVP matrix establishes selected algorithm and adapter behavior under three security-class settings on native Windows, Linux and macOS targets. The separate historical Linux x86-64 campaign exercised scalar, AVX2 and AVX-512-enabled compiler profiles and instrumented Clang builds. Those results support portability and cryptographic execution for the stated configurations; they are not profile-by-profile code-footprint or performance benchmarks of the newer CMake composition system.

7.9 Deployment economics and engineering controls

Selective source composition can reduce integration dependencies and focus a product on its required cryptographic services. Embedded firmware, boot chains, device identities, secure gateways and enterprise HSM-integrated applications have materially different key-management and compiler requirements. Quantitative size, start-up time, memory, throughput and energy comparisons should be measured on target hardware before they are included in a commercial product claim. Profile selection also changes the service boundary, so each released binary requires its own regression and configuration record.

Chapter conclusion

QSC provides nine explicit CMake composition profiles, controlled CPU targeting, optional hardware-provider integration and installed capability manifests. The system gives product teams concrete tools for repeatable classical, post-quantum and hybrid deployments; the report assigns qualification findings only to combinations actually exercised.

08 System Support: Memory, Integer, Concurrency and Networking

Cryptographic implementations depend on non-cryptographic support code for state, parsing, file operations, memory handling and timing. In the reviewed library these facilities are first-class modules. They affect memory safety, developer experience and the scope of any future cryptographic-module boundary.

8.1 Core utilities

The archive exposes `memutils.h` and `intutils.h` among the large public utility groups. Related headers support arrays, strings, collections, queues, encoding, files and system information. Preferential use of the project's audited utility conventions can reduce behavioral differences across platforms, but each function's actual implementation and documented contract still require review.

8.2 Threading and event interfaces

Separate modules support asynchronous operation, thread pools, sockets, socket clients and servers. Those capabilities are valuable for deployment, but they introduce concurrency concerns that do not arise in a single-threaded known-answer test. Thread-safe behavior, callback reentrancy, cancellation and error propagation need explicit treatment when the library is embedded in a network-facing service.

8.3 Boundary classification

For FIPS module planning, an engineering team must decide which support code lies inside the cryptographic boundary. A narrow primitive-only boundary may reduce evaluation scope, while an integrated TLS or hardware-provider boundary may better reflect product operation but require more operational controls. The present test results do not themselves make that architectural decision.

8.4 Integration risk

Utility and networking entry points tend to be used pervasively. Their broad exports are a reason to invest in static checks, memory sanitization, deterministic test fixtures and stable contracts. Tests specific to cryptographic vectors provide only indirect evidence about parser and network subsystem safety.

Chapter conclusion

The support layer is a relevant contributor to QSC's integration value and codebase complexity. Its inclusion in a validated module should be deliberate, documented and evidence-backed.

PART III

PRIMITIVE TECHNICAL EXAMINATION

Cryptographic algorithm families, API roles, relevant test results and clearly bounded assurance findings.

EVIDENCE-LED TECHNICAL EXAMINATION

CHAPTER 09 | III. PRIMITIVE TECHNICAL EXAMINATION

09 Advanced Encryption Standard: Modes and Operational Safety

AES is a core cryptographic primitive in the source snapshot. The reviewed header exposes multiple block and authenticated modes, while the hosted offline harness records an eight-marker AES legacy suite and additional AES adapter and interoperability gates. The historical Linux campaign also exercised long-message GCM cases and rejection behavior.

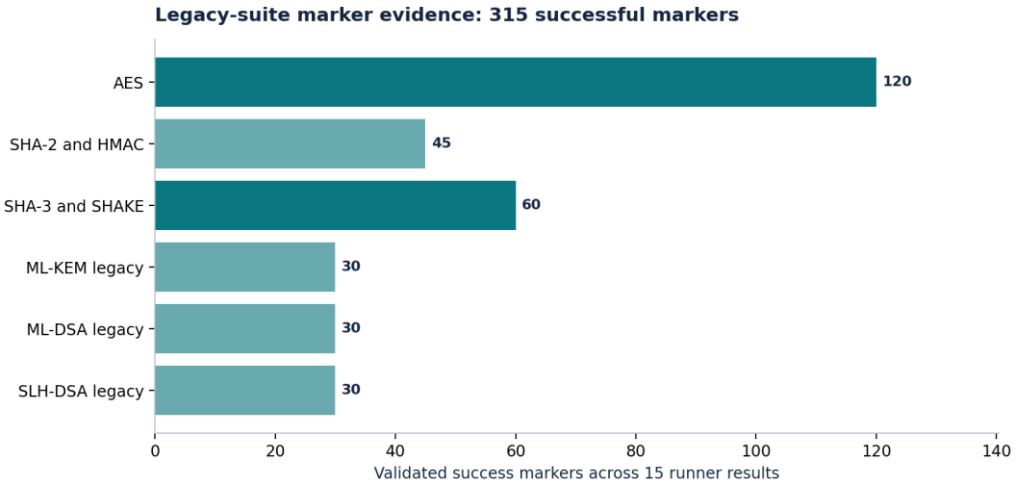


Figure 6. Accepted legacy algorithm-suite markers in each native qualification configuration.

9.1 Implementation organization

The AES source implements symmetric encryption functions and associated state management. The presence of ECB, CBC, CTR and GCM interfaces supports different protocol roles, but modes are not interchangeable from a security perspective. ECB lacks semantic protection, CBC needs correct padding and authentication at a higher level, CTR requires non-reused counters, and GCM must maintain nonce uniqueness and authenticate associated data.

9.2 Correctness evidence

The archived offline matrix reports AES suite PASS on all fifteen configurations, with eight legacy success markers per result. The accompanying adapter and interoperability stage fields are true in each record. In the earlier five-profile local study, 590 long-message AES-GCM assertions and 52 AAD-boundary assertions passed per profile. Those historical totals describe a different archived-source local campaign and are not added to the final fifteen-runner marker count.

9.3 Rejection behavior

Authenticated-decryption handling must never release unauthenticated plaintext as a successful operation. The recorded GCM AAD and CBC negative tests indicate deliberate boundary exercise in the local study; they do not prove all fault-injection or timing cases. The most useful production tests

should include truncated tags, malformed lengths, overlapping buffers, repeated IV policy and failed finalization semantics.

9.4 Accelerated paths

AES-NI and vectorized paths can materially affect performance and implementation structure. Separate acceleration-profile tests reduce the risk that an intrinsic mode was introduced without cryptographic regression checks. This report does not assign a speedup or throughput figure because current-source, independent performance measurements are not supplied in the consolidated CI evidence.

Chapter conclusion

The AES testing record is positive across the specified modes and platforms. The correct external claim is demonstrated selected AES behavior under the offline harness, not universal GCM parameter validation or a certificate.

10 SHA-2 and HMAC: Digest and Authentication Foundations

SHA-2 and HMAC are foundational dependencies for key derivation, signatures, protocol negotiation and integrity checks. The source includes a dedicated SHA-2 module and the ACVP evidence reports repeated SHA-2/HMAC success across the native qualification matrix.

10.1 Hash construction boundaries

The library distinguishes digest generation from keyed message authentication. Hash-collision resistance, HMAC unforgeability and output-length considerations apply differently; using a bare digest as an authentication tag is not equivalent to HMAC. Separate public APIs for SHA variants and keyed operations help application authors maintain this separation.

10.2 Hosted test evidence

Each of the fifteen consolidated runner records reports the SHA-2/HMAC legacy suite PASS with three success markers. The broader workflow includes SHA/HMAC simulation or adapter gates in addition to these legacy markers. A marker count is a harness-reporting quantity, not a number of separate NIST vector sets.

10.3 Local differential testing

The earlier Linux x86-64 campaign recorded 1,935 SHA-2/HMAC streaming checks per compiler configuration. Differential streaming exercises incremental update/finalization sequences, an important distinction from a one-shot digest. The local study was reported with five different compilation profiles; its raw logs are not included in the two current consolidated CI ZIPs.

10.4 Implementation diligence

The security of a hash implementation depends on bit ordering, message padding, length arithmetic, state reset and API contracts around truncation. For HMAC, key normalization and error handling are additional concerns. Public documentation should indicate whether an output length is fixed, configurable or truncated, and how the implementation handles invalid key and message sizes.

Chapter conclusion

The combination of a mature algorithm family, streaming-oriented historical tests and repeated cross-platform offline success supplies meaningful engineering assurance for the tested SHA-2/HMAC interfaces, within the bounds of the available evidence.

11 SHA-3, SHAKE and Keccak State Discipline

The Keccak family forms a critical common dependency for modern post-quantum algorithms and several proprietary constructions. The supplied `sha3.h` exposes one of the largest cryptographic API groups, making state lifecycle, domain separation and input-length handling high-value review targets.

11.1 Primitive architecture

SHA-3 fixed-output hashing and SHAKE extendable-output functions share a sponge foundation but use different domain separation and output contracts. A sound API must control absorb/finalize/squeeze transitions and reject invalid sequencing. The presence of related Keccak state operations in the header indicates an implementation that supports both higher-level convenience functions and lower-level sponge usage.

11.2 Observed platform evidence

Every hosted runner records the SHA-3/SHAKE legacy suite PASS with four success markers. In the prior local five-profile campaign, 5,200 SHA-3/SHAKE streaming assertions per profile passed. These tests are relevant because Keccak boundary defects frequently appear around rate transitions, padding and multi-block squeezing rather than in a single short example.

11.3 Role in post-quantum systems

SHAKE and SHA-3 underpin the implemented ML-KEM, ML-DSA and SLH-DSA families in different roles. Shared hash-platform correctness is necessary for those systems, but passing SHAKE tests does not by itself establish the correctness of a KEM decapsulation path or signature parser. This is why the qualification campaign maintains separate post-quantum adapters.

11.4 Assurance limitations

The offline engineering report expressly limits the cSHAKE/KMAC result to the tested AFT scope and does not assert full server-side NIST parameter validation. That limitation is not an indication of failure; it prevents a partial output-domain test from being represented as every bit-length and customization-string case.

Chapter conclusion

The Keccak/XOF evidence is consistent and technically important. Selected cross-platform hashing and squeezing behavior is demonstrated, with full ACVP server scope treated as a separate validation activity.

CHAPTER 12 | III. PRIMITIVE TECHNICAL EXAMINATION

12 KMAC, cSHAKE, HKDF, PBKDF and KDF Interfaces

Key derivation and keyed sponge constructions are easy to misuse even when their underlying hash primitives are correct. This examination treats KMAC/cSHAKE, HKDF, PBKDF and KDA integration as independent API contracts requiring parameter, context and revision discipline.

12.1 KMAC and customization

KMAC uses Keccak-based keyed processing and encodes key, customization and output-length information. Correctness depends on unambiguous encodings and domain separation. The recorded offline XOF/KMAC stage passed across the matrix, while the earlier local study reports 216 independent KMAC comparisons per profile. That is useful engineering evidence but not a substitute for the full set of ACVP variants.

12.2 HKDF and extract/expand

HKDF is a composition of extract and expand steps with information and salt inputs. Boundaries involving zero-length inputs, maximum output length and hash-dependent blocks must be honored. Earlier local tests recorded 48 HKDF boundary checks per profile. The hosted KDF stage includes PBKDF/HKDF/KDA adapter work and was a key focus of the Stage 3 regression repairs.

12.3 Revision-aware KDA coverage

The harness originally conflated KDA-HKDF `Sp800-56Cr1` and `Sp800-56Cr2` registrations, which led to a duplicate capability key. The correction incorporated revision into the capability identity, preserving strict coverage without weakening checks. This example illustrates how a test harness can fail for bookkeeping reasons even when a primitive has returned correct cryptographic outputs.

12.4 Commercial consequence

Enterprise users often care more about correct key lifecycle than raw hash throughput. The same function family can support storage encryption, tunnel key establishment, credential processing or protocol transcript derivation, each with different context-uniqueness requirements. Product security guidance should document salt, personalization, counter and context inputs for every supported usage profile.

Chapter conclusion

The qualification demonstrates selected KDF and keyed-sponge behavior, including repair of revision-specific harness accounting. It does not claim all NIST KDA server-side combinations are certified.

13 ChaCha20, Poly1305 and Stream-Authenticated Processing

The archive contains dedicated `chacha.c/.h` and `poly1305.c/.h` implementations. Their presence is relevant to protocols and constrained environments that use stream encryption and polynomial authentication rather than hardware-accelerated AES. The present consolidated ACVP evidence, however, does not separately certify a ChaCha20-Poly1305 configuration.

13.1 Algorithmic roles

ChaCha20 is a stream cipher whose security depends on nonce/counter uniqueness and correct key setup. Poly1305 is a one-time authenticator; safe compositions derive or allocate one-time MAC keys and authenticate the specified associated data, ciphertext and length fields. These are protocol-level contracts, not interchangeable generic buffer transforms.

13.2 Source evidence

Separate source/header modules confirm that code for both primitives is supplied. Public entry points expose initialization, transformation and authentication operations, and their API names appear in the inventory. This source finding is distinct from the fifteen-runner AES, SHA and PQC suite results, which do not record dedicated ChaCha success-marker counts.

13.3 Engineering assessment

Performance tuning, scalar fallbacks and vectorized operations can affect side-channel behavior as well as speed. The archived engineering work records optimization and benchmark work, but no current-source primary benchmark artifact for ChaCha20-Poly1305 is part of the reviewed consolidated ZIPs. It would be misleading to convert an earlier anecdotal throughput figure into a laboratory-validated cross-platform comparison.

13.4 Deployment discipline

When exposed to developers, APIs should define authenticated-decryption failure handling, nonce-length policies, per-record counter management and buffer overlap. A cryptographic suite should include RFC vectors, tampered-ciphertext rejection and counter-exhaustion tests. Those are appropriate diligence questions, not demonstrated full-scope results of this publication.

Chapter conclusion

The source establishes a meaningful ChaCha/Poly1305 capability. The current qualification record is strongest for the algorithms explicitly represented in the offline suite, so no separate certification claim is assigned to this family.

14 QRCS-Specific RCS, CSX and Related Constructions

The source includes QRCS-specific cryptographic components such as RCS and CSX alongside widely standardized primitives. A diligence review must distinguish the existence and integration of proprietary constructions from an independent security conclusion about their cryptanalytic resistance.

14.1 Source-level differentiation

Files including `rcs.c/.h` and `csx.c/.h` demonstrate maintained implementation families outside conventional AES and SHA API surfaces. These modules may serve products with requirements that differ from a standard AEAD or stream cipher. Their proprietary engineering value depends on useful implementation characteristics, explicit specifications and reliable deployment contracts.

14.2 Assurance standard

Novel constructions call for independent design review, formal treatment of nonce and key reuse, test vectors, authenticated-data misuse analysis, implementation-level side-channel assessment and ongoing cryptanalytic evaluation. Passing a standard AES or SHA test suite does not automatically qualify an RCS or CSX construction, even where those functions use the same underlying utility code.

14.3 Source and performance evidence

Historical internal engineering reports include selected RCS/CSX throughput measurements and accelerated/scalar comparisons under earlier compiler and source conditions. The final source-pinned qualification records provide functional and adapter evidence; they do not contain a reproducible, current-revision comparative benchmark series. Accordingly, no headline throughput advantage is assigned to the final implementation.

14.4 Investor relevance

These constructions may represent differentiated intellectual property, but patent scope, ownership, license clearance, cryptanalytic scrutiny and interoperability adoption must be assessed independently. A technical acquirer should value the actual implementation and supporting specifications while separately examining whether a specialized primitive is necessary for the intended product line.

Chapter conclusion

The archive confirms QSC-specific cryptographic engineering. A defensible commercial claim is the existence of implemented proprietary capability, not a universal cryptanalytic guarantee or a formal NIST approval.

15 Elliptic-Curve Cryptography: P-256, P-384 and P-521

The archive contains dedicated ECDH and ECDSA implementations for NIST prime-field curves P-256, P-384 and P-521. The historical development program emphasized correctness, negative-input behavior, compiled-path comparison, performance and timing discipline. The final hosted matrix includes ECC interoperability and ECC ACVP adapter gates.

15.1 Implementation separation

The structure includes `ecdhp256`, `ecdhp384`, `ecdhp521`, `ecdsap256`, `ecdsap384` and `ecdsap521` modules, with facade headers for higher-level ECC use. This arrangement is useful for controlling curve-specific arithmetic and serialization but also increases the number of code paths that must be tested after a security or performance change.

15.2 Correctness and interoperation

The archived fifteen-runner results report ECC interoperability and ECC ACVP adapter flags as passing. Earlier local negative-matrix work recorded 213 ECDSA input-rejection checks per x64 compiler profile. Rejection tests matter because malformed public keys, invalid scalars and incorrect signature encodings often reveal logic defects that a valid-signature happy-path test would not detect.

15.3 Determinism and side channels

ECDSA signature generation may use randomized or deterministic nonce derivation. Both need strict treatment of nonce secrecy and scalar timing. Constant-time arithmetic, fixed-base precomputation and point validation are source-level implementation concerns that cannot be inferred solely from successful output comparisons. The NIST Demo adapter's registration for deterministic signing must be matched to the exact supported mode.

15.4 Implementation lineage and regression discipline

The P-384 and P-521 implementation program involved arithmetic translation, fixed-generator tables and hardening work. The completed offline ECC targets demonstrate selected current-snapshot execution under the hosted qualification matrix. Machine-code equivalence, provenance and full side-channel properties remain matters for separately documented engineering analysis.

Chapter conclusion

P-256/P-384/P-521 support and their hosted offline ECC qualification are substantial technical results. Cryptographic robustness additionally depends on key validation, fault response and constant-time implementation analysis beyond the recorded CI PASS.

16 Edwards-Curve Signatures and Diffie-Hellman

The source archive includes Ed25519/Ed448 related arithmetic and higher-level EDDH/EdDSA modules. These primitives are important for protocols seeking compact keys, predictable operations and well-defined signature or key-establishment semantics.

16.1 Module inventory

Files include `ed25519`, `ed448`, `eddh25519`, `eddh448`, `eddsa25519`, `eddsa448` and associated facades. This establishes implementation coverage of two Edwards/Montgomery families at the source level. It does not imply every file is enabled or exposed in every compilation profile.

16.2 Protocol behavior

For EDDH, public-key validation and rejection of small-order or non-contributory results are significant implementation requirements. For EdDSA, signature malleability, message prehash/context variants and strict point encoding deserve attention. These cases should be tied to explicit public APIs and test vectors rather than assumed to be covered by a generic ECC PASS.

16.3 Offline evidence

The hosted matrix reports ECC ACVP adapter and ECC interoperability stage success; the evidence package does not include a distinct per-curve count of every Ed448 or Curve448 test vector in its consolidated summary. A technically accurate report therefore associates the family with the broader ECC adapter evidence while withholding a claim of exhaustive Edwards-variant validation.

16.4 Portability consideration

Multiprecision arithmetic and limb representation are sensitive to word size, endian interpretation and compiler arithmetic shifts. ARM64/macOS execution is valuable because it exercises compiler/architecture combinations different from the development workstation. Additional control-flow and timing experiments are required if the library seeks a formal constant-time assurance claim.

Chapter conclusion

The source demonstrates both Edwards and Montgomery families, with selected ECC-related offline validation. Full per-variant ACVP coverage and side-channel assurance remain distinct properties.

17 ML-KEM and FIPS 203 Parameter-Set Coverage

ML-KEM is a principal standardized post-quantum key-encapsulation family. The uploaded archive includes parameter-specific implementations for ML-KEM-512, ML-KEM-768 and ML-KEM-1024, together with shared Kyber-related code. The corresponding offline adapter and legacy suite were successfully exercised across the fifteen configurations.

17.1 Parameter-specific structure

Separate `kyber512`, `kyber768` and `kyber1024` source/header pairs expose keypair, encapsulation and decapsulation APIs. The archive also contains a `kyber1280` implementation; this additional source should not be described as one of the three FIPS 203 parameter sets. The presence of family-level and parameter-level APIs provides useful flexibility but requires explicit algorithm identifiers in product protocols.

17.2 Offline execution evidence

Each consolidated runner reports the ML-KEM legacy interface PASS with two success markers and a passing ML-KEM adapter stage. The Stage 3 coverage gate was designed to prevent a registration from passing merely because one vector set appeared: key generation, encapsulation and decapsulation at each registered parameter set are distinct capabilities. The repaired matrix passed with this stricter accounting.

17.3 Failure behavior

Key decapsulation is security-sensitive because malformed ciphertext must not become an oracle revealing secret-key information. The correctness of a valid decapsulation is necessary but insufficient. Testing should include invalid ciphertext, secret-key corruption policy, deterministic seeded KAT generation and constant-time comparisons where the implementation claims them.

17.4 Commercial relevance

ML-KEM support is directly relevant to post-quantum migration of session key establishment, hybrid key exchange, transport protocols and embedded devices. A purchaser should ask whether the product uses one of the NIST parameter sets, how shared secrets are incorporated into a key schedule, and whether public and secret key serialization follows the intended FIPS 203 representation.

Chapter conclusion

The source and offline CI provide strong implementation-specific evidence of ML-KEM functionality at the registered parameter sets. Official ACVP certification and any module-level approval are separate outcomes.

18 ML-DSA and FIPS 204 Signature Assurance

ML-DSA provides the standardized lattice-based digital signature capability in the reviewed implementation. The uploaded archive supplies the 44, 65 and 87 parameter families, with public APIs for key generation, signing and verification. The hosted offline qualification includes both legacy success markers and an ML-DSA/SLH-DSA adapter stage.

18.1 Algorithm interface

The `mldsa.h` header groups the parameter-specific interfaces, and dedicated `.c` files implement each supported set. Stable key-size, signature-size and encoding guarantees are central when these functions are used in certificates, signed software and mutually authenticated application protocols.

18.2 Registration granularity

The Stage 3 test program distinguished keyGen, sigGen and sigVer for each registered ML-DSA parameter set. It rejected an unsupported randomized-signing registration and required advertised pure-signature, external-interface and key-format attributes to match the implementation. This is a valuable control against claiming capabilities that a live server would never exercise correctly.

18.3 Evidence level

Each final runner marks the ML-DSA legacy suite PASS with two success markers; the combined PQ signature adapter stage also passed. These results give source-specific confidence in the tested implementation paths. The consolidated record does not claim an issued NIST ML-DSA algorithm validation number.

18.4 Threat model

Lattice-based signatures are sensitive to distribution sampling, secret-dependent rejection behavior, key parsing and randomness or deterministic seed handling. Constant-time claims and resistance to physical leakage require more than KAT matching. Product protocols must also address how certificates advertise ML-DSA signature identifiers and whether consumers understand the larger public keys and signatures.

Chapter conclusion

ML-DSA source and selected offline execution are established across the native CI matrix. The recorded tests are an engineering qualification, bounded by its defined test and deployment configuration.

19 SLH-DSA SHAKE Variants and FIPS 205

SLH-DSA is a standardized stateless hash-based signature family. The controlled source contains six SHAKE-based implementations: 128s, 128f, 192s, 192f, 256s and 256f. Their differing signing-time and signature-size tradeoffs matter to infrastructure products as well as constrained clients.

19.1 Variant taxonomy

The `s` and `f` designations correspond to different parameter tradeoffs within the named security-strength levels. The source contains dedicated compilation units per SHAKE variant, plus shared implementation includes. A deployment should choose a variant according to signature size, processing cost, verification budget and protocol constraints, not merely by key strength.

19.2 Legacy and adapter testing

Every hosted configuration records the SLH-DSA legacy suite PASS with two success markers, together with a passing combined PQ signature adapter stage. The Stage 3 capability gate represented each variant and the keyGen/sigGen/sigVer modes separately. This matters because a single 128s signature check would not support a claim about 256f verification.

19.3 Historical independent-interoperation evidence

The earlier local Linux x64 program recorded 42 SLH-DSA/OpenSSL interoperability checks per profile. The sanitizer build initially interrupted the final 256s tests; the two remaining checks were executed separately and passed. That interruption/resumption was documented rather than counted as one uninterrupted test process.

19.4 Deployment constraints

Hash-based signatures avoid lattice arithmetic but introduce substantial serialization and compute costs for some variants. For TLS, X.509 and firmware signing, these costs must be assessed in actual product message sizes and trust-store behavior. The hosted ACVP offline result by itself does not determine which variant is best for a given product.

Chapter conclusion

The evidence supports six SHAKE-based SLH-DSA implementations with selected offline and historical interoperation success. No untested SHA2-based SLH-DSA family or external validation status is implied.

20 Additional Post-Quantum Interfaces: HQC and Classic McEliece

The QSC archive extends beyond the three original NIST PQC FIPS families. Headers such as `hqc_multi.h` and `mceliece_multi.h` identify code paths for additional post-quantum public-key techniques. The presence of these interfaces is a source-inventory finding, not a claim that their entire algorithm suites were exercised by the fifteen-job ACVP matrix.

20.1 Algorithm diversity

HQC and Classic McEliece use substantially different mathematical assumptions from lattice-based ML-KEM. Supporting multiple families may be valuable for research, algorithm agility and long-term product planning. It also expands the maintenance, test-vector, licensing and side-channel review burdens.

20.2 Source organization

The exported interface inventory identifies multiple parameter-specific names in these families. The `mceliece_multi.h` group is among the larger public headers by export count. This documentation indicates a broad interface footprint but does not establish which parameter sets are enabled under a particular compile-time profile.

20.3 Qualification boundaries

The hosted consolidated report enumerates AES, SHA, ML-KEM, ML-DSA and SLH-DSA legacy suites, plus ECC/PQC adapter gates. It does not declare a dedicated comprehensive HQC or Classic McEliece NIST validation result. Accordingly, this report lists them as implemented source capabilities under study, not as certified or fully tested by the reported matrix.

20.4 Technical diligence

Code-based systems demand attention to large keys, memory allocation patterns, decryption/decapsulation failure policy and constant-time decoding. A serious acquisition review should require independent KATs, invalid-input tests, compression/serialization review and licensing analysis for each family selected for commercial deployment.

Chapter conclusion

The additional interfaces strengthen the breadth of the software asset but must be evaluated according to their own algorithm specifications and evidence. They are not subsumed by the ML-KEM qualification record.

PART IV

PROTOCOL AND PLATFORM INTEGRATION

How primitives interact with TLS, PKI, hardware providers, entropy, networking and native deployment.

EVIDENCE-LED TECHNICAL EXAMINATION

21 TLS 1.3, Key Schedule and Cryptographic Binding

QSC includes a substantial TLS-oriented subsystem: engine and handshake modules, key schedule, record processing, certificate exchange, socket integration, extensions and signature-scheme selection. This increases the library's utility for secure communications, but it creates assurance requirements beyond stand-alone primitive testing.

21.1 Protocol composition

TLS security depends on correct orchestration of key exchange, transcript hashes, key derivation, cipher-suite negotiation, authentication and record protection. Correct AES or SHA functions are prerequisites, yet they do not prove a complete handshake state machine is secure. The archive provides TLS-specific source and headers that deserve their own protocol conformance and adversarial test program.

21.2 Implementation evidence

Files include `tlsengine`, `tlshandshake`, `tlskeyschedule`, `tlsrecord`, `tlsextensions`, `tlssigalgs`, `tlscert` and client/server facilities. The public API inventory confirms configuration and state-management entry points. This is source-level evidence of integrated TLS functionality, not evidence that every TLS 1.3 interoperation or negative handshake scenario was executed in the ACVP suite.

21.3 Crypto agility

A post-quantum transport product benefits from explicit algorithm identification and interchangeable key-exchange/signature interfaces. Migration must preserve downgrade prevention, transcript binding and certificate compatibility. Technical due diligence should inspect negotiation transcripts, failure paths and software-update behavior for each advertised mode.

21.4 Boundary and product fit

FIPS module approval generally applies to defined cryptographic services and module boundaries, not to an entire transport protocol merely because its primitives are approved. Customers should ask which TLS operations are inside any prospective module boundary and how non-approved algorithms or diagnostic modes are isolated.

Chapter conclusion

The TLS subsystem is an important integration asset. The documented offline primitive and adapter successes support its foundations but do not replace protocol-level security or interoperability qualification.

22 X.509, ASN.1, Certificate Lifecycle and PKI Components

The source includes broad X.509 and ASN.1 facilities for certificate parsing, verification, writing, signing, private keys, certificate requests and revocation structures. This portion of the API accounts for a substantial fraction of the exported surface and is commercially relevant to identity, device provisioning and secure endpoint authentication.

22.1 Functional scope

Headers such as `x509wrap.h`, `x509write.h`, `x509certwrite.h`, `x509verify.h`, `x509csr.h`, `x509key.h`, `x509spki.h` and `asn1.h` describe decoding, building, validation and serialization APIs. Support for these operations may reduce an application's need to blend unrelated PKI stacks with cryptographic algorithms.

22.2 Parsing risks

Certificate and ASN.1 processing is security-critical because input comes from untrusted parties. Length encodings, nested structures, OID matching, signature algorithm identifiers, critical extensions, name constraints, path construction and key usage enforcement are frequent defect classes. A primitive KAT does not validate these parser semantics.

22.3 Lifecycle requirements

Commercial PKI deployment requires more than signing a certificate: key usage rules, revocation freshness, trust-anchor storage, renewal, certificate policy and identity binding must be explicit. Hardware keys may change the private-key handling model, while post-quantum signatures can change certificate and handshake sizes.

22.4 Evidence interpretation

The provided consolidated ACVP package does not include a complete independent X.509 compliance corpus or third-party parser fuzzing campaign. The correct statement is that the source contains extensive PKI implementation APIs, with cryptographic components covered by selected offline tests. A purchaser should require a separate certificate-path-validation report for PKI-heavy use cases.

Chapter conclusion

QSC's PKI surface is substantial and relevant to enterprise deployment. Security claims about path validation and parser robustness must rest on dedicated evidence, not inferred from algorithm success markers.

23 Hardware Key Services, Provider Contracts and Platform Integration

QSC includes a backend-independent, versioned hardware-key provider interface backed by optional platform and device adapters. It distinguishes key references and policy metadata from private-key bytes, and makes signing, public-key retrieval, agreement, random generation and attestation explicit provider operations. Its architecture is relevant to HSM-backed enterprise services, TPM-based device identities, certificate workflows and embedded secure elements.

23.1 Versioned provider interface and ownership model

hwprovider.h declares a versioned qsc_hw_provider_ops callback table with mandatory capability discovery and transient-handle release operations. A backend context remains caller-owned, while the QSC wrapper holds a validated local copy of the callback descriptor. Opaque provider and key-reference objects separate application requests from vendor handles. Key references include current provider-generation information, so device reconnection can invalidate stale references.

23.2 Key custody, usage policy and approved operations

Opening or generating a key requires a policy describing expected algorithm, permitted usage and custody properties. The interface returns a transient reference and public metadata, rather than private key material. This model supports provider-mediated signing and key management with explicit rejection of unsupported operations. The actual non-exportability property must be enforced by the selected backend and its hardware security policy.

23.3 Public APIs and operation boundaries

The exported facade provides qsc_hw_provider_open, qsc_hw_provider_capabilities, qsc_hw_key_open, qsc_hw_key_generate, qsc_hw_key_get_info, qsc_hw_key_public, qsc_hw_key_sign, qsc_hw_key_agree, qsc_hw_key_attest, qsc_hw_provider_random, qsc_hw_provider_invalidate, qsc_hw_key_release and qsc_hw_provider_close. Function-specific ownership, lengths and failure conditions are documented in the source headers. Key agreement explicitly returns an exportable derived secret, so this interface alone does not assert that the derived secret remains in hardware.

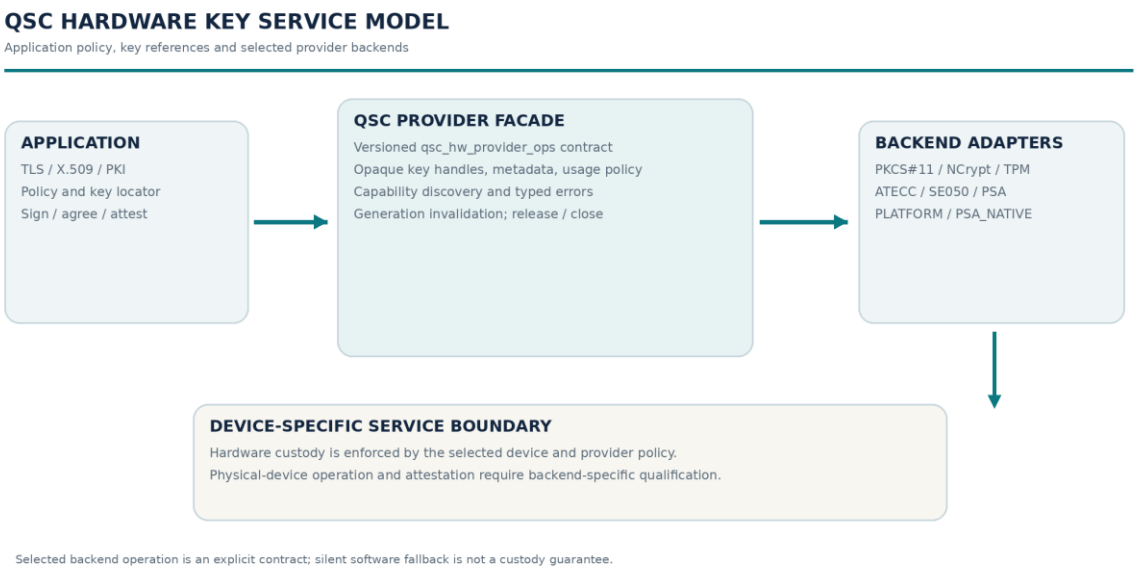


Figure 7. QSC hardware-key facade and adapter boundaries; key custody is enforced by the selected backend.

23.4 Supported backend integration paths

QSCHardware.cmake enumerates eight selectable backend names: PKCS11, NCRYPT, TPM, ATECC, SE050, PSA, PSA_NATIVE and PLATFORM. The Windows NCrypt adapter uses the platform cryptographic key APIs; Linux TPM integration uses TPM2-TSS; PKCS#11 adapters use provider sessions and token object references; ATECC and SE050 address embedded secure-element integration; and PSA routes through an application or native vendor SDK. Availability of an adapter does not imply a compatible device is present.

Table 23.1 Hardware-provider build options and platform dependencies

Provider	Build integration boundary
PKCS11	Token sessions and key operations through PKCS#11
NCRYPT	Windows only; ncrypt and bcrypt platform libraries
TPM	Linux; TPM2-TSS esys and tctildr dependencies
ATECC	Embedded ATECC backend adapter
SE050	Secure-element adapter using PKCS#11 support modules
PSA	PSA provider adapter
PSA_NATIVE	Supplier SDK includes and library required
PLATFORM	Platform provider adapter

23.5 PKCS#11 session and TLS bridge design

The archive includes p11loader, p11session, p11keys, p11sign and p11agree units, together with hwpkcs11tls bindings. Those units provide a modular route from key storage and session management to TLS signing callbacks. The implementation separates key handle selection from TLS signature-scheme handling and certificate identity association. The full, cloud-HSM and edge-gateway build profiles select TLS bridges when the corresponding provider is enabled.

23.6 Windows NCrypt and native platform services

hwncrypt.c and hwncrypttls.c expose Windows key-provider integration and TLS signature callbacks. The CMake backend requests the Windows ncrypt and bcrypt libraries, and its platform guard prevents

NCrypt selection on non-Windows builds. This arrangement enables product architectures that use operating-system-managed keys and associated certificate identities rather than placing an application-owned private-key copy in process memory.

23.7 TPM, attestation and embedded device identity

The Linux TPM adapter and quote-related implementation introduce hardware trust signals that are distinct from normal signing operations. qsc_hw_key_attest can retrieve a device-specific evidence envelope for a challenge, while independent attestation verification and a valid trust chain remain essential to any relying-party decision. ATECC and SE050 options address device-scale secure elements; each requires its own provisioning, key-identity and firmware-compatibility checks.

23.8 Lifecycle and fail-closed engineering

Device sessions can expire, a token can be removed, or a provider may return unsupported-algorithm, buffer-capacity or stale-handle errors. The facade defines typed status returns, policy checking and provider invalidation. Device-specific adapters must preserve failure atomicity, enforce custody requirements and prevent silent software substitution for keys designated as hardware-owned. Provider closures and concurrent operations require explicit caller/backend synchronization as documented in the header contract.

23.9 Hardware qualification method and evidence class

Software mock tests can establish dispatch selection, key-policy validation, invalid-handle rejection, buffer sizing, callback error handling and lifecycle semantics. Physical devices add separate tests for slot availability, authentication policy, actual non-exportability, reset behavior, secure element firmware and concurrent sessions. The hosted fifteen-runner records principally document offline algorithm and adapter testing; this publication does not attribute device-specific performance or physical HSM assurance to those results.

23.10 Enterprise and embedded deployment relevance

The common provider contract creates a consistent integration boundary for banking services, secure gateways, public-key infrastructure, industrial controllers, fleet device identities and embedded security processors. Operational due diligence should request provider compatibility matrices, supported algorithms, device firmware, signing formats, authentication rules and recovery procedures. The principal commercial advantage is integration flexibility with policy-aware key custody, not a blanket security guarantee.

Chapter conclusion

The source contains a documented hardware-provider facade, opaque key-handle lifecycle, policy-based operations, multiple backend adapters and TLS signing bridges. This is a significant implementation capability for institutional and embedded deployment, with device-specific assurance evaluated separately.

24 Entropy, Randomness and Sensitive-Key Lifecycle

Randomness is a cross-cutting dependency for key generation, nonces, seeds and protocol challenges. The source contains random-generation facilities and hardware-provider integrations. A FIPS-oriented dossier must distinguish known-answer correctness from the provenance and validation of entropy supplied to production operations.

24.1 Different randomness obligations

Deterministic seeded key generation is valuable for KAT reproducibility; production key generation typically requires an approved random-bit-generation process with a documented entropy source. Conflating the two would create a false assurance claim. The ACVP Demo workflow explicitly required external authorization and verified DRBG prerequisites for certain ECC cases.

24.2 State and error handling

A secure lifecycle includes initialization, health or self-tests, reseeding policy, failure propagation, lifecycle ownership and explicit disposal. Error paths should not silently substitute weak randomness or allow operations after a fatal generator failure. The source contains random utility facilities, but the reviewed consolidated evidence does not certify the end-to-end entropy path for every platform/provider combination.

24.3 Physical and operational provenance

Entropy source quality is partly an operational attribute of hardware, OS services and deployment environment. An offline test container can establish correct deterministic outputs but cannot reproduce the statistical or physical entropy assurance of an embedded device. For regulated customers, vendor documentation and a defined boundary are essential.

24.4 Product integration

Keys may be held in process memory, encrypted storage, HSM objects, or secure elements. Each location implies different control, backup, rotation, export and zeroization requirements. A strong purchaser-facing key-management specification would identify every sensitive-security parameter, generation location, lifetime and permitted migration path.

Chapter conclusion

The source includes the infrastructure needed for sophisticated key management, but the available evidence is a source-level implementation assessment of randomness facilities. Qualification of production randomness and key custody must be established by deployment and boundary-specific records.

CHAPTER 25 | IV. PROTOCOL AND PLATFORM INTEGRATION

25 Windows, Linux and macOS Portability

The final hosted matrix demonstrates that the defined offline cryptographic qualification harness executes successfully across three operating-system families and two CPU architectures. This is a materially stronger statement than availability of portable source code.

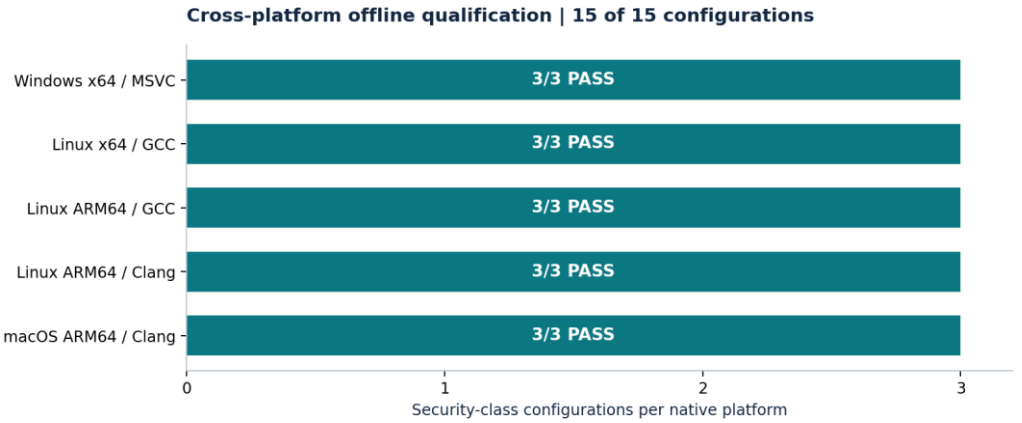


Figure 8. Operating-system, architecture and compiler distribution of completed offline qualification jobs.

25.1 Windows x64

Windows Server 2022 jobs passed for security classes 1, 3 and 5. Their build and execution exercised Windows compiler and runtime behavior under the CI workflow. The consolidated results show the same QSC source archive digest as the Linux and macOS jobs, which makes the comparisons meaningful.

25.2 Linux x64 and ARM64

Ubuntu 24.04 x64 passed for all three classes, and Linux ARM64 passed all three classes under both GCC and Clang. The latter expands coverage across distinct compiler implementations on the same non-x86 architecture. The matrix measures selected crypto harness correctness, not all possible Linux distributions or kernel configurations.

25.3 macOS ARM64

Native macOS ARM64 Apple Clang passed classes 1, 3 and 5. This provides direct evidence for Apple Silicon execution rather than relying on emulation or a source-only build. The job group also passed the final native consolidation gate.

25.4 Interpretation for customer platforms

The 15 configurations reduce the practical risk of cross-platform arithmetic, endian and ABI errors within tested code paths. They do not establish Android, iOS, BSD, 32-bit targets, big-endian CPUs or arbitrary toolchain versions as qualified. Those platforms should be documented according to their own release and test evidence.

Chapter conclusion

The completed matrix is the strongest present portability result: fifteen native configuration passes under identical source provenance and strict artifact completeness criteria.

PART V

LABORATORY METHODS AND RECORDED RESULTS

Testing methodology, historical local work, ACVP harness remediation and final fifteen-runner evidence.

EVIDENCE-LED TECHNICAL EXAMINATION

CHAPTER 26 | V. LABORATORY METHODS AND RECORDED RESULTS

26 Methodology: Positive, Negative and Differential Tests

This qualification program combines fixed-input known-answer testing, differential comparison, malformed-input rejection and source-revision controls. The assessment of each result is based on archived execution evidence and the applicable test acceptance criteria.

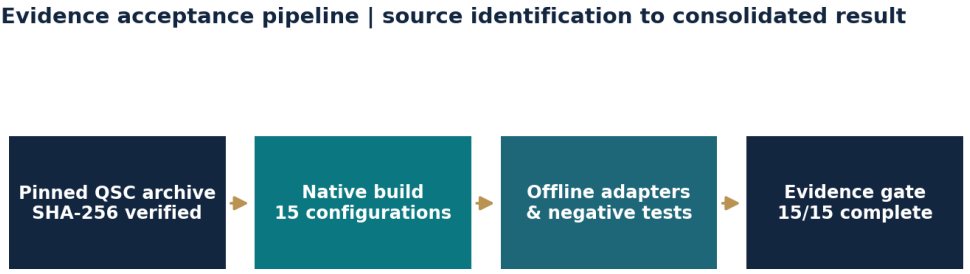


Figure 9. Source-pinned acquisition, compilation, test execution and consolidated acceptance sequence.

26.1 Known-answer and deterministic testing

Known-answer tests establish that a specific implementation maps fixed inputs to expected outputs. Seeded key-generation functions permit reproducible tests of inherently probabilistic primitives. A robust harness should distinguish a known-good vector from an output generated by the implementation itself; otherwise a self-referential test may pass while both implementation and expectation are wrong.

26.2 Independent or differential comparison

The earlier local campaign compared selected QSC outputs with OpenSSL for ML-KEM, ML-DSA and SLH-DSA, and for KMAC/HKDF-related operations. Differential testing is valuable because it can expose implementation-specific divergence even when both programs accept an input. It must still account for randomized signatures and permissible output variation.

26.3 Adversarial and boundary tests

ECDSA invalid-key/signature tests, AES-GCM associated-data boundary checks and AES-CBC rejection tests are examples of negative paths. A useful laboratory scorecard should show not only that valid operations work but that invalid operations are rejected without unintended state mutation or data release.

26.4 Evidence accounting

The hosted CI program uses output markers and stage-level predicates rather than a claim that every function call has an individual test. Legacy suites produce 21 expected success markers per configuration, and the consolidated records require all suites and supporting adapter stages to succeed.

The matrix totals 315 validated legacy success-marker occurrences across fifteen configurations; this is not 315 independent algorithm certifications.

Chapter conclusion

The combined test design addresses several independent failure modes. Results should be presented by method and source revision, with marker counts and assertion counts never substituted for one another.

CHAPTER 27 | V. LABORATORY METHODS AND RECORDED RESULTS

27 Historical Native Linux x86-64 Differential Campaign

A separate, previously completed local campaign exercised the uploaded earlier library under five native Linux x86-64 build configurations. It is useful as historical engineering qualification evidence because it expands test depth in streaming, boundary and interoperability behavior. Its source and binary identity must remain separate from the later 15-runner campaign.

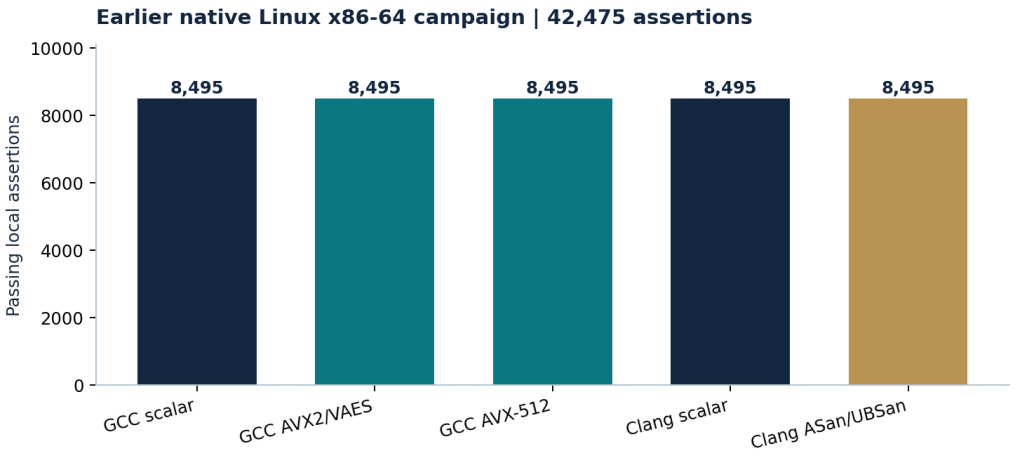


Figure 10. Five recorded native Linux x64 compiler configurations and their functional assertion totals.

27.1 Build configurations

The earlier matrix comprised GCC scalar, GCC AVX2/VAES, GCC AVX-512, Clang scalar, and Clang ASan/UBSan scalar. The recorded source baseline was a different November 5 archive. Each configuration compiled and linked 160 C translation units and executed the same functional suite.

27.2 Assertion accounting

The local report records 8,495 passing checks per build, for 42,475 passing assertions total. Per build, this included SHA-2/HMAC 1,935, SHA-3/SHAKE 5,200, AES-GCM long-message 590, AES-GCM AAD 52, ECDSA negative 213, KMAC 216, ML-KEM 48, ML-DSA 54, SLH-DSA 42, HKDF 48, CBC rejection 96 and a manual symmetric self-test.

27.3 Sanitizer process integrity

In the Clang sanitizer run, the last two slow SLH-DSA SHAKE-256s checks were completed as a separate resumed process after the initial timeout. They were logged as distinct checks. The recorded sanitizer options disabled leak detection; therefore the absence of recorded ASan/UBSan reports is not equivalent to a complete memory-leak assessment.

27.4 Use in this publication

This report presents the 42,475 figure as a documented historical local program, not as a new reproduction from the final branch and not as an independently witnessed benchmark. The primary

archived source artifacts supplied with this task are the two later consolidated CI ZIPs, and their validated assertions are kept in their own accounting.

Chapter conclusion

The historical campaign strengthens the engineering narrative through targeted adversarial and differential work. Its evidentiary value is greatest when cited distinctly from the final common-source 15-job qualification.

CHAPTER 28 | V. LABORATORY METHODS AND RECORDED RESULTS

28 ACVP Harness Architecture and Controlled Source Revisions

The staged ACVP work focused on a failure mode that can invalidate an otherwise impressive green CI dashboard: a harness that runs but fails to verify what it claims to have run. The completion program improved source pinning, stage failure detection, capability accounting and artifact consolidation.

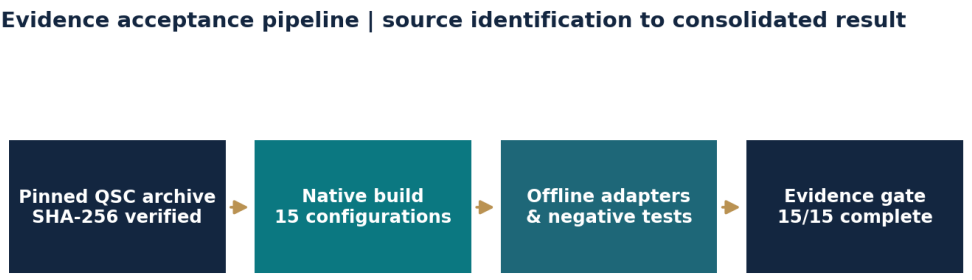


Figure 11. Integrity-gated offline qualification workflow from source digest to final evidence acceptance.

28.1 Pinned execution

The final workflows restore the same controlled source snapshot on each runner and bind results to a qualification branch commit. A SHA-256 digest identifies the archive; a separate source digest and commit establish content and harness provenance. This is important because the main QSC repository changed during the development cycle and older local results could not safely be attributed to an unverified later revision.

28.2 Distinct gates

The harness separates legacy CAVP test invocations from offline ACVP vector adapters and interoperability exercises. Its gate design rejects missing expected success markers, logged failures, skips and absent stage evidence. The consolidated report then rejects missing, failed, unreadable or unexpected runner artifacts and mixed source revisions.

28.3 Cross-platform orchestration

The x64 workflow covers Windows and Ubuntu jobs for classes 1, 3 and 5. The native workflow covers Linux ARM64 GCC, Linux ARM64 Clang and macOS ARM64 for the same classes. Native macOS linker options were separated from Linux-specific ELF flags during preparation so the same source could compile appropriately on both operating-system families.

28.4 Evidence interpretation

The completed workflows report overall success under strict gates, while individual job status remains PASS_WITH_COVERAGE_GAPS. This is intentional: the qualification harness can prove its configured

offline suite passed without claiming the full remote ACVP server protocol or a government certificate. The distinction is a mark of sound reporting rather than a contradictory outcome.

Chapter conclusion

The harness now demonstrates coherent revision control, stage accounting and consolidated fail-closed behavior. The resulting evidence is suitable for internal technical qualification and as input to external due diligence.

29 Stage 1: Build Completeness and Immutable Source Evidence

Stage 1 established complete link dependencies for offline ECC testing and required unambiguous source identification throughout the qualification pipeline. Both controls were exercised successfully in the completed hosted campaign.

29.1 ECC target completeness

The ACVP ECC target includes the asynchronous support compilation unit required by its dependency graph. This source-selection control ensures that the integrated ECC adapter is tested as a complete native target in the configured environment, rather than relying on an incomplete link recipe.

29.2 Pinned revision contract

Workflow source selection was revised to use an immutable Git revision, with an explicit expected source identifier delivered to the consolidator. The final job records carry a uniform hash and source_digest. This makes it possible to reject a mixed build even when each individual job returns success.

29.3 Source-controlled integration

The implementation baseline and test harness were integrated as complete repository-relative files and tracked under controlled revisions. Local consolidation and target-configuration checks complemented the final hosted matrix, which records the definitive successful source-pinned execution.

29.4 Final matrix effect

The final hosted 15-job matrix used the later corrective commit and the November 6 uploaded archive. That run is the decisive evidence of integration of the required Stage 1 behavior in the branch under examination, rather than merely a passing unit test of the proposed patch.

Chapter conclusion

Stage 1 establishes repeatable ECC target construction and immutable source attribution, two prerequisites for credible multi-platform test evidence.

30 Stage 2: Strict Qualification and Evidence Integrity

Stage 2 established independent acceptance controls for ordered legacy test suites, qualification stages, source identity and consolidated runner artifacts. Its design ensures that partial evidence does not satisfy the matrix acceptance criteria.

30.1 Complete legacy-suite acceptance

The parser checks the expected six ordered suites and the twenty-one corresponding success markers. Missing, duplicated, reordered or skipped suite records, nonzero process exits and failure indicators prevent a successful acceptance. The final runner records report complete ordered legacy acceptance on all fifteen native configurations.

30.2 Qualification stage control

The orchestrator tracks a fixed ordered series of offline stages. Each stage must exit successfully and supply nonempty evidence. Logs are hashed and included in provenance records so later reports can confirm the expected input has not been silently replaced.

30.3 Consolidated artifact checking

The combiner checks that the expected platform/class artifacts exist and agree on source revision. The native extension added a nine-job identity set and negative regression tests for missing or extra runner artifacts. These checks are particularly important because parallel CI workflows can otherwise produce a successful-looking dashboard while one security-class job was omitted.

30.4 Recorded verification

An initial 61-test local regression set exercised runner, orchestration and consolidation controls. The integrated program added further configuration and negative tests, and both final CI matrices passed under the strict acceptance policy. These are evidence-integrity checks and are counted separately from cryptographic vector assertions.

Chapter conclusion

Stage 2 provides a tested evidence-acceptance layer, with ordered execution, artifact integrity, complete platform coverage and source-revision consistency.

31 Stage 3: Registered-Capability and Parameter Accounting

Stage 3 maps algorithm registrations to the modes, parameter sets, curves and revisions that must be exercised for the defined offline session. Its implemented coverage controls prevent a partial set of vectors from being accepted as the complete registered set.

31.1 Parameter identities

ML-KEM key generation, encapsulation and decapsulation were tracked separately at the three standardized parameter sets. ML-DSA and SLH-DSA key generation, signing and verification were likewise enumerated by parameter set. AES direction and key length, and relevant ECC curves, were represented explicitly. This prevents a single successful case from satisfying an entire multi-mode registration.

31.2 KDA-HKDF revision identity

KDA-HKDF coverage accounts independently for Sp800-56Cr1 and Sp800-56Cr2 registration and vector revisions. The final code uses revision-aware capability identity and includes regression checks for invalid revisions and omitted registered capabilities. This ensures that execution evidence corresponds to each claimed KDA registration.

31.3 Consistent KAS-ECC-SSC mode mapping

KAS-ECC-SSC vectors identify the SSC operation explicitly, while the registration represents that operation through its algorithm identity. Coverage accounting normalizes these equivalent representations without crediting unrelated modes, missing curves or untested test groups.

31.4 Conservative AES session acceptance

AES mock-session regressions confirm that sampling only some registered directions and key lengths results in FAIL_INCOMPLETE_ALGORITHM_COVERAGE. The validation criteria remain conservative: a partial sample is distinguished from the complete registered capability set.

Chapter conclusion

Stage 3 supplies source-traceable registration coverage, revision identity and fail-closed incomplete-coverage detection for the supported offline algorithm sessions.

32 Final Fifteen-Configuration Hosted Matrix

The qualification program concluded with two independent GitHub Actions matrices on the same corrective commit and source archive. Both consolidated evidence ZIPs were supplied for review and independently parsed for job membership, status, digest consistency and legacy-suite results.

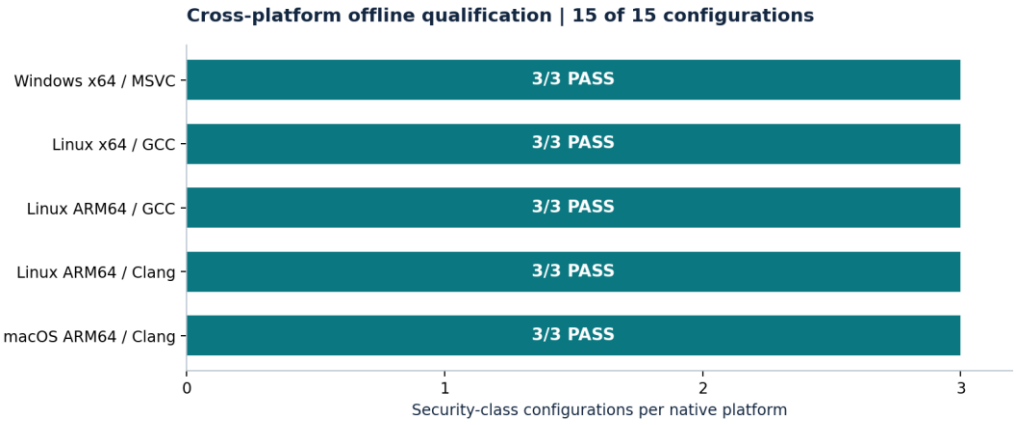


Figure 12. Fifteen platform and security-class job combinations accepted by the qualification matrix.

32.1 The six-job x64 matrix

Windows Server 2022 x64 and Ubuntu 24.04 x64 each executed security classes 1, 3 and 5. Every runner reports PASS_WITH_COVERAGE_GAPS, stages_pass true, legacy_evidence_pass true and PASS status for all six legacy cryptographic suites. The six-job consolidated JSON states success true with empty missing, failed and unexpected lists.

32.2 The nine-job ARM matrix

Linux ARM64 under GCC, Linux ARM64 under Clang and macOS ARM64 under Apple Clang each completed classes 1, 3 and 5. Every runner has the same stage and suite indicators, and the native consolidated JSON likewise states success true, source_uniform true and no missing, failed or unexpected artifacts.

32.3 Source and artifact coherence

Both matrices reference Git commit 07f32d8a18a91f7a9035994adcbecd234a09b5ac and source archive SHA-256 2e64df12c4956c4f58cac1aa468a0765f21a05b23dc4f86a257d48ed15dbf45c. Their source_digest values match across all fifteen results. Individual log hashes differ appropriately because platforms and classes generate distinct test records.

32.4 Meaning of success

The success label refers to completion of the defined offline qualification gates. The per-result PASS_WITH_COVERAGE_GAPS label identifies parameter or external-validation domains outside that gate. The original engineering report separately marks official algorithm validations NOT_ESTABLISHED and NIST Demo NOT_RUN. This document does not collapse those different statuses into one misleading “certified” label.

Chapter conclusion

The final CI evidence unequivocally demonstrates fifteen successful native offline configurations and two successful consolidated acceptance gates on one identified source archive.

CHAPTER 33 | V. LABORATORY METHODS AND RECORDED RESULTS

33 Success Markers, Suite Inventory and Stage Evidence

Forensic reading of CI evidence requires clarity about counted units. The consolidated archive records six legacy cryptographic suites per runner, with a fixed count of accepted success markers. Additional adapter and interoperability stages are represented as passing booleans rather than an additive count of marker strings.

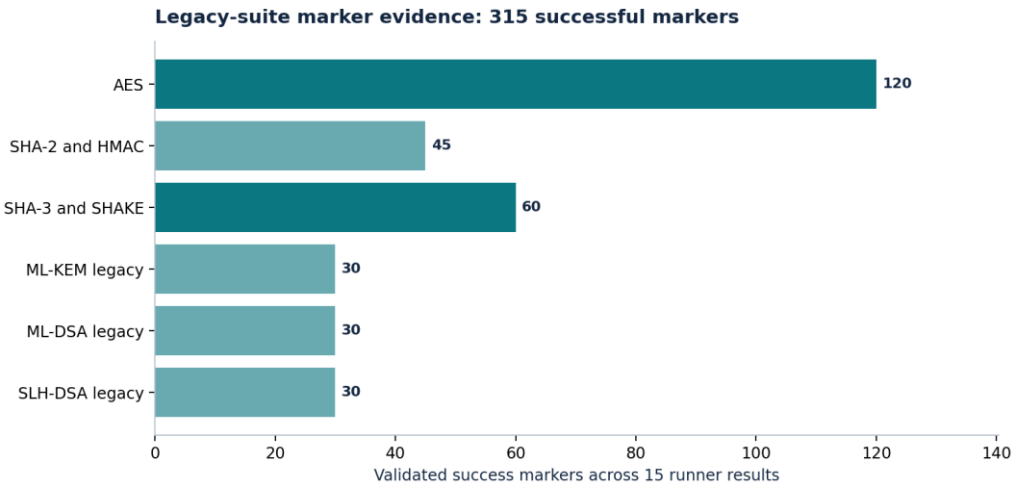


Figure 13. Per-runner accepted legacy test-marker distribution across algorithm families.

33.1 Six legacy suites

The suites are AES, SHA-2/HMAC, SHA-3/SHAKE, ML-KEM legacy interface, ML-DSA legacy interface and SLH-DSA legacy interface. Their marker counts per runner are respectively 8, 3, 4, 2, 2 and 2. Across fifteen configurations this gives 120, 45, 60, 30, 30 and 30 marker occurrences, totaling 315.

33.2 Separate adapter predicates

The results also report passing ECC ACVP adapter, AES interoperability, PQ signature adapter, ML-KEM adapter, local SHA simulation, XOF adapter, PBKDF/HKDF adapter, ECC interoperability and PQ smoke stages. These are meaningful qualification gates, but the consolidated summary does not break each into a verified number of independent cryptographic test cases.

33.3 Correct counting and interpretation

A marker appearing on fifteen platforms represents fifteen successful executions of that marker, rather than fifteen unique vector definitions. The historical Linux campaign is counted separately, with 8,495 assertions across each of five compiler profiles. These units are specified explicitly so the two campaigns remain comparable without double counting.

33.4 Source-controlled acceptance

Each job records success only if both the six-suite legacy evidence and the additional stages satisfy the harness. The final consolidated acceptance adds another layer: expected job identity, valid class, uniform source and archive hashes, and absence of missing or unexpected results. The table and flowchart visualize these independent constraints.

Chapter conclusion

The marker and stage accounting is consistent across the records and supports the defined offline PASS. Its quantitative strength should be described without inflating individual cryptographic assurance claims.

34 Qualification Control Verification and Rerun Evidence

The recorded qualification program applied revision-controlled regression tests and full-matrix reruns after changes to the ACVP control logic. The decisive result is the successful execution of all required jobs against the final pinned source and harness revision.

34.1 Test-control verification

The verified control set includes complete suite ordering, KDA revision identification, KAS mode accounting, AES capability completeness, source-integrity checking and rejection of missing or unexpected runner artifacts. Negative tests demonstrate that insufficient evidence is rejected rather than silently counted as success.

34.2 Targeted change scope

The final ACVP correction affected five harness and test files. It refined capability accounting and test expectations without modifying the qualified cryptographic source snapshot. The changes are linked to the qualification commit and can be inspected independently.

34.3 Final hosted execution

The completed run at commit 07f32d8a18a91f7a9035994adcbecd234a09b5ac repeated the full six-job Windows/Linux matrix and nine-job native ARM64/macOS matrix. Both consolidated reports accepted their complete expected artifact sets and the uniform source revision.

34.4 Operational significance

A dependable qualification process combines minimally scoped changes with negative regression cases and source-bound full-matrix acceptance. This combination supports reproducibility and reduces the risk that a release candidate advances on the strength of incomplete evidence.

Chapter conclusion

The final qualification campaign records an auditable control set and successful same-source reruns across the complete native platform matrix.

PART VI

SECURITY AND ASSURANCE INTERPRETATION

What functional, static and sanitizer evidence can establish, and how it relates to module-level control requirements.

EVIDENCE-LED TECHNICAL EXAMINATION

35 Cross-Compiler and Cross-Architecture Consistency

Cryptographic source code can behave differently under compiler optimization, ABI, platform calling conventions, integer promotion, alignment rules and intrinsic selection. Native execution on multiple platforms is therefore a meaningful assurance layer, particularly for complex finite-field arithmetic and bit-oriented algorithms.

35.1 Native rather than simulated execution

The hosted matrix uses Windows x64, Ubuntu x64, Linux ARM64 and macOS ARM64 runners. ARM64 execution includes GCC and Clang variants on Linux and Apple Clang on macOS. The result is not limited to compiling ARM objects on an x64 host; it includes actual qualification-stage execution on the designated runner architecture.

35.2 Class selection

Security classes 1, 3 and 5 are matrix dimensions relevant to parameter-family selection. Passing all three supplies evidence across the configured parameter security levels. The report does not infer that every arbitrary compilation profile or nonstandard parameter set was tested simply because these three settings passed.

35.3 Endian and width caution

The present platform matrix spans 64-bit little-endian operating environments, not a comprehensive big-endian or 32-bit campaign. Big-endian scalar portability requires a separate source-specific test campaign on the corresponding hardware and ABI.

35.4 Practical assurance

The ability to execute a fixed suite under dissimilar compiler families reduces the plausibility of certain undefined-behavior and platform-assumption defects in the exercised paths. It is complementary to sanitizers, static code analysis, formal analysis, negative testing and high-level protocol verification rather than a replacement.

Chapter conclusion

Cross-platform native success is a significant strength of the qualification record. Its assurance scope is bounded by the actual operating environments and compile-time options tested.

36 Memory Safety, Sanitizer Results and Static Analysis

Memory safety matters in cryptography both as a correctness property and as a route to key exposure. The development record records CodeQL success for the main repository and earlier native sanitizer testing, but those are independent evidence streams that must remain source-specific.

36.1 Static analysis evidence

The main QSC GitHub repository compiled and reported successful CodeQL checks on multiple operating-system targets during the November development cycle. The record shows that certain configured static-analysis queries and build workflows passed. It does not show full formal verification or a comprehensive absence of exploitable defects.

36.2 Earlier sanitizer campaign

The historical local study compiled a Clang scalar AddressSanitizer/UndefinedBehaviorSanitizer variant and exercised the same 8,495 assertion ledger as other local profiles. No sanitizer diagnostic was reported in the stored summary for the exercised paths. Leak detection was explicitly disabled; therefore memory leaks were not qualified by that pass.

36.3 Relevant defect classes

Potential high-impact classes include out-of-bounds reads/writes, signed integer overflow, undefined shifts, mishandled overlapping buffers, use-after-free, uninitialized state and secret-bearing error strings. Static checks and sanitizers cover subsets of these risks, under toolchain- and run-specific conditions. They cannot certify that all cryptographic operations are constant time.

36.4 Productization control

The highest-value next engineering control is a signed, machine-readable mapping from a release commit to its CodeQL configuration, sanitizer options, compilation profile and test inventory. A buyer can then reproduce a finding and distinguish an older result from the current binary without interpretive ambiguity.

Chapter conclusion

The available evidence demonstrates active static-analysis and memory-sanitizer engineering. It should be presented as successful executed checks within their recorded configurations, not as a universal memory-safety proof.

37 Constant-Time Behavior, Fault Handling and Side-Channel Claims

Timing and power side channels are not established or excluded by functional KATs alone. This chapter sets out the correct interpretation of the timing-conscious implementation work recorded in the development program and the level of assurance appropriate for an external technical report.

37.1 Functional versus leakage correctness

An implementation can produce correct ciphertexts, shared secrets and signatures while leaking secret-dependent information through memory access or execution time. Differential output tests and valid-signature verification cannot directly detect those leaks. Separate static and dynamic experiments are required for each secret-bearing operation and compiled branch.

37.2 Risk concentration

Scalar multiplication, secret-dependent scalar reduction, lattice rejection sampling, KEM decapsulation comparison, key parsing and non-constant-time failure paths merit particular attention. Hardware intrinsics can improve performance but may also change the timing model. External provider boundaries introduce distinct power and fault assumptions.

37.3 Recorded development direction

The implementation development record identifies constant-time review and binary-level comparison as engineering techniques for ECC arithmetic. The final CI consolidated artifacts do not include a signed timing-leakage statistical assessment or a formal constant-time proof. It would be incorrect to present those desired analyses as completed third-party certification.

37.4 Assurance language

For procurement, the appropriate statement is that the reviewed implementation uses multiple cryptographic testing mechanisms and has passed the described functional qualification. Any stronger claim about side-channel resistance should cite a dedicated report identifying threat model, hardware, compiler, statistical method, sample size and observed leakage result.

Chapter conclusion

The collected records establish functional and negative-path results for the exercised implementations. Side-channel resistance is a distinct assurance property requiring explicit measurement under a specified compiler, processor and leakage model.

38 Self-Tests, Module Error States and FIPS-Readiness

FIPS 140-3 testing evaluates a cryptographic module as an operational security system, not solely as a collection of correct algorithms. The source contains a self-test module, and the earlier local study includes a manually invoked symmetric self-test. Those observations are materially narrower than a formal validated-module assessment.

38.1 Power-up and conditional behavior

A module security policy must specify when approved algorithms can be used, which self-tests are mandatory, how startup integrity is checked, which conditional tests apply and how the module responds to errors. A manual PASS from a self-test function does not establish enforcement before every service is available.

38.2 Persistent error state

Fatal self-test failure should transition the module to a defined state and prohibit relevant cryptographic services until permitted recovery. This must be verified with failure injection and process/state observations, not inferred from a boolean self-test return value. The present consolidated ACVP ZIPs do not include such a complete module-state demonstration.

38.3 Approved versus non-approved interfaces

The reviewed source includes standardized cryptography, proprietary constructions, utilities and hardware services. A proposed validated boundary must document exactly which functions are approved, non-approved, bypassed or excluded and how applications prevent inappropriate substitution.

38.4 Transition to external validation

A realistic certification package requires module boundary diagrams, versioned binaries, operational environments, entropy documentation, service tables, role/authentication rules, key-management information, security policy, lifecycle evidence and laboratory testing under the applicable CMVP derived requirements. The offline matrix is an important input, not the final certificate.

Chapter conclusion

The self-test implementation and recorded functional checks provide a basis for defining a controlled cryptographic module boundary, startup self-test enforcement, error states and approved-service policies in a subsequent module-level evaluation.

39 Threat Model, Misuse Resistance and Negative Inputs

Cryptographic implementation integrity depends as much on rejecting dangerous requests as on returning correct answers for valid ones. The documented program includes negative-input cases, boundary tests and strict harness failure accounting. This chapter translates those efforts into practical application security considerations.

39.1 Misuse categories

Relevant misuse includes repeated AEAD nonces, invalid tag lengths, malformed public keys, signature encoding ambiguity, partial buffer overlap, untrusted certificate structures, HSM session-loss errors and unsupported algorithm identifiers. Each category has a different security consequence and requires an application-visible error contract.

39.2 Evidence from negative testing

The earlier local record includes 213 ECDSA negative-input assertions, 52 GCM AAD boundary checks and 96 CBC rejection checks per profile. The final CI evidence records strict rejection of incomplete capability sets at the harness level. These are selected examples of protective testing rather than a comprehensive catalogue of malicious inputs.

39.3 API obligation

For production users, errors must be distinguishable without exposing sensitive intermediate state. A failed decryption should not produce an authenticated success indicator; a failed KEM decapsulation should not create an oracle; and a malformed certificate should be rejected before trust decisions. The full API semantics should be captured in product-facing documentation and regression tests.

39.4 Integration discipline

A controlled algorithm profile reduces the number of combinations requiring review. Diligence should request a supported-services manifest keyed to header, entry point, allowed parameters, preconditions and test IDs. This makes it possible to map each high-risk external interface to an explicit negative-path qualification result.

Chapter conclusion

The tested negative paths are a substantive strength. They justify confidence in those exercised conditions but cannot be extrapolated to every protocol misuse scenario or fault model.

PART VII

TECHNICAL AND COMMERCIAL DUE DILIGENCE

Engineering differentiation, procurement implications, source governance, licensing and measured conclusions.

EVIDENCE-LED TECHNICAL EXAMINATION

40 Technology Portfolio and Differentiation

QSC's apparent commercial differentiation is not a single benchmark score. It is the combination of native C implementations, modern post-quantum algorithms, conventional interoperability primitives, protocol/PKI support and a documented multi-platform qualification process. The value of that combination depends on integration quality and maintenance governance.

40.1 Cryptographic breadth

The source inventory spans AES, SHA-2/HMAC, SHA-3/SHAKE/KMAC, ChaCha/Poly1305, NIST prime-field ECC, Edwards curves, ML-KEM, ML-DSA, SLH-DSA and additional post-quantum families. Standardized algorithms make the library broadly relevant; specialized constructions and utilities may create opportunities for differentiated products.

40.2 Software integration reach

TLS, X.509, networking, memory tools and hardware-provider APIs suggest a broader development platform rather than a stand-alone primitive package. Customers seeking one maintained cryptographic dependency can find this attractive, provided supported APIs, build profiles and licensing terms are explicit.

40.3 Quality evidence as product substance

Fifteen successful offline native configurations and strict source/hash provenance are concrete assets for technical sales. They can shorten an evaluator's initial qualification effort because the vendor can supply reproducible artifacts. The benefit is strongest when the customer's exact deployment environment is covered or can be reproduced.

40.4 Differentiation boundaries

Competitive product claims require reproducible speed, size, security and support comparisons against defined alternatives under the same workload and hardware. Such a balanced market benchmark is not supplied by the current consolidated artifacts. This report therefore does not rank QSC over OpenSSL or Botan or assign speculative market share.

Chapter conclusion

QSC presents a credible technical platform with substantial implementation breadth and formalized engineering evidence. The acquisition case is strongest when technical scope is combined with independently verified ownership, licensing, demand and maintainability.

41 Customer Integration and Procurement Considerations

Enterprise and government customers do not buy a source archive in isolation; they acquire an operational dependency with service commitments, licensing obligations and a defined assurance position. This chapter translates the technical record into verifiable procurement questions.

41.1 Supported operating environments

The tested platform matrix is directly relevant to Windows enterprise servers, Linux services, ARM servers and Apple Silicon development or endpoint deployments. It establishes successful offline harness execution on those native runner families and the selected class profiles. Customer-specific OS patch levels and deployment packaging are outside the matrix unless separately tested.

41.2 API onboarding

The library's broad export count offers flexibility but increases the importance of approved recipes, safe defaults, stable facades and versioned usage examples. A buyer should request integration guides for AES-GCM, ML-KEM key establishment, ML-DSA signing, TLS/X.509 bindings and external key providers, including their negative-path contracts.

41.3 Security claims and contractual language

Contracts should distinguish algorithm implementation support, vendor-conducted offline qualification, third-party audit and official certification. The supplied evidence supports the first two within their defined scope. Procurement teams that require a FIPS certificate must insist on the issued CMVP certificate and validated module version, not infer it from this report.

41.4 Operational maintenance

A mature supplier should provide release hashes, documented build profiles, a security advisory process, vulnerability response commitments, deprecation notices and reproducible regression evidence. These controls are especially valuable for cryptographic software, where a performance optimization can unintentionally affect timing or error behavior.

Chapter conclusion

The current evidence can materially accelerate technical evaluation. A production procurement decision should pair it with explicit support and security-policy artifacts appropriate to the buyer's risk and compliance requirements.

42 Intellectual Property, Licensing and Asset Transfer

The source headers identify Quantum Resistant Cryptographic Solutions Corporation as the rights claimant and include QRCS public research/evaluation license language. This provides a clear indication of the asserted ownership and permitted evaluation scope, but a technical report cannot substitute for legal title verification or a transaction-ready license schedule.

42.1 Source notices

The sampled headers distinguish standardized/public-domain algorithms from QRCS-authored implementations and proprietary constructions. The text also indicates commercial production use requires a separate license. An acquirer must verify which version of the license governs each module and whether repository distribution and prior grants align with that position.

42.2 Third-party provenance

Cryptographic source often incorporates or draws from public specifications and existing implementations. The correct diligence review identifies attribution obligations, external dependencies, algorithm patents, code-derived material and any dual-license or GPL-like effects. Technical similarity or machine-code comparison is not a substitute for a clean legal provenance record.

42.3 Asset inventory

The provided 354-file archive and the 1,846-export lexical inventory are useful foundations for a software asset schedule. A full legal inventory would additionally cover tests, build and CI scripts, documentation, licensed external tools, trademarks, patents, firmware and customer-specific modifications.

42.4 Acquisition transfer

A buyer would need warranties or disclosures concerning ownership, development contributions, employee/contractor assignments, open-source obligations, export-control restrictions, existing license rights and key vendor dependencies. This report establishes no legal conclusion on those matters; it identifies the technical artifacts relevant to legal review.

Chapter conclusion

QSC's technical source inventory is concrete, but ownership and licensing assurance must be obtained through an appropriately scoped legal examination before closing an investment or acquisition.

43 Engineering Governance and Reproducibility

The development history shows multiple correction cycles, controlled complete-file replacements, strict tests and branch isolation. Those practices are commercially valuable because cryptographic maintenance is an ongoing obligation, not a one-time algorithm implementation task.

43.1 Version discipline

The program explicitly avoided changing the main QSC branch during laboratory experiments and moved the final matrix to a dedicated QSC-ARM-Tests branch. This separation reduced the risk that unfinished harness changes would disturb production history and provided a clear test revision.

43.2 Minimal corrective scope

The final ACVP regression repair changed five targeted harness/test files instead of modifying cryptographic algorithms to make tests pass. Failure logs were used to identify capability-bookkeeping mismatches and outdated mock expectations. A purchaser can review this as evidence of root-cause-oriented correction rather than symptom suppression.

43.3 Reproduction artifacts

The consolidated JSON provides source and harness digests, platform identities, security classes and suite outcomes. The original per-job artifacts remain available through the recorded GitHub Actions runs. Together, these are the basis of a repeatable audit trail, subject to artifact retention and preservation policies.

43.4 Release process opportunity

For a commercial SDK, the natural continuation is to promote this discipline into release signing, retention policies, source-to-binary attestations, CI cost controls and customer-specific qualification packages. These are implementation recommendations rather than claimed achievements of the November 6 test campaign.

Chapter conclusion

The project demonstrates operational testing discipline and a credible evidence trail. Its sustainability depends on maintaining that discipline across future releases, toolchain upgrades and added cryptographic modes.

44 Due-Diligence Assessment and Evidence-Weighted Conclusions

A technical study for investors and potential acquirers should state what findings are sufficiently supported to be used in valuation discussions and what assumptions require separate underwriting. The results here are positive, but they apply to clearly identified properties of the November 6 implementation.

44.1 Supported technical findings

The controlled source snapshot is identifiable by SHA-256; it contains substantial symmetric, asymmetric, post-quantum, protocol, PKI and hardware-provider implementation code; the public headers expose a large measurable interface; and the offline qualification reports show fifteen complete passing native configurations with consistent source and complete artifacts.

44.2 Strength of testing practice

The program includes automated known-answer and adapter checks, source pinning, negative cases, failure-injection controls, independent interoperability work from an earlier local campaign, and explicit distinction between engineering and official validation. These features reduce uncertainty about baseline quality and testing process.

44.3 Boundaries of conclusions

There is no evidence in the supplied consolidated CI archives of an issued ACVP certificate, FIPS 140-3 module validation, universal constant-time proof, complete physical hardware certification or a platform-by-platform independent penetration test. A commercial memorandum should not describe any of these as completed. Nor should current-source benchmarking superiority be inferred from the historical tests.

44.4 Disposition for external review

The recommended external characterization is: “QSC has completed a source-pinned, fifteen-configuration native offline cryptographic qualification campaign across Windows x64, Linux x64, Linux ARM64 and macOS ARM64; the defined harness passed with documented scope exclusions.” This is precise, testable and useful to a sophisticated purchaser.

Chapter conclusion

The documented library is a significant technical asset with positive reproduced CI evidence and disciplined qualification controls. Further claims must be established on their own evidence, not borrowed from the successful offline matrix.

CHAPTER 45 | VII. TECHNICAL AND COMMERCIAL DUE DILIGENCE

45 Final Technical Conclusion

This final chapter consolidates the measured results without extending their logical scope. The library exhibits broad source-level implementation coverage, substantial public interfaces, cryptographic algorithm diversity and an effective native cross-platform offline qualification process.

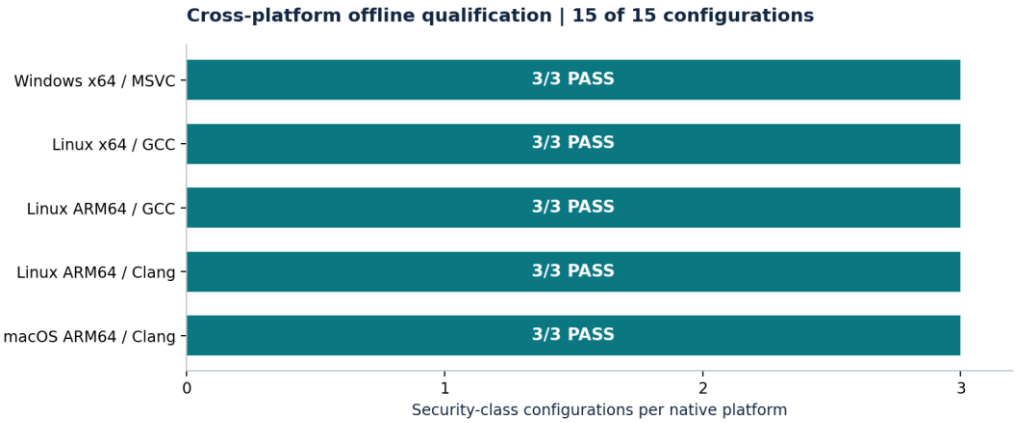


Figure 14. Final platform coverage matrix associated with the controlled qualification revision.

45.1 Architecture

QSC is a native C software corpus with conventional and post-quantum cryptography, protocol and PKI infrastructure, and hardware-provider integration. Its measured source inventory includes 354 files, 176 C units, 164 headers and 1,846 lexical public export declarations. These are reproducible source attributes.

45.2 Cross-platform testing

The primary hosted qualification achieved six of six x64 native runner passes and nine of nine ARM64/macOS runner passes, with successful consolidation of each matrix. Every job reports all six legacy cryptographic suites passing and all higher-level adapter stage gates passing. All jobs use the same source archive and qualification commit.

45.3 Engineering depth

A separate local Linux x64 campaign recorded 42,475 functional assertion executions across five configurations, with compiler, differential, negative-input and sanitizer aspects. Those historical local results reinforce the engineering narrative but are appropriately separate from the final-source hosted matrix.

45.4 Commercially appropriate statement

The evidence supports product evaluation, technical due diligence, investor discussion and preparation for independently supervised certification. It does not independently establish the issuance of a NIST algorithm certificate, an accredited module validation or a cryptanalytic security proof for every

proprietary primitive. Recognition of those boundaries strengthens rather than weakens the reliability of this document.

Chapter conclusion

QSC's documented qualification result is a complete and source-consistent fifteen-configuration offline engineering PASS, supported by extensive implementation evidence and a prior deep local test campaign. This is a substantive foundation for enterprise evaluation and future formal assurance work.

APPENDIX A

Runner-by-Runner Qualification Evidence

This appendix reproduces key machine-readable fields of all fifteen primary consolidated CI results. The numerical unit is a native platform/class job. All entries are sourced from the two supplied consolidated JSON artifacts; no result has been invented or inferred from a green status badge alone.

Native target	Class	Gates	Legacy markers	Source digest prefix
Linux ARM64 / Clang	1	PASS	21	0f66a603af66
Linux ARM64 / Clang	3	PASS	21	0f66a603af66
Linux ARM64 / Clang	5	PASS	21	0f66a603af66
Linux ARM64 / GCC	1	PASS	21	0f66a603af66
Linux ARM64 / GCC	3	PASS	21	0f66a603af66
Linux ARM64 / GCC	5	PASS	21	0f66a603af66
macOS ARM64 / Clang	1	PASS	21	0f66a603af66
macOS ARM64 / Clang	3	PASS	21	0f66a603af66
macOS ARM64 / Clang	5	PASS	21	0f66a603af66
Ubuntu 24.04 x64	1	PASS	21	0f66a603af66
Ubuntu 24.04 x64	3	PASS	21	0f66a603af66
Ubuntu 24.04 x64	5	PASS	21	0f66a603af66
Windows Server 2022 x64	1	PASS	21	0f66a603af66
Windows Server 2022 x64	3	PASS	21	0f66a603af66
Windows Server 2022 x64	5	PASS	21	0f66a603af66

Per-job acceptance indicators

Every result contains `stages_pass: true`, `legacy_evidence_pass: true`, `class_match: true`, `source_match: true`, `archive_match: true`, and passing ECC, AES, post-quantum, SHA simulation, XOF and KDF adapter predicates. The `source_digest` is identical across all fifteen results. Runner-specific `log_sha256` values differ because they identify individual evidence logs.

A.1 qsc-offline-linux-arm64-clang-class-1

Class 1; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 79d509f965616123f6ef8c968e722e361a8071205547c71c709cd0f8087ca13e; harness digest fd6d0bfa862054ec030527ab25cb82bf3a5f84eef7bb83c4000067169b2740c2.

A.2 qsc-offline-linux-arm64-clang-class-3

Class 3; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 8329f5063d137a3096b3012066fe310292299338649c3f1b5be3f10c6f7e7be7; harness digest 6efb2c9ba44a206047d69dc074617f61f1ab2f9d5dfa19fed70b67ba9352b622.

A.3 qsc-offline-linux-arm64-clang-class-5

Class 5; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 0fe16bb793ff1b0e4f7b25e5ca15db244f0cf85f999a6abf8fe8397bdbb79080; harness digest 761396340e15748dd329b6ec1a887bd23da893e105156f99a684903946152d16.

A.4 qsc-offline-linux-arm64-gcc-class-1

Class 1; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 79d509f965616123f6ef8c968e722e361a8071205547c71c709cd0f8087ca13e; harness digest 601a9fa3a6d867d6633865b560f63c0ab313b3adce39351d708bcd97017c2c40.

A.5 qsc-offline-linux-arm64-gcc-class-3

Class 3; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 8329f5063d137a3096b3012066fe310929299338649c3f1b5be3f10c6f7e7be7; harness digest ea0cdd9bcf6cc28cb6237c72541d15be9dae747c8a4fcccf2b6739ac3540d911.

A.6 qsc-offline-linux-arm64-gcc-class-5

Class 5; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 0fe16bb793ff1b0e4f7b25e5ca15db244f0cf85f999a6abf8fe8397bdbb79080; harness digest bb3b55e23672b8a70034fa927ff00a29b12435fa020689d5d2c4fd8aa74f4942.

A.7 qsc-offline-macos-arm64-class-1

Class 1; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 79d509f965616123f6ef8c968e722e361a8071205547c71c709cd0f8087ca13e; harness digest 0064c3a42a23a1dfc74b5d9a430d97035f09093d511b055ab21a66b1625a46c7.

A.8 qsc-offline-macos-arm64-class-3

Class 3; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 8329f5063d137a3096b3012066fe310929299338649c3f1b5be3f10c6f7e7be7; harness digest 47c974eec4eaae22895d0ffc8c1e8a435f40e39c256fe68df8d367906af2a140.

A.9 qsc-offline-macos-arm64-class-5

Class 5; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 0fe16bb793ff1b0e4f7b25e5ca15db244f0cf85f999a6abf8fe8397bdbb79080; harness digest 42b2b3153f81c5ac819495911d845f13d82b5e761dffcb80b74d7e33d0cea15a.

A.10 qsc-offline-ubuntu-24.04-class-1

Class 1; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 ac7b3a3340b8c1b9d676ff106bafc29ff32f16d2d53fdcfa52d162f09df68b73; harness digest 0b27f878258166283e2f7b1af3f9e256d5c71fbd728611f8d6947bab2a25d9cd.

A.11 qsc-offline-ubuntu-24.04-class-3

Class 3; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 3d904ab19a7dac5ed39c19653388945578de7fe1e025ba87b77e384d96da619e; harness digest 69661ad237d66dec2bd42ba2f0c20fcd1ccb41df5b204e84c8226ac614d2a807.

A.12 qsc-offline-ubuntu-24.04-class-5

Class 5; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 2aeede5315720e84159a32c9cf4ef91bb2421c6221bf69f6431c34ecf0507263; harness digest 68caab271f7005e1b65cc0c95b1feaa14696d72a5a5ec06595fbc6aa4a7b9721.

A.13 qsc-offline-windows-2022-class-1

Class 1; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 17a2420bd4f610738c8426d9c743486abfacb40730bd11b8b7d79ce216370c36; harness digest 249f11b681a623adfd9fc6b9e50e4e9e5220cd9da6b663356bcfde3599419efa.

A.14 qsc-offline-windows-2022-class-3

Class 3; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 486b7fae75d7bd189389d81a2816b2c0293775849f6f8bb3613ab5a7a6f5964b; harness digest 80fe93e05bec4f0f08a2c4572f2b2c7d4abff1558048a39f2d95c8ca230527c3.

A.15 qsc-offline-windows-2022-class-5

Class 5; status PASS_WITH_COVERAGE_GAPS; stages PASS; legacy suites PASS; log SHA-256 12917b5a43528ab317511eb04ffafb0cf90a877e69d7477cf17e57d1ad2907b9; harness digest 98daea899b125304eaf26eb0b97ebac0c7e025e8962b616532b5eab99676fffe.

APPENDIX B

Legacy Suite and Stage Evidence

The archive records fixed legacy-suite marker quantities and selected adapter-stage predicates. This appendix presents the actual counted quantities without converting them into unrelated algorithm or certification counts.

Suite	Markers / runner	Successful runner occurrences	Total accepted markers
AES	8	15	120
SHA-2 and HMAC	3	15	45
SHA-3 and SHAKE	4	15	60
ML-KEM legacy interface	2	15	30
ML-DSA legacy interface	2	15	30
SLH-DSA legacy interface	2	15	30
TOTAL	21	15	315

Additional stage predicates

Recorded predicate	Passing runner records
ecc_acvp_adapter_pass	15
aes_interoperability_pass	15
pq_signature_adapter_pass	15
mlkem_adapter_pass	15
local_sha_simulation_pass	15
xof_offline_adapter_pass	15
pbkdf_hkdf_adapter_pass	15
ecc_interoperability_pass	15
pq_smoke_pass	15
stages_pass	15
legacy_evidence_pass	15

Interpretation

A predicate documents the acceptance of a stage according to the harness contract. The consolidated summary does not enumerate every underlying test case within that stage, so the numerical counts in this appendix are limited to marker totals and runner-level predicate observations.

APPENDIX C

Complete Source-File Inventory

All 354 non-directory archive members are listed below with textual line counts. Lines include comments, licenses and generated implementation tables. The inventory is intended for provenance and source-scope review, not as a software-effort estimate.

Archive member	Text lines	Type
QSC.vcxproj	605	vcxproj
QSC.vcxproj.filters	1251	filters
QSC.vcxproj.user	4	user
acp.c	242	c
acp.h	178	h
aes.c	5295	c
aes.h	965	h
arrayutils.c	603	c
arrayutils.h	287	h
asn1.c	1464	c
asn1.h	516	h
async.c	984	c
async.h	507	h
chacha.c	1411	c
chacha.h	309	h
cjp.c	680	c
cjp.h	117	h
cmake/QSCCapabilities.json.in	13	in
cmake/QSCCompiler.cmake	82	cmake
cmake/QSCConfig.cmake.in	62	in
cmake/QSCFeatures.h.in	81	in
cmake/QSCHardware.cmake	152	cmake
cmake/QSCInstall.cmake	86	cmake
cmake/QSCProfiles.cmake	100	cmake
cmake/QSCSources.cmake	300	cmake
cmake/README.md	85	md
collection.c	912	c
collection.h	314	h
consoleutils.c	834	c
consoleutils.h	381	h
cpuidex.c	1363	c
cpuidex.h	193	h
csg.c	296	c
csg.h	181	h
csp.c	202	c
csp.h	159	h
csx.c	1244	c
csx.h	295	h
dilithium.c	143	c
dilithium.h	281	h
dilithiumbase.c	2087	c
dilithiumbase.h	151	h
dilithiumbase_avx2.c	2468	c

Archive member	Text lines	Type
dilithiumbase_avx2.h	144	h
donna128.c	199	c
donna128.h	192	h
doxymain.h	611	h
ecdh.c	163	c
ecdh.h	270	h
ecdhp256.c	3223	c
ecdhp256.h	2583	h
ecdhp384.c	4036	c
ecdhp384.h	204	h
ecdhp521.c	5123	c
ecdhp521.h	195	h
ecdsa.c	267	c
ecdsa.h	421	h
ecdsap256.c	9307	c
ecdsap256.h	614	h
ecdsap384.c	5179	c
ecdsap384.h	470	h
ecdsap521.c	3588	c
ecdsap521.h	357	h
ed25519.c	8539	c
ed25519.h	646	h
ed448.c	1845	c
ed448.h	589	h
ed448base.h	884	h
eddh.c	155	c
eddh.h	227	h
eddh25519.c	172	c
eddh25519.h	178	h
eddh448.c	1075	c
eddh448.h	188	h
eddsa.c	118	c
eddsa.h	221	h
eddsa25519.c	288	c
eddsa25519.h	152	h
eddsa448.c	341	c
eddsa448.h	152	h
encoding.c	2657	c
encoding.h	801	h
etr.c	139	c
etr.h	218	h
event.c	299	c
event.h	206	h
falcon.c	223	c
falcon.h	188	h
falcon1024.h	124	h
falcon512.h	124	h

Inventory continued (91–180)

Archive member	Text lines	Type
falconbase.c	9802	c

Archive member	Text lines	Type
falconbase.h	265	h
falconbase_avx2.c	10007	c
falconbase_avx2.h	285	h
fileutils.c	1555	c
fileutils.h	508	h
folderutils.c	615	c
folderutils.h	239	h
hcg.c	218	c
hcg.h	202	h
hqc.c	95	c
hqc.h	257	h
hqc1.c	219	c
hqc3.c	219	c
hqc5.c	221	c
hqc_multi.h	235	h
hqcbase.c	2587	c
hqcbase.h	536	h
hqcbase_avx2.c	2778	c
hqcbase_avx2.h	535	h
hwatecc.c	462	c
hwatecc.h	205	h
hwncrypt.c	728	c
hwncrypt.h	121	h
hwncrypttls.c	512	c
hwncrypttls.h	157	h
hwpkcs11.h	277	h
hwpkcs11tls.c	494	c
hwpkcs11tls.h	157	h
hwplatform.c	667	c
hwplatform.h	235	h
hwprovider.c	576	c
hwprovider.h	315	h
hwpsa.c	744	c
hwpsa.h	243	h
hwpsanative.c	471	c
hwpsanative.h	100	h
hwse050.c	145	c
hwse050.h	102	h
hwtpm.c	771	c
hwtpm.h	129	h
hwtpmquote.c	338	c
hwtpmquote.h	125	h
hwtypes.h	203	h
intrinsics.h	200	h
intutils.c	905	c
intutils.h	672	h
ipinfo.c	990	c
ipinfo.h	521	h
kyber.c	165	c
kyber.h	273	h
kyber1024.c	81	c
kyber1024.h	142	h

Archive member	Text lines	Type
kyber1280.c	81	c
kyber1280.h	142	h
kyber512.c	81	c
kyber512.h	142	h
kyber768.c	81	c
kyber768.h	142	h
kyber_parameter_impl.h	148	h
kyberbase.c	1206	c
kyberbase.h	241	h
kyberbase_avx2.c	1801	c
kyberbase_avx2.h	247	h
list.c	786	c
list.h	308	h
mceliece.c	143	c
mceliece.h	272	h
mceliece_460896.c	150	c
mceliece_460896f.c	157	c
mceliece_460896pc.c	143	c
mceliece_460896pcf.c	150	c
mceliece_6688128.c	150	c
mceliece_6688128f.c	157	c
mceliece_6688128pc.c	143	c
mceliece_6688128pcf.c	150	c
mceliece_6960119.c	150	c
mceliece_6960119f.c	157	c
mceliece_6960119pc.c	143	c
mceliece_6960119pcf.c	150	c
mceliece_8192128.c	150	c
mceliece_8192128f.c	157	c
mceliece_8192128pc.c	143	c
mceliece_8192128pcf.c	150	c
mceliece_multi.h	915	h
mceliecebase.c	2390	c
mceliecebase.h	100	h
mceliecebase_avx2.c	60	c
mceliecebase_avx2.h	67	h
memutils.c	2034	c

Inventory continued (181–270)

Archive member	Text lines	Type
memutils.h	452	h
mldsa.h	312	h
mldsa44.c	59	c
mldsa65.c	59	c
mldsa87.c	59	c
mldsa_parameter_impl.h	346	h
netutils.c	1141	c
netutils.h	277	h
oid.c	504	c
oid.h	268	h
p11agree.c	153	c

Archive member	Text lines	Type
p11internal.h	159	h
p11keys.c	631	c
p11loader.c	673	c
p11session.c	195	c
p11sign.c	102	c
pbmac1.c	409	c
pbmac1.h	248	h
pkcs11.h	2075	h
poly1305.c	1013	c
poly1305.h	207	h
qmac.c	425	c
qmac.h	241	h
qsccommon.h	1578	h
qsort.c	984	c
qsort.h	119	h
queue.c	688	c
queue.h	269	h
rcs.c	2288	c
rcs.h	333	h
rdp.c	203	c
rdp.h	130	h
scb.c	340	c
scb.h	198	h
secrand.c	661	c
secrand.h	306	h
selftest.c	3358	c
selftest.h	124	h
sha2.c	3195	c
sha2.h	671	h
sha3.c	6721	c
sha3.h	1317	h
slhdsa.h	824	h
slhdsa_shake128f.c	203	c
slhdsa_shake128s.c	203	c
slhdsa_shake192f.c	206	c
slhdsa_shake192s.c	203	c
slhdsa_shake256f.c	203	c
slhdsa_shake256s.c	203	c
slhdsa_shake_api.inc	459	inc
slhdsa_shake_core.inc	1450	inc
socket.c	2	c
socket.h	146	h
socketbase.c	2911	c
socketbase.h	973	h
socketclient.c	225	c
socketclient.h	226	h
socketflags.h	218	h
socketserver.c	792	c
socketserver.h	321	h
sphincsplus.c	200	c
sphincsplus.h	366	h
sphincsplusbase.c	1388	c

Archive member	Text lines	Type
sphincsplusbase.h	164	h
sphincsplusbase_avx2.c	1	c
sphincsplusbase_avx2.h	67	h
stringutils.c	1228	c
stringutils.h	522	h
sysutils.c	754	c
sysutils.h	229	h
threadpool.c	651	c
threadpool.h	213	h
timerex.c	307	c
timerex.h	155	h
timestamp.c	563	c
timestamp.h	250	h
tlsalert.c	355	c
tlsalert.h	175	h
tlscert.c	1080	c
tlscert.h	431	h
tlscertx509.c	176	c
tlscertx509.h	112	h
tlsclient.c	3024	c
tlsclient.h	307	h
tlscodec.c	630	c
tlscodec.h	327	h
tlsdefs.h	324	h
tlsecdsader.c	334	c
tlsecdsader.h	104	h
tlsengine.c	1231	c

Inventory continued (271–354)

Archive member	Text lines	Type
tlsengine.h	376	h
tlserrors.c	35	c
tlserrors.h	90	h
tlsextensions.c	1681	c
tlsextensions.h	679	h
tlsgroups.c	1260	c
tlsgroups.h	190	h
tlshandshake.c	240	c
tlshandshake.h	223	h
tlsio.c	666	c
tlsio.h	196	h
tlskeyschedule.c	938	c
tlskeyschedule.h	601	h
tlslimits.h	436	h
tlsrecord.c	938	c
tlsrecord.h	213	h
tlsserver.c	3869	c
tlsserver.h	298	h
tlssession.c	231	c
tlssession.h	160	h
tlssigalgs.c	265	c

Archive member	Text lines	Type
tlssigalgs.h	250	h
tlssignerdefault.c	640	c
tlssignerdefault.h	148	h
tlssocket.c	3705	c
tlssocket.h	1315	h
tlsstate.h	159	h
tlstranscript.c	303	c
tlstranscript.h	144	h
tlstypes.h	246	h
transpose.c	175	c
transpose.h	124	h
winutils.c	1645	c
winutils.h	327	h
x509aia.c	83	c
x509aia.h	87	h
x509cert.c	1482	c
x509cert.h	114	h
x509certwrite.c	1900	c
x509certwrite.h	581	h
x509crl.c	881	c
x509crl.h	326	h
x509crlwrite.c	987	c
x509crlwrite.h	289	h
x509csr.c	1623	c
x509csr.h	494	h
x509ext.c	1465	c
x509ext.h	470	h
x509host.c	256	c
x509host.h	174	h
x509key.c	1850	c
x509key.h	413	h
x509keywrite.c	1027	c
x509keywrite.h	284	h
x509name.c	948	c
x509name.h	179	h
x509ocsp.c	1792	c
x509ocsp.h	203	h
x509pem.c	695	c
x509pem.h	409	h
x509pkcs12.c	1767	c
x509pkcs12.h	244	h
x509pkcs12crypto.c	519	c
x509rev.c	252	c
x509rev.h	194	h
x509revext.c	2337	c
x509revext.h	144	h
x509sig.c	392	c
x509sig.h	209	h
x509sigver.c	513	c
x509sigver.h	189	h
x509spki.c	998	c
x509spki.h	340	h

Archive member	Text lines	Type
x509store.c	380	c
x509store.h	208	h
x509time.c	342	c
x509time.h	193	h
x509types.h	700	h
x509verify.c	2118	c
x509verify.h	420	h
x509wrap.c	2563	c
x509wrap.h	1127	h
x509write.c	1904	c
x509write.h	618	h

APPENDIX D

Public Exported API Inventory

The following is a complete lexical inventory of 1,846 distinct QSC-prefixed functions found following `QSC_EXPORT_API` markers in the 164 headers of the uploaded snapshot. These names are not a claim that every function is available under every compiler/profile, nor that all were executed in the qualification matrix. Function names are organized by defining header for technical review.

Header	Identified exports
acp.h	4
aes.h	30
arrayutils.h	15
asn1.h	28
async.h	33
chacha.h	10
cjp.h	5
collection.h	17
consoleutils.h	29
cpuidex.h	3
csg.h	4
csp.h	4
csx.h	6
dilithium.h	6
donna128.h	8
ecdh.h	4
ecdhp256.h	11
ecdhp384.h	11
ecdhp521.h	9
ecdsa.h	8
ecdsap256.h	34
ecdsap384.h	28
ecdsap521.h	20
eddh.h	4
eddh25519.h	7
eddh448.h	8
eddsa.h	4
eddsa25519.h	4
eddsa448.h	4
encoding.h	25
etr.h	3
event.h	6
falcon.h	3
falcon1024.h	3
falcon512.h	3
fileutils.h	33
folderutils.h	10
hcg.h	4
hqc.h	4
hqc_multi.h	15

Header	Identified exports
hwatecc.h	4
hwncrypt.h	4
hwncrypttls.h	7
hwpkcs11.h	16
hwpkcs11tls.h	7
hwplatform.h	4
hwprovider.h	13
hwpsa.h	4
hwpsanative.h	1
hwse050.h	1
hwtpm.h	4
hwtpmquote.h	2
intutils.h	54
ipinfo.h	36
kyber.h	4
kyber1024.h	5
kyber1280.h	5
kyber512.h	5
kyber768.h	5
list.h	19
mceliece.h	3
mceliece_multi.h	48
memutils.h	37
mldsa.h	18
netutils.h	14
oid.h	10
pbmac1.h	8
poly1305.h	8
qmac.h	5
qsort.h	4
queue.h	14
rcs.h	6
rdp.h	5
scb.h	4
secrand.h	24
selftest.h	7
sha2.h	38
sha3.h	52
slhdsa.h	48
socketbase.h	61
socketclient.h	12
socketserver.h	17
sphincsplus.h	7
stringutils.h	41
sysutils.h	14
threadpool.h	10
timerex.h	8
timestamp.h	14
tlsalert.h	8
tlscert.h	14
tlscertx509.h	2
tlsclient.h	8

Header	Identified exports
tlscodec.h	23
tlsecdsader.h	2
tlsengine.h	13
tlserrors.h	1
tlsextensions.h	35
tlsgroups.h	6
tlshandshake.h	10
tlsio.h	6
tlskeyschedule.h	26
tlsrecord.h	9
tlsserver.h	11
tlssession.h	3
tlssigalgs.h	13
tlssignerdefault.h	2
tlssocket.h	73
tlstranscript.h	6
transpose.h	4
winutils.h	17
x509aia.h	1
x509cert.h	4
x509certwrite.h	28
x509crl.h	9
x509crlwrite.h	12
x509csr.h	28
x509ext.h	23
x509host.h	5
x509key.h	11
x509keywrite.h	10
x509name.h	8
x509ocsp.h	3
x509pem.h	18
x509pkcs12.h	6
x509rev.h	3
x509revext.h	3
x509sig.h	9
x509sigver.h	5
x509spki.h	18
x509store.h	8
x509time.h	7
x509verify.h	18
x509wrap.h	74
x509write.h	39

acp.h | 4 identified exports

Public identifier A	Public identifier B
`qsc_acp_generate`	`qsc_acp_uint16`
`qsc_acp_uint32`	`qsc_acp_uint64`

aes.h | 30 identified exports

Public identifier A	Public identifier B

Public identifier A	Public identifier B
qsc_aes_dispose	qsc_aes_initialize
qsc_aes_cbc_decrypt	qsc_aes_cbc_encrypt
qsc_aes_cbc_decrypt_block	qsc_aes_cbc_encrypt_block
qsc_pkcs7_add_padding	qsc_pkcs7_padding_length
qsc_aes_ctrbe_transform	qsc_aes_ctrle_transform
qsc_aes_ecb_decrypt_block	qsc_aes_ecb_encrypt_block
qsc_aes_ecb_encrypt_blocks	qsc_aes_ecb_decrypt_blocks
qsc_aes_hba256_dispose	qsc_aes_hba256_initialize
qsc_aes_hba256_set_associated	qsc_aes_hba256_transform
qsc_aes_gcm128_decrypt	qsc_aes_gcm128_dispose
qsc_aes_gcm128_encrypt	qsc_aes_gcm128_initialize
qsc_aes_gcm128_set_associated	qsc_aes_gcm128_transform
qsc_aes_gcm256_decrypt	qsc_aes_gcm256_dispose
qsc_aes_gcm256_encrypt	qsc_aes_gcm256_initialize
qsc_aes_gcm256_set_associated	qsc_aes_gcm256_transform

arrayutils.h | 15 identified exports

Public identifier A	Public identifier B
qsc_arrayutils_find_string	qsc_arrayutils_hex_to_uint8
qsc_arrayutils_uint8_to_hex	qsc_arrayutils_uint16_to_hex
qsc_arrayutils_uint32_to_hex	qsc_arrayutils_uint64_to_hex
qsc_arrayutils_string_to_uint8	qsc_arrayutils_string_to_uint16
qsc_arrayutils_string_to_uint32	qsc_arrayutils_string_to_uint64
qsc_arrayutils_try_parse_uint8	qsc_arrayutils_try_parse_uint16
qsc_arrayutils_try_parse_uint32	qsc_arrayutils_try_parse_uint64
qsc_arrayutils_self_test	

asn1.h | 28 identified exports

Public identifier A	Public identifier B
qsc_asn1_der_decode_exact	qsc_asn1_der_get_child_region
qsc_asn1_oid_compare	qsc_asn1_require_sequence
qsc_asn1_get_child	qsc_asn1_is_boolean
qsc_asn1_is_integer	qsc_asn1_decode_integer_u64
qsc_asn1_is_null	qsc_asn1_is_oid
qsc_asn1_element_is_tag	qsc_asn1_require_tag
qsc_asn1_child_count	qsc_asn1_child_at
qsc_asn1_find_context_child	qsc_asn1_get_explicit_child
qsc_asn1_decode_boolean	qsc_asn1_decode_uint64
qsc_asn1_decode_octet_string	qsc_asn1_decode_bit_string
qsc_asn1_decode_null	qsc_asn1_decode_oid
qsc_asn1_oid_are_equal	qsc_asn1_oid_to_string
qsc_asn1_decode_string	qsc_asn1_decode_time
qsc_asn1_time_compare	qsc_asn1_der_size

async.h | 33 identified exports

Public identifier A	Public identifier B
qsc_async_atomic_bool_load	qsc_async_atomic_bool_store
qsc_async_atomic_bool_exchange	qsc_async_atomic_bool_compare_exchange
qsc_async_atomic_int32_load	qsc_async_atomic_int32_store
qsc_async_atomic_int32_exchange	qsc_async_atomic_int32_compare_exchange
qsc_async_atomic_int32_add	qsc_async_atomic_int32_subtract
qsc_async_atomic_int32_increment	qsc_async_atomic_int32_decrement
qsc_async_launch_thread	qsc_async_launch_parallel_threads
qsc_async_mutex_create	qsc_async_mutex_destroy

Public identifier A	Public identifier B
qsc_async_mutex_lock	qsc_async_mutex_lock_ex
qsc_async_mutex_unlock	qsc_async_mutex_unlock_ex
qsc_async_parallel_for	qsc_async_processor_count
qsc_async_thread_create	qsc_async_thread_create_detached
qsc_async_thread_create_ex	qsc_async_thread_create_noargs
qsc_async_thread_resume	qsc_async_thread_sleep
qsc_async_thread_suspend	qsc_async_thread_terminate
qsc_async_thread_wait	qsc_async_thread_wait_time
qsc_async_thread_wait_all	

chacha.h | 10 identified exports

Public identifier A	Public identifier B
qsc_chacha_dispose	qsc_chacha_initialize
qsc_chacha_transform	qsc_chacha_transform_checked
qsc_chacha_poly1305_set_associated	qsc_chacha_poly1305_decrypt
qsc_chacha_poly1305_dispose	qsc_chacha_poly1305_initialize
qsc_chacha_poly1305_encrypt	qsc_chacha_poly1305_encrypt_checked

cjp.h | 5 identified exports

Public identifier A	Public identifier B
qsc_cjp_available	qsc_cjp_generate
qsc_cjp_uint16	qsc_cjp_uint32
qsc_cjp_uint64	

collection.h | 17 identified exports

Public identifier A	Public identifier B
qsc_collection_add	qsc_collection_reserve
qsc_collection_enable_index	qsc_collection_disable_index
qsc_collection_deserialize_sized	qsc_collection_serialize_sized
qsc_collection_deserialize	qsc_collection_dispose
qsc_collection_erase	qsc_collection_item_exists
qsc_collection_find	qsc_collection_initialize
qsc_collection_item	qsc_collection_remove
qsc_collection_serialize	qsc_collection_size
qsc_collection_test	

consoleutils.h | 29 identified exports

Public identifier A	Public identifier B
qsc_consoleutils_colored_message	qsc_consoleutils_get_char
qsc_consoleutils_get_formatted_line	qsc_consoleutils_get_line
qsc_consoleutils_get_quoted_string	qsc_consoleutils_get_wait
qsc_consoleutils_hex_to_bin	qsc_consoleutils_line_contains
qsc_consoleutils_line_equals	qsc_consoleutils_masked_password
qsc_consoleutils_message_confirm	qsc_consoleutils_print_array
qsc_consoleutils_print_hex	qsc_consoleutils_print_formatted
qsc_consoleutils_print_formatted_line	qsc_consoleutils_print_safe
qsc_consoleutils_print_line	qsc_consoleutils_print_uint
qsc_consoleutils_print_ulong	qsc_consoleutils_print_double
qsc_consoleutils_print_concatenated_line	qsc_consoleutils_progress_counter
qsc_consoleutils_send_enter	qsc_consoleutils_set_window_buffer
qsc_consoleutils_set_window_clear	qsc_consoleutils_set_window_prompt

Public identifier A	Public identifier B
qsc_consoleutils_set_window_size	qsc_consoleutils_set_window_title
qsc_consoleutils_set_virtual_terminal	

cpuidex.h | 3 identified exports

Public identifier A	Public identifier B
qsc_cpuidex_features_set	qsc_cpuidex_features_set_cached
qsc_cpuidex_print_stats	

csg.h | 4 identified exports

Public identifier A	Public identifier B
qsc_csg_dispose	qsc_csg_initialize
qsc_csg_generate	qsc_csg_update

csp.h | 4 identified exports

Public identifier A	Public identifier B
qsc_csp_generate	qsc_csp_uint16
qsc_csp_uint32	qsc_csp_uint64

csx.h | 6 identified exports

Public identifier A	Public identifier B
qsc_csx_dispose	qsc_csx_initialize
qsc_csx_set_associated	qsc_csx_store_nonce
qsc_csx_transform	qsc_csx_extended_transform

dilithium.h | 6 identified exports

Public identifier A	Public identifier B
qsc_dilithium_generate_keypair	qsc_dilithium_seeded_generate_keypair
qsc_dilithium_sign	qsc_dilithium_sign_ex
qsc_dilithium_verify	qsc_dilithium_verify_ex

donna128.h | 8 identified exports

Public identifier A	Public identifier B
qsc_donna128_add	qsc_donna128_andh
qsc_donna128_andl	qsc_donna128_multiply
qsc_donna128_or	qsc_donna128_shift_left
qsc_donna128_shift_right	qsc_donna128_subtract

ecdh.h | 4 identified exports

Public identifier A	Public identifier B
qsc_ecdh_public_from_private	qsc_ecdh_generate_keypair
qsc_ecdh_generate_seeded_keypair	qsc_ecdh_key_exchange

ecdhp256.h | 11 identified exports

Public identifier A	Public identifier B

Public identifier A	Public identifier B
qsc_ecdhp256_publickey_from_privatekey	qsc_ecdhp256_scalar_multiply
qsc_ecdhp256_generate_seeded_keypair	qsc_ecdhp256_generate_keypair
qsc_ecdhp256_key_exchange	qsc_p256_public_from_private
qsc_p256_generate_seeded_keypair	qsc_p256_generate_keypair
qsc_p256_key_exchange	qsc_crypto_scalarmult_secp256r1_base
qsc_crypto_scalarmult_secp256r1	

ecdhp384.h | 11 identified exports

Public identifier A	Public identifier B
qsc_ecdhp384_publickey_from_privatekey	qsc_ecdhp384_scalar_multiply
qsc_ecdhp384_generate_seeded_keypair	qsc_ecdhp384_generate_keypair
qsc_ecdhp384_key_exchange	qsc_p384_public_from_private
qsc_p384_generate_seeded_keypair	qsc_p384_generate_keypair
qsc_p384_key_exchange	qsc_crypto_scalarmult_secp384r1_base
qsc_crypto_scalarmult_secp384r1	

ecdhp521.h | 9 identified exports

Public identifier A	Public identifier B
qsc_ecdhp521_publickey_from_privatekey	qsc_ecdhp521_scalar_multiply
qsc_ecdhp521_generate_seeded_keypair	qsc_ecdhp521_generate_keypair
qsc_ecdhp521_key_exchange	qsc_p521_public_from_private
qsc_p521_generate_seeded_keypair	qsc_p521_generate_keypair
qsc_p521_key_exchange	

ecdsa.h | 8 identified exports

Public identifier A	Public identifier B
qsc_ecdsa_publickey_to_sec1	qsc_ecdsa_publickey_from_privatekey
qsc_ecdsa_publickey_from_sec1	qsc_ecdsa_generate_seeded_keypair
qsc_ecdsa_generate_keypair	qsc_ecdsa_sign
qsc_ecdsa_sign_scalar	qsc_ecdsa_verify

ecdsap256.h | 34 identified exports

Public identifier A	Public identifier B
qsc_ecdsap256_publickey_from_privatekey	qsc_ecdsap256_keypair
qsc_ecdsap256_privatekey_validate	qsc_ecdsap256_sign
qsc_ecdsap256_sign_scalar	qsc_ecdsap256_verify
qsc_ecdsap256_verify_prehashed	qsc_ecdsap256_joint_verify_context_initialize
qsc_ecdsap256_joint_verify_context_clear	qsc_ecdsap256_verify_with_joint_context
qsc_ecdsap256_verify_context_initialize	qsc_ecdsap256_verify_context_clear
qsc_ecdsap256_verify_with_context	qsc_ecdsap256_comb_verify_context_initialize
qsc_ecdsap256_comb_verify_context_clear	qsc_ecdsap256_verify_with_comb_context
qsc_ecdsap256_verify_prehashed_with_comb_context	qsc_p256_publickey_from_privatekey
qsc_p256_keypair	qsc_p256_privatekey_validate
qsc_p256_sign	qsc_p256_sign_scalar
qsc_p256_verify	qsc_p256_verify_prehashed
qsc_p256_joint_verify_context_initialize	qsc_p256_joint_verify_context_clear
qsc_p256_verify_with_joint_context	qsc_p256_verify_context_initialize
qsc_p256_verify_context_clear	qsc_p256_verify_with_context
qsc_p256_comb_verify_context_initialize	qsc_p256_comb_verify_context_clear
qsc_p256_verify_with_comb_context	qsc_p256_verify_prehashed_with_comb_context

ecdsap384.h | 28 identified exports

Public identifier A	Public identifier B
qsc_ecdsap384_publickey_from_privatekey	qsc_ecdsap384_keypair
qsc_ecdsap384_sign	qsc_ecdsap384_sign_scalar
qsc_ecdsap384_verify	qsc_ecdsap384_verify_prehashed
qsc_ecdsap384_verify_context_initialize	qsc_ecdsap384_verify_context_clear
qsc_ecdsap384_verify_with_context	qsc_ecdsap384_verify_prehashed_with_context
qsc_ecdsap384_comb_verify_context_initialize	qsc_ecdsap384_comb_verify_context_clear
qsc_ecdsap384_verify_with_comb_context	qsc_ecdsap384_verify_prehashed_with_comb_context
qsc_p384_publickey_from_privatekey	qsc_p384_keypair
qsc_p384_sign	qsc_p384_sign_scalar
qsc_p384_verify	qsc_p384_verify_prehashed
qsc_p384_verify_context_initialize	qsc_p384_verify_context_clear
qsc_p384_verify_with_context	qsc_p384_verify_prehashed_with_context
qsc_p384_comb_verify_context_initialize	qsc_p384_comb_verify_context_clear
qsc_p384_verify_with_comb_context	qsc_p384_verify_prehashed_with_comb_context

ecdsap521.h | 20 identified exports

Public identifier A	Public identifier B
qsc_ecdsap521_publickey_from_privatekey	qsc_ecdsap521_keypair
qsc_ecdsap521_sign	qsc_ecdsap521_sign_scalar
qsc_ecdsap521_verify	qsc_ecdsap521_verify_prehashed
qsc_ecdsap521_comb_verify_context_initialize	qsc_ecdsap521_comb_verify_context_clear
qsc_ecdsap521_verify_with_comb_context	qsc_ecdsap521_verify_prehashed_with_comb_context
qsc_p521_publickey_from_privatekey	qsc_p521_keypair
qsc_p521_sign	qsc_p521_sign_scalar
qsc_p521_verify	qsc_p521_verify_prehashed
qsc_p521_comb_verify_context_initialize	qsc_p521_comb_verify_context_clear
qsc_p521_verify_with_comb_context	qsc_p521_verify_prehashed_with_comb_context

eddh.h | 4 identified exports

Public identifier A	Public identifier B
qsc_eddh_public_from_private	qsc_eddh_generate_keypair
qsc_eddh_generate_seeded_keypair	qsc_eddh_key_exchange

eddh25519.h | 7 identified exports

Public identifier A	Public identifier B
qsc_crypto_scalarmult_curve25519	qsc_crypto_scalarmult_curve25519_ref10
qsc_crypto_scalarmult_curve25519_ref10_base	qsc_x25519_generate_keypair
qsc_x25519_generate_seeded_keypair	qsc_x25519_key_exchange
qsc_x25519_public_from_private	

eddh448.h | 8 identified exports

Public identifier A	Public identifier B
qsc_crypto_scalarmult_curve448_ref10_base	qsc_crypto_scalarmult_curve448_ref10
qsc_crypto_sc448_clamp	qsc_crypto_scalarmult_curve448
qsc_x448_generate_keypair	qsc_x448_generate_seeded_keypair
qsc_x448_key_exchange	qsc_x448_public_from_private

eddsa.h | 4 identified exports

Public identifier A	Public identifier B

Public identifier A	Public identifier B
qsc_eddsa_generate_keypair	qsc_eddsa_generate_seeded_keypair
qsc_eddsa_sign	qsc_eddsa_verify

eddsa25519.h | 4 identified exports

Public identifier A	Public identifier B
qsc_ed25519_generate_keypair	qsc_ed25519_generate_seeded_keypair
qsc_ed25519_sign	qsc_ed25519_verify

eddsa448.h | 4 identified exports

Public identifier A	Public identifier B
qsc_ed448_generate_keypair	qsc_ed448_generate_seeded_keypair
qsc_ed448_sign	qsc_ed448_verify

encoding.h | 25 identified exports

Public identifier A	Public identifier B
qsc_encoding_base64_stream_initialize	qsc_encoding_base64_stream_encode_update
qsc_encoding_base64_stream_encode_final	qsc_encoding_base64_stream_decode_update
qsc_encoding_base64_stream_decode_final	qsc_encoding_base64_decode
qsc_encoding_base64_decoded_size	qsc_encoding_base64_encode
qsc_encoding_base64_encoded_size	qsc_encoding_base64_is_valid_char
qsc_encoding_ber_decode_element	qsc_encoding_ber_decode_length
qsc_encoding_ber_decode_tag	qsc_encoding_ber_encode_element
qsc_encoding_ber_encode_length	qsc_encoding_ber_encode_tag
qsc_encoding_ber_free_element	qsc_encoding_der_validate
qsc_encoding_der_decode_element	qsc_encoding_der_encode_element
qsc_encoding_hex_decode	qsc_encoding_hex_encode
qsc_encoding_pem_decode	qsc_encoding_pem_encode
qsc_encoding_tests	

etr.h | 3 identified exports

Public identifier A	Public identifier B
qsc_etr_dispose	qsc_etr_generate
qsc_etr_initialize	

event.h | 6 identified exports

Public identifier A	Public identifier B
qsc_event_clear_listener	qsc_event_get_callback
qsc_event_dispose	qsc_event_listener_name_exists
qsc_event_initialize	qsc_event_register

falcon.h | 3 identified exports

Public identifier A	Public identifier B
qsc_falcon_generate_keypair	qsc_falcon_sign
qsc_falcon_verify	

falcon1024.h | 3 identified exports

Public identifier A	Public identifier B
qsc_falcon1024_generate_keypair	qsc_falcon1024_sign
qsc_falcon1024_verify	

falcon512.h | 3 identified exports

Public identifier A	Public identifier B
qsc_falcon512_generate_keypair	qsc_falcon512_sign
qsc_falcon512_verify	

fileutils.h | 33 identified exports

Public identifier A	Public identifier B
qsc_fileutils_append_to_file	qsc_fileutils_copy_file_to_object
qsc_fileutils_copy_file_to_stream	qsc_fileutils_copy_object_to_file
qsc_fileutils_copy_stream_to_file	qsc_fileutils_create
qsc_fileutils_delete	qsc_fileutils_erase
qsc_fileutils_flush	qsc_fileutils_file_copy
qsc_fileutils_get_access	qsc_fileutils_get_directory
qsc_fileutils_get_extension	qsc_fileutils_get_name
qsc_fileutils_get_line	qsc_fileutils_get_size
qsc_fileutils_get_working_directory	qsc_fileutils_list_files
qsc_fileutils_close	qsc_fileutils_exists
qsc_fileutils_open	qsc_fileutils_read
qsc_fileutils_safe_read	qsc_fileutils_safe_write
qsc_fileutils_seekto	qsc_fileutils_read_line
qsc_fileutils_sync	qsc_fileutils_truncate_file
qsc_fileutils_valid_path	qsc_fileutils_write
qsc_fileutils_write_line	qsc_fileutils_zeroise
qsc_fileutils_test	

folderutils.h | 10 identified exports

Public identifier A	Public identifier B
qsc_folderutils_append_delimiter	qsc_folderutils_create_directory
qsc_folderutils_create_directory_tree	qsc_folderutils_delete_directory
qsc_folderutils_directory_exists	qsc_folderutils_directory_list
qsc_folderutils_enumerate	qsc_folderutils_get_directory
qsc_folderutils_directory_has_delimiter	qsc_folderutils_test

hcg.h | 4 identified exports

Public identifier A	Public identifier B
qsc_hcg_dispose	qsc_hcg_initialize
qsc_hcg_generate	qsc_hcg_update

hqc.h | 4 identified exports

Public identifier A	Public identifier B
qsc_hqc_decapsulate	qsc_hqc_encapsulate
qsc_hqc_generate_keypair	qsc_hqc_generate_seeded_keypair

hqc_multi.h | 15 identified exports

Public identifier A	Public identifier B
qsc_hqc1_generate_keypair	qsc_hqc1_generate_seeded_keypair
qsc_hqc1_encapsulate	qsc_hqc1_seeded_encapsulate
qsc_hqc1_decapsulate	qsc_hqc3_generate_keypair
qsc_hqc3_generate_seeded_keypair	qsc_hqc3_encapsulate
qsc_hqc3_seeded_encapsulate	qsc_hqc3_decapsulate
qsc_hqc5_generate_keypair	qsc_hqc5_generate_seeded_keypair
qsc_hqc5_encapsulate	qsc_hqc5_seeded_encapsulate
qsc_hqc5_decapsulate	

hwatecc.h | 4 identified exports

Public identifier A	Public identifier B
qsc_hw_atecc_initialize	qsc_hw_atecc_get_operations
qsc_hw_atecc_locator	qsc_hw_atecc_dispose

hwncrypt.h | 4 identified exports

Public identifier A	Public identifier B
qsc_hw_ncrypt_initialize	qsc_hw_ncrypt_get_operations
qsc_hw_ncrypt_algorithm_available	qsc_hw_ncrypt_dispose

hwncrypttls.h | 7 identified exports

Public identifier A	Public identifier B
qsc_hw_ncrypt_tls_signer_initialize	qsc_hw_ncrypt_tls_sign_callback
qsc_hw_ncrypt_tls_client_config_set_identity	qsc_hw_ncrypt_tls_server_config_set_identity
qsc_hw_ncrypt_tls_server_config_add_identity	qsc_hw_ncrypt_x509_sign_callback
qsc_hw_ncrypt_x509_csr_sign	

hwpkcs11.h | 16 identified exports

Public identifier A	Public identifier B
qsc_hw_pkcs11_key_find	qsc_hw_pkcs11_key_generate
qsc_hw_pkcs11_key_public	qsc_hw_pkcs11_key_sign
qsc_hw_pkcs11_key_agree	qsc_hw_pkcs11_key_release
qsc_hw_pkcs11_module_open	qsc_hw_pkcs11_module_close
qsc_hw_pkcs11_slot_list	qsc_hw_pkcs11_token_info_get
qsc_hw_pkcs11_mechanism_list	qsc_hw_pkcs11_mechanism_info_get
qsc_hw_pkcs11_session_open	qsc_hw_pkcs11_session_login
qsc_hw_pkcs11_session_logout	qsc_hw_pkcs11_session_close

hwpkcs11tls.h | 7 identified exports

Public identifier A	Public identifier B
qsc_hw_pkcs11_tls_signer_initialize	qsc_hw_pkcs11_tls_sign_callback
qsc_hw_pkcs11_tls_client_config_set_identity	qsc_hw_pkcs11_tls_server_config_set_identity
qsc_hw_pkcs11_tls_server_config_add_identity	qsc_hw_pkcs11_x509_sign_callback
qsc_hw_pkcs11_x509_csr_sign	

hwplatform.h | 4 identified exports

Public identifier A	Public identifier B

Public identifier A	Public identifier B
qsc_hw_platform_initialize	qsc_hw_platform_get_operations
qsc_hw_platform_locator	qsc_hw_platform_dispose

hwprovider.h | 13 identified exports

Public identifier A	Public identifier B
qsc_hw_provider_open	qsc_hw_provider_capabilities
qsc_hw_key_open	qsc_hw_key_generate
qsc_hw_key_get_info	qsc_hw_key_public
qsc_hw_key_sign	qsc_hw_key_agree
qsc_hw_key_attest	qsc_hw_provider_random
qsc_hw_provider_invalidate	qsc_hw_key_release
qsc_hw_provider_close	

hwpsa.h | 4 identified exports

Public identifier A	Public identifier B
qsc_hw_psa_initialize	qsc_hw_psa_get_operations
qsc_hw_psa_locator	qsc_hw_psa_dispose

hwpsanative.h | 1 identified exports

Public identifier A	Public identifier B
qsc_hw_psa_native_initialize	

hwse050.h | 1 identified exports

Public identifier A	Public identifier B
qsc_hw_se050_preflight	

hwtpm.h | 4 identified exports

Public identifier A	Public identifier B
qsc_hw_tpm_initialize	qsc_hw_tpm_get_operations
qsc_hw_tpm_get_ak_identity	qsc_hw_tpm_dispose

hwtpmquote.h | 2 identified exports

Public identifier A	Public identifier B
qsc_hw_tpm_quote_verify	qsc_hw_tpm_quote_envelope_verify

intutils.h | 54 identified exports

Public identifier A	Public identifier B
qsc_intutils_are_equal	qsc_intutils_are_equal8
qsc_intutils_be8to16	qsc_intutils_be8to32
qsc_intutils_be8to64	qsc_intutils_be16to8
qsc_intutils_be32to8	qsc_intutils_be64to8
qsc_intutils_be8increment	qsc_intutils_be8increment_rfc3686
qsc_intutils_bit_reverse	qsc_intutils_bit_reverse_u64
qsc_intutils_bit_reverse_u32	qsc_intutils_bit_reverse_u16
qsc_intutils_bswap32	qsc_intutils_bswap64

Public identifier A	Public identifier B
qsc_intutils_calculate_abs	qsc_intutils_calculate_exp
qsc_intutils_calculate_fabs	qsc_intutils_calculate_log
qsc_intutils_calculate_sqrt	qsc_intutils_clear8
qsc_intutils_clear16	qsc_intutils_clear32
qsc_intutils_clear64	qsc_intutils_cmov
qsc_intutils_expand_mask	qsc_intutils_is_gte
qsc_intutils_is_power_of_two	qsc_intutils_next_power_of_2
qsc_intutils_hex_to_bin	qsc_intutils_hex_to_bin_checked
qsc_intutils_bin_to_hex	qsc_intutils_le8increment
qsc_intutils_leincrement_x128	qsc_intutils_leincrement_x512
qsc_intutils_le8to16	qsc_intutils_le8to32
qsc_intutils_le8to64	qsc_intutils_le16to8
qsc_intutils_le32to8	qsc_intutils_le64to8
qsc_intutils_lsb_is_set	qsc_intutils_max
qsc_intutils_min	qsc_intutils_popcount32
qsc_intutils_popcount64	qsc_intutils_reverse_bytes_x128
qsc_intutils_reverse_bytes_x512	qsc_intutils_rotl32
qsc_intutils_rotl64	qsc_intutils_rotr32
qsc_intutils_rotr64	qsc_intutils_verify

ipinfo.h | 36 identified exports

Public identifier A	Public identifier B
qsc_ipinfo_ipv4_address_any	qsc_ipinfo_ipv4_address_clear
qsc_ipinfo_ipv4_address_from_array	qsc_ipinfo_ipv4_address_from_bytes
qsc_ipinfo_ipv4_address_from_string	qsc_ipinfo_ipv4_address_is_equal
qsc_ipinfo_ipv4_address_is_routable	qsc_ipinfo_ipv4_address_is_valid
qsc_ipinfo_ipv4_address_string_is_valid	qsc_ipinfo_ipv4_address_is_zeroed
qsc_ipinfo_ipv4_address_loopback	qsc_ipinfo_ipv4_address_get_mask
qsc_ipinfo_ipv4_address_get_cidr_mask	qsc_ipinfo_ipv4_address_to_array
qsc_ipinfo_ipv4_address_to_string	qsc_ipinfo_ipv4_array_to_string
qsc_ipinfo_ipv6_address_type	qsc_ipinfo_ipv6_address_any
qsc_ipinfo_ipv6_address_clear	qsc_ipinfo_ipv6_address_from_array
qsc_ipinfo_ipv6_address_from_string	qsc_ipinfo_ipv6_address_is_equal
qsc_ipinfo_ipv6_address_is_routable	qsc_ipinfo_ipv6_address_is_valid
qsc_ipinfo_ipv6_address_string_is_valid	qsc_ipinfo_ipv6_address_is_zeroed
qsc_ipinfo_ipv6_address_loopback	qsc_ipinfo_ipv6_address_get_mask
qsc_ipinfo_ipv6_address_get_cidr_mask	qsc_ipinfo_ipv6_address_to_array
qsc_ipinfo_ipv6_address_to_string	qsc_ipinfo_ipv6_array_to_string
qsc_ipinfo_ipv4_mask_to_cidr	qsc_ipinfo_ipv4_cidr_to_mask
qsc_ipinfo_ipv4_prefix_contains	qsc_ipinfo_ipv6_prefix_contains

kyber.h | 4 identified exports

Public identifier A	Public identifier B
qsc_kyber_decapsulate	qsc_kyber_encapsulate
qsc_kyber_generate_keypair	qsc_kyber_generate_seeded_keypair

kyber1024.h | 5 identified exports

Public identifier A	Public identifier B
qsc_kyber1024_decapsulate	qsc_kyber1024_encapsulate
qsc_kyber1024_seeded_encapsulate	qsc_kyber1024_generate_keypair
qsc_kyber1024_generate_seeded_keypair	

kyber1280.h | 5 identified exports

Public identifier A	Public identifier B
qsc_kyber1280_decapsulate	qsc_kyber1280_encapsulate
qsc_kyber1280_seeded_encapsulate	qsc_kyber1280_generate_keypair
qsc_kyber1280_generate_seeded_keypair	

kyber512.h | 5 identified exports

Public identifier A	Public identifier B
qsc_kyber512_decapsulate	qsc_kyber512_encapsulate
qsc_kyber512_seeded_encapsulate	qsc_kyber512_generate_keypair
qsc_kyber512_generate_seeded_keypair	

kyber768.h | 5 identified exports

Public identifier A	Public identifier B
qsc_kyber768_decapsulate	qsc_kyber768_encapsulate
qsc_kyber768_seeded_encapsulate	qsc_kyber768_generate_keypair
qsc_kyber768_generate_seeded_keypair	

list.h | 19 identified exports

Public identifier A	Public identifier B
qsc_list_add	qsc_list_reserve
qsc_list_trim	qsc_list_deserialize_sized
qsc_list_serialize_sized	qsc_list_copy
qsc_list_count	qsc_list_deserialize
qsc_list_dispose	qsc_list_initialize
qsc_list_empty	qsc_list_full
qsc_list_item	qsc_list_rshuffle
qsc_list_remove	qsc_list_serialize
qsc_list_size	qsc_list_sort
qsc_list_self_test	

mceliece.h | 3 identified exports

Public identifier A	Public identifier B
qsc_mceliece_decapsulate	qsc_mceliece_encapsulate
qsc_mceliece_generate_keypair	

mceliece_multi.h | 48 identified exports

Public identifier A	Public identifier B
qsc_mceliece_460896pc_decapsulate	qsc_mceliece_460896pc_encapsulate
qsc_mceliece_460896pc_generate_keypair	qsc_mceliece_6688128pc_decapsulate
qsc_mceliece_6688128pc_encapsulate	qsc_mceliece_6688128pc_generate_keypair
qsc_mceliece_6960119pc_decapsulate	qsc_mceliece_6960119pc_encapsulate
qsc_mceliece_6960119pc_generate_keypair	qsc_mceliece_8192128pc_decapsulate
qsc_mceliece_8192128pc_encapsulate	qsc_mceliece_8192128pc_generate_keypair
qsc_mceliece_460896_decapsulate	qsc_mceliece_460896_encapsulate
qsc_mceliece_460896_generate_keypair	qsc_mceliece_6688128_decapsulate
qsc_mceliece_6688128_encapsulate	qsc_mceliece_6688128_generate_keypair
qsc_mceliece_6960119_decapsulate	qsc_mceliece_6960119_encapsulate
qsc_mceliece_6960119_generate_keypair	qsc_mceliece_8192128_decapsulate

Public identifier A	Public identifier B
qsc_mceliece_8192128_encapsulate	qsc_mceliece_8192128_generate_keypair
qsc_mceliece_460896f_decapsulate	qsc_mceliece_460896f_encapsulate
qsc_mceliece_460896f_generate_keypair	qsc_mceliece_460896pcf_decapsulate
qsc_mceliece_460896pcf_encapsulate	qsc_mceliece_460896pcf_generate_keypair
qsc_mceliece_6688128f_decapsulate	qsc_mceliece_6688128f_encapsulate
qsc_mceliece_6688128f_generate_keypair	qsc_mceliece_6688128pcf_decapsulate
qsc_mceliece_6688128pcf_encapsulate	qsc_mceliece_6688128pcf_generate_keypair
qsc_mceliece_6960119f_decapsulate	qsc_mceliece_6960119f_encapsulate
qsc_mceliece_6960119f_generate_keypair	qsc_mceliece_6960119pcf_decapsulate
qsc_mceliece_6960119pcf_encapsulate	qsc_mceliece_6960119pcf_generate_keypair
qsc_mceliece_8192128f_decapsulate	qsc_mceliece_8192128f_encapsulate
qsc_mceliece_8192128f_generate_keypair	qsc_mceliece_8192128pcf_decapsulate
qsc_mceliece_8192128pcf_encapsulate	qsc_mceliece_8192128pcf_generate_keypair

memutils.h | 37 identified exports

Public identifier A	Public identifier B
qsc_memutils_flush_cache_line	qsc_memutils_prefetch_l1
qsc_memutils_prefetch_l2	qsc_memutils_prefetch_l3
qsc_memutils_calloc	qsc_memutils_malloc
qsc_memutils_page_size	qsc_memutils_realloc
qsc_memutils_alloc_free	qsc_memutils_aligned_alloc
qsc_memutils_aligned_realloc	qsc_memutils_aligned_resize
qsc_memutils_aligned_free	qsc_memutils_clear
qsc_memutils_array_uniform	qsc_memutils_are_equal
qsc_memutils_are_equal_128	qsc_memutils_are_equal_256
qsc_memutils_are_equal_512	qsc_memutils_copy
qsc_memutils_clmulepi64_si128	qsc_memutils_clmulepi64_si256_avx
qsc_memutils_clmulepi64_si256	qsc_memutils_greater_than_be128
qsc_memutils_greater_than_be256	qsc_memutils_greater_than_be512
qsc_memutils_greater_than_le128	qsc_memutils_greater_than_le256
qsc_memutils_greater_than_le512	qsc_memutils_move
qsc_memutils_secure_erase	qsc_memutils_secure_free
qsc_memutils_secure_malloc	qsc_memutils_set_value
qsc_memutils_xor	qsc_memutils_xorv
qsc_memutils_zeroed	

mldsa.h | 18 identified exports

Public identifier A	Public identifier B
qsc_mldsa44_generate_keypair	qsc_mldsa44_seeded_generate_keypair
qsc_mldsa44_sign	qsc_mldsa44_sign_ex
qsc_mldsa44_verify	qsc_mldsa44_verify_ex
qsc_mldsa65_generate_keypair	qsc_mldsa65_seeded_generate_keypair
qsc_mldsa65_sign	qsc_mldsa65_sign_ex
qsc_mldsa65_verify	qsc_mldsa65_verify_ex
qsc_mldsa87_generate_keypair	qsc_mldsa87_seeded_generate_keypair
qsc_mldsa87_sign	qsc_mldsa87_sign_ex
qsc_mldsa87_verify	qsc_mldsa87_verify_ex

netutils.h | 14 identified exports

Public identifier A	Public identifier B
qsc_netutils_get_adaptor_info	qsc_netutils_get_mac_address
qsc_netutils_atoi	qsc_netutils_get_domain_name
qsc_netutils_get_host_name	qsc_netutils_get_name_from_ipv4_address
qsc_netutils_get_ipv4_address	qsc_netutils_get_ipv6_address

Public identifier A	Public identifier B
qsc_netutils_get_ipv4_info	qsc_netutils_get_ipv6_info
qsc_netutils_get_peer_name	qsc_netutils_get_socket_name
qsc_netutils_port_name_to_number	qsc_netutils_values_print

oid.h | 10 identified exports

Public identifier A	Public identifier B
qsc_oid_registry_count	qsc_oid_registry_at
qsc_oid_get_entry	qsc_oid_identify
qsc_oid_equals_id	qsc_oid_get_encoded
qsc_oid_to_asn1	qsc_oid_get_dotted
qsc_oid_get_name	qsc_oid_get_name_from_oid

pbmac1.h | 8 identified exports

Public identifier A	Public identifier B
qsc_pbmac1_compute	qsc_pbmac1_derive_key
qsc_pbmac1_dispose	qsc_pbmac1_finalize
qsc_pbmac1_initialize	qsc_pbmac1_mac_size
qsc_pbmac1_update	qsc_pbmac1_verify

poly1305.h | 8 identified exports

Public identifier A	Public identifier B
qsc_poly1305_blockupdate	qsc_poly1305_compute
qsc_poly1305_dispose	qsc_poly1305_finalize
qsc_poly1305_initialize	qsc_poly1305_reset
qsc_poly1305_update	qsc_poly1305_verify

qmac.h | 5 identified exports

Public identifier A	Public identifier B
qsc_qmac_compute	qsc_qmac_dispose
qsc_qmac_finalize	qsc_qmac_initialize
qsc_qmac_update	

qsort.h | 4 identified exports

Public identifier A	Public identifier B
qsc_qsort_sort_i8	qsc_qsort_sort_i16
qsc_qsort_sort_i32	qsc_qsort_sort_i64

queue.h | 14 identified exports

Public identifier A	Public identifier B
qsc_queue_dispose	qsc_queue_flush
qsc_queue_initialize	qsc_queue_items
qsc_queue_full	qsc_queue_empty
qsc_queue_pop	qsc_queue_push
qsc_queue_try_push	qsc_queue_try_pop
qsc_queue_push_batch	qsc_queue_pop_batch
qsc_queue_flush_bounded	qsc_queue_self_test

rcs.h | 6 identified exports

Public identifier A	Public identifier B
qsc_rcs_dispose	qsc_rcs_initialize
qsc_rcs_set_associated	qsc_rcs_store_nonce
qsc_rcs_transform	qsc_rcs_extended_transform

rdp.h | 5 identified exports

Public identifier A	Public identifier B
qsc_rdrand_available	qsc_rdp_generate
qsc_rdp_uint16	qsc_rdp_uint32
qsc_rdp_uint64	

scb.h | 4 identified exports

Public identifier A	Public identifier B
qsc_scb_dispose	qsc_scb_initialize
qsc_scb_generate	qsc_scb_update

secrand.h | 24 identified exports

Public identifier A	Public identifier B
qsc_secrand_next_char	qsc_secrand_next_uchar
qsc_secrand_next_double	qsc_secrand_next_int16
qsc_secrand_next_int16_max	qsc_secrand_next_int16_maxmin
qsc_secrand_next_uint16	qsc_secrand_next_uint16_max
qsc_secrand_next_uint16_maxmin	qsc_secrand_next_int32
qsc_secrand_next_int32_max	qsc_secrand_next_int32_maxmin
qsc_secrand_next_uint32	qsc_secrand_next_uint32_max
qsc_secrand_next_uint32_maxmin	qsc_secrand_next_int64
qsc_secrand_next_int64_max	qsc_secrand_next_int64_maxmin
qsc_secrand_next_uint64	qsc_secrand_next_uint64_max
qsc_secrand_next_uint64_maxmin	qsc_secrand_dispose
qsc_secrand_initialize	qsc_secrand_generate

selftest.h | 7 identified exports

Public identifier A	Public identifier B
qsc_selftest_aes_test	qsc_selftest_chacha_test
qsc_selftest_csx_test	qsc_selftest_rcs_test
qsc_selftest_sha2_test	qsc_selftest_sha3_test
qsc_selftest_symmetric_run	

sha2.h | 38 identified exports

Public identifier A	Public identifier B
qsc_sha256_compute	qsc_sha256_dispose
qsc_sha256_finalize	qsc_sha256_initialize
qsc_sha256_permute	qsc_sha256_update
qsc_sha384_compute	qsc_sha384_dispose
qsc_sha384_finalize	qsc_sha384_initialize
qsc_sha384_update	qsc_sha512_compute
qsc_sha512_dispose	qsc_sha512_finalize
qsc_sha512_initialize	qsc_sha512_permute
qsc_sha512_update	qsc_hmac256_compute

Public identifier A	Public identifier B
qsc_hmac256_dispose	qsc_hmac256_finalize
qsc_hmac256_initialize	qsc_hmac256_update
qsc_hmac384_compute	qsc_hmac384_dispose
qsc_hmac384_finalize	qsc_hmac384_initialize
qsc_hmac384_update	qsc_hmac512_compute
qsc_hmac512_dispose	qsc_hmac512_finalize
qsc_hmac512_initialize	qsc_hmac512_update
qsc_hkdf256_expand	qsc_hkdf256_extract
qsc_hkdf384_expand	qsc_hkdf384_extract
qsc_hkdf512_expand	qsc_hkdf512_extract

sha3.h | 52 identified exports

Public identifier A	Public identifier B
qsc_keccak_absorb	qsc_keccak_absorb_custom
qsc_keccak_absorb_key_custom	qsc_keccak_dispose
qsc_keccak_finalize	qsc_keccak_incremental_absorb
qsc_keccak_incremental_finalize	qsc_keccak_incremental_squeeze
qsc_keccak_permute	qsc_keccak_permute_p1600c
qsc_keccak_permute_p1600u	qsc_keccak_squeezeblocks
qsc_keccak_initialize_state	qsc_keccak_update
qsc_sha3_compute128	qsc_sha3_compute256
qsc_sha3_compute512	qsc_sha3_finalize
qsc_sha3_initialize	qsc_sha3_update
qsc_shake128_compute	qsc_shake256_compute
qsc_shake512_compute	qsc_shake_initialize
qsc_shake_squeezeblocks	qsc_cshake128_compute
qsc_cshake256_compute	qsc_cshake512_compute
qsc_cshake_initialize	qsc_cshake_squeezeblocks
qsc_cshake_update	qsc_kmac128_compute
qsc_kmac_xof128_compute	qsc_kmac_xof256_compute
qsc_kmac_xof512_compute	qsc_kmac256_compute
qsc_kmac512_compute	qsc_kmac_update
qsc_kmac_finalize	qsc_kmac_initialize
qsc_shake_128x4	qsc_shake_256x4
qsc_shake_512x4	qsc_shake_128x8
qsc_shake_256x8	qsc_shake_512x8
qsc_kmac_128x4	qsc_kmac_256x4
qsc_kmac_512x4	qsc_kmac_128x8
qsc_kmac_256x8	qsc_kmac_512x8

slhdsa.h | 48 identified exports

Public identifier A	Public identifier B
qsc_slhdsa_shake128s_generate_keypair	qsc_slhdsa_shake128s_generate_seeded_keypair
qsc_slhdsa_shake128s_sign	qsc_slhdsa_shake128s_sign_deterministic
qsc_slhdsa_shake128s_sign_prehash	qsc_slhdsa_shake128s_sign_prehash_deterministic
qsc_slhdsa_shake128s_verify_prehash	qsc_slhdsa_shake128s_verify
qsc_slhdsa_shake128f_generate_keypair	qsc_slhdsa_shake128f_generate_seeded_keypair
qsc_slhdsa_shake128f_sign	qsc_slhdsa_shake128f_sign_deterministic
qsc_slhdsa_shake128f_sign_prehash	qsc_slhdsa_shake128f_sign_prehash_deterministic
qsc_slhdsa_shake128f_verify_prehash	qsc_slhdsa_shake128f_verify
qsc_slhdsa_shake192s_generate_keypair	qsc_slhdsa_shake192s_generate_seeded_keypair
qsc_slhdsa_shake192s_sign	qsc_slhdsa_shake192s_sign_deterministic
qsc_slhdsa_shake192s_sign_prehash	qsc_slhdsa_shake192s_sign_prehash_deterministic
qsc_slhdsa_shake192s_verify_prehash	qsc_slhdsa_shake192s_verify
qsc_slhdsa_shake192f_generate_keypair	qsc_slhdsa_shake192f_generate_seeded_keypair
qsc_slhdsa_shake192f_sign	qsc_slhdsa_shake192f_sign_deterministic
qsc_slhdsa_shake192f_sign_prehash	qsc_slhdsa_shake192f_sign_prehash_deterministic

Public identifier A	Public identifier B
qsc_slhdsa_shake192f_verify_prehash	qsc_slhdsa_shake192f_verify
qsc_slhdsa_shake256s_generate_keypair	qsc_slhdsa_shake256s_generate_seeded_keypair
qsc_slhdsa_shake256s_sign	qsc_slhdsa_shake256s_sign_deterministic
qsc_slhdsa_shake256s_sign_prehash	qsc_slhdsa_shake256s_sign_prehash_deterministic
qsc_slhdsa_shake256s_verify_prehash	qsc_slhdsa_shake256s_verify
qsc_slhdsa_shake256f_generate_keypair	qsc_slhdsa_shake256f_generate_seeded_keypair
qsc_slhdsa_shake256f_sign	qsc_slhdsa_shake256f_sign_deterministic
qsc_slhdsa_shake256f_sign_prehash	qsc_slhdsa_shake256f_sign_prehash_deterministic
qsc_slhdsa_shake256f_verify_prehash	qsc_slhdsa_shake256f_verify

socketbase.h | 61 identified exports

Public identifier A	Public identifier B
qsc_socket_exception_callback	qsc_socket_receive_async_callback
qsc_socket_receive_poll_callback	qsc_socket_ipv4_valid_address
qsc_socket_ipv6_valid_address	qsc_socket_is_blocking
qsc_socket_set_blocking	qsc_socket_is_connected
qsc_socket_accept	qsc_socket_attach
qsc_socket_bind	qsc_socket_bind_ipv4
qsc_socket_bind_ipv6	qsc_socket_clear_socket
qsc_socket_close_socket	qsc_socket_connect
qsc_socket_connect_ipv4	qsc_socket_connect_ipv6
qsc_socket_create	qsc_socket_listen
qsc_socket_max_send_buffer_size	qsc_socket_peek
qsc_socket_receive	qsc_socket_receive_all
qsc_socket_receive_progress	qsc_socket_receive_vectored
qsc_socket_receive_async	qsc_socket_receive_from
qsc_socket_receive_poll	qsc_socket_receive_timeout
qsc_socket_send	qsc_socket_send_to
qsc_socket_send_all	qsc_socket_send_progress
qsc_socket_send_vectored	qsc_socket_shut_down
qsc_socket_get_option	qsc_socket_error_to_string
qsc_socket_get_last_error	qsc_socket_ioctl
qsc_socket_receive_ready	qsc_socket_send_ready
qsc_socket_set_last_error	qsc_socket_shut_down_sockets
qsc_socket_set_option	qsc_socket_start_sockets
qsc_socket_event_loop_create	qsc_socket_event_loop_register
qsc_socket_event_loop_modify	qsc_socket_event_loop_unregister
qsc_socket_event_loop_wait	qsc_socket_event_loop_wake
qsc_socket_event_loop_destroy	qsc_socket_iocp_create
qsc_socket_iocp_attach	qsc_socket_iocp_submit_receive
qsc_socket_iocp_submit_send	qsc_socket_iocp_wait
qsc_socket_iocp_wake	qsc_socket_iocp_cancel
qsc_socket_iocp_destroy	

socketclient.h | 12 identified exports

Public identifier A	Public identifier B
qsc_socket_client_address_family	qsc_socket_client_socket_protocol
qsc_socket_client_connect_host	qsc_socket_client_connect_ipv4
qsc_socket_client_connect_ipv6	qsc_socket_client_socket_transport
qsc_socket_client_initialize	qsc_socket_client_receive
qsc_socket_client_receive_from	qsc_socket_client_send
qsc_socket_client_send_to	qsc_socket_client_shut_down

socketserver.h | 17 identified exports

Public identifier A	Public identifier B
qsc_socket_server_accept_callback	qsc_socket_server_error_callback
qsc_socket_server_address_family	qsc_socket_server_socket_protocol
qsc_socket_server_socket_transport	qsc_socket_server_async_dispose
qsc_socket_server_async_stop	qsc_socket_server_close_socket
qsc_socket_server_initialize	qsc_socket_server_listen
qsc_socket_server_listen_ipv4	qsc_socket_server_listen_ipv6
qsc_socket_server_listen_async	qsc_socket_server_listen_async_ipv4
qsc_socket_server_listen_async_ipv6	qsc_socket_server_set_options
qsc_socket_server_shut_down	

sphincsplus.h | 7 identified exports

Public identifier A	Public identifier B
qsc_sphincsplus_generate_keypair	qsc_sphincsplus_generate_seeded_keypair
qsc_sphincsplus_sign	qsc_sphincsplus_sign_ex
qsc_sphincsplus_seeded_sign_ex	qsc_sphincsplus_verify
qsc_sphincsplus_verify_ex	

stringutils.h | 41 identified exports

Public identifier A	Public identifier B
qsc_stringutils_formatting_count	qsc_stringutils_formatting_filter
qsc_stringutils_add_line_breaks	qsc_stringutils_remove_line_breaks
qsc_stringutils_clear_string	qsc_stringutils_clear_substring
qsc_stringutils_compare_strings	qsc_stringutils_concat_strings
qsc_stringutils_concat_and_copy	qsc_stringutils_copy_substring
qsc_stringutils_copy_string	qsc_stringutils_find_char
qsc_stringutils_find_string	qsc_stringutils_byte_to_hex
qsc_stringutils_hex_to_byte	qsc_stringutils_insert_string
qsc_stringutils_is_empty	qsc_stringutils_is_hex
qsc_stringutils_is_numeric	qsc_stringutils_register_string
qsc_stringutils_remove_null_chars	qsc_stringutils_reverse_find_string
qsc_stringutils_reverse_sub_string	qsc_stringutils_string_comparison
qsc_stringutils_string_contains	qsc_stringutils_strings_equal
qsc_stringutils_split_string	qsc_stringutils_split_strings
qsc_stringutils_sub_string	qsc_stringutils_string_to_int
qsc_stringutils_string_size	qsc_stringutils_int_to_string
qsc_stringutils_uint32_to_string	qsc_stringutils_int64_to_string
qsc_stringutils_uint64_to_string	qsc_stringutils_to_lowercase
qsc_stringutils_to_uppercase	qsc_stringutils_trim_newline
qsc_stringutils_trim_spaces	qsc_stringutils_whitespace_count
qsc_stringutils_whitespace_filter	

sysutils.h | 14 identified exports

Public identifier A	Public identifier B
qsc_sysutils_computer_name	qsc_sysutils_computer_name_bounded
qsc_sysutils_user_name_bounded	qsc_sysutils_cpu_count
qsc_sysutils_drive_space	qsc_sysutils_memory_statistics
qsc_sysutils_get_os_drive_letter	qsc_sysutils_process_id
qsc_sysutils_rdtsc_available	qsc_sysutils_user_name
qsc_sysutils_system_uptime	qsc_sysutils_system_timestamp
qsc_sysutils_user_identity	qsc_system_values_print

threadpool.h | 10 identified exports

Public identifier A	Public identifier B
qsc_threadpool_add_task	qsc_threadpool_destroy
qsc_threadpool_initialize	qsc_threadpool_sort
qsc_threadpool_thread_active	qsc_threadpool_remove_task
qsc_threadpool_worker_create	qsc_threadpool_worker_submit
qsc_threadpool_worker_wait	qsc_threadpool_worker_destroy

timerex.h | 8 identified exports

Public identifier A	Public identifier B
qsc_timerex_get_date	qsc_timerex_get_datetime
qsc_timerex_get_time	qsc_timerex_stopwatch_start
qsc_timerex_stopwatch_elapsed	qsc_timerex_stopwatch_elapsed_microseconds
qsc_timerex_stopwatch_elapsed_nanoseconds	qsc_timerex_print_values

timestamp.h | 14 identified exports

Public identifier A	Public identifier B
qsc_timestamp_current_date	qsc_timestamp_current_datetime
qsc_timestamp_current_time	qsc_timestamp_epochtime_seconds
qsc_timestamp_epochtime_milliseconds	qsc_timestamp_epochtime_microseconds
qsc_timestamp_time_struct_to_string	qsc_timestamp_string_to_time_struct_checked
qsc_timestamp_string_to_time_struct	qsc_timestamp_datetime_seconds_remaining
qsc_timestamp_datetime_to_seconds	qsc_timestamp_datetime_utc
qsc_timestamp_seconds_to_datetime	qsc_timestamp_print_values

tlsalert.h | 8 identified exports

Public identifier A	Public identifier B
qsc_tls_alert_decode	qsc_tls_alert_encode
qsc_tls_alert_encode_record	qsc_tls_alert_from_status
qsc_tls_alert_is_close_notify	qsc_tls_alert_is_fatal_level
qsc_tls_alert_is_valid	qsc_tls_alert_to_string

tlscert.h | 14 identified exports

Public identifier A	Public identifier B
qsc_tls_certificate_decode_message	qsc_tls_certificate_encode_message
qsc_tls_certificate_interface_initialize	qsc_tls_certificate_interface_is_valid
qsc_tls_x509_context_initialize	qsc_tls_x509_context_clone
qsc_tls_certificate_interface_get_last_alert	qsc_tls_certificate_interface_initialize_qsc_x509
qsc_tls_certificate_request_decode	qsc_tls_certificate_request_encode
qsc_tls_x509_alert_from_verify_status	qsc_tls_x509_signature_algorithm_from_tls
qsc_tls_x509_validate_chain	qsc_tls_x509_verify_certificate_verify

tlscertx509.h | 2 identified exports

Public identifier A	Public identifier B
qsc_tls_cert_x509_state_initialize	qsc_tls_cert_x509_bind

tlsclient.h | 8 identified exports

Public identifier A	Public identifier B
qsc_tls_client_config_set_certificate_interface	qsc_tls_client_config_set_local_certificate

Public identifier A	Public identifier B
qsc_tls_client_initialize	qsc_tls_client_dispose
qsc_tls_client_send_hello	qsc_tls_client_process_record
qsc_tls_client_is_handshake_complete	qsc_tls_client_get_negotiated_cipher_suite

tlscodec.h | 23 identified exports

Public identifier A	Public identifier B
qsc_tls_codec_write_u8	qsc_tls_codec_write_u16
qsc_tls_codec_write_u24	qsc_tls_codec_write_u32
qsc_tls_codec_write_bytes	qsc_tls_codec_write_vector8
qsc_tls_codec_write_vector16	qsc_tls_codec_write_vector24
qsc_tls_codec_read_u8	qsc_tls_codec_read_u16
qsc_tls_codec_read_u24	qsc_tls_codec_read_u32
qsc_tls_codec_read_bytes	qsc_tls_codec_read_vector8_span
qsc_tls_codec_read_vector16_span	qsc_tls_codec_read_vector24_span
qsc_tls_codec_read_u64	qsc_tls_codec_vector_begin_u8
qsc_tls_codec_vector_begin_u16	qsc_tls_codec_vector_begin_u24
qsc_tls_codec_vector_end_u8	qsc_tls_codec_vector_end_u16
qsc_tls_codec_vector_end_u24	

tlsecdsader.h | 2 identified exports

Public identifier A	Public identifier B
qsc_tls_ecdsa_der_encode	qsc_tls_ecdsa_der_decode

tlsengine.h | 13 identified exports

Public identifier A	Public identifier B
qsc_tls_engine_initialize_client	qsc_tls_engine_initialize_server
qsc_tls_engine_dispose	qsc_tls_engine_handshake
qsc_tls_engine_write_application_data	qsc_tls_engine_read_application_data
qsc_tls_engine_read_application_data_ex	qsc_tls_engine_request_key_update
qsc_tls_engine_emit_session_ticket	qsc_tls_engine_consume_session_ticket
qsc_tls_engine_take_session_ticket	qsc_tls_engine_close
qsc_tls_engine_is_handshake_complete	

tlserrors.h | 1 identified exports

Public identifier A	Public identifier B
qsc_tls_error_to_string	

tlsextensions.h | 35 identified exports

Public identifier A	Public identifier B
qsc_tls_extensions_bitmap_initialize	qsc_tls_extensions_bitmap_set
qsc_tls_extensions_is_permitted	qsc_tls_extensions_encode_supported_versions_client
qsc_tls_extensions_encode_supported_versions_server	qsc_tls_extensions_encode_cookie
qsc_tls_extensions_encode_supported_groups	qsc_tls_extensions_encode_signature_algorithms
qsc_tls_extensions_encode_signature_algorithms_cert	qsc_tls_extensions_encode_key_share_client
qsc_tls_extensions_encode_key_share_server	qsc_tls_extensions_encode_key_share_hello_retry
qsc_tls_extensions_encode_server_name	qsc_tls_extensions_encode_alpn
qsc_tls_extensions_decode_alpn	qsc_tls_extensions_select_alpn
qsc_tls_extensions_encode_psk_key_exchange_modes	qsc_tls_extensions_encode_early_data_empty
qsc_tls_extensions_encode_early_data_max	qsc_tls_extensions_decode_early_data_max

Public identifier A	Public identifier B
qsc_tls_extensions_encode_pre_shared_key_offer	qsc_tls_extensions_encode_pre_shared_key_server
qsc_tls_extensions_decode_pre_shared_key_offer	qsc_tls_extensions_decode_pre_shared_key_server
qsc_tls_extensions_decode_supported_versions_client	qsc_tls_extensions_decode_supported_versions_server
qsc_tls_extensions_decode_cookie	qsc_tls_extensions_decode_supported_groups
qsc_tls_extensions_decode_signature_algorithms	qsc_tls_extensions_decode_key_share_client_hello
qsc_tls_extensions_decode_key_share_server_hello	qsc_tls_extensions_decode_key_share_hello_retry
qsc_tls_extensions_decode_server_name	qsc_tls_extensions_select_cipher_suite
qsc_tls_extensions_select_key_share	

tlsgroups.h | 6 identified exports

Public identifier A	Public identifier B
qsc_tls_groups_descriptor_get	qsc_tls_groups_is_supported
qsc_tls_groups_generate_client_keypair	qsc_tls_groups_client_derive_shared_secret
qsc_tls_groups_server_respond	qsc_tls_groups_key_exchange_state_dispose

tlshandshake.h | 10 identified exports

Public identifier A	Public identifier B
qsc_tls_handshake_write_header	qsc_tls_handshake_read_header
qsc_tls_handshake_send_change_cipher_spec_compat	qsc_tls_handshake_encode_finished
qsc_tls_handshake_decode_finished	qsc_tls_handshake_encode_certificate_verify
qsc_tls_handshake_decode_certificate_verify	qsc_tls_handshake_encode_encrypted_extensions
qsc_tls_handshake_encode_key_update	qsc_tls_handshake_decode_key_update

tlsio.h | 6 identified exports

Public identifier A	Public identifier B
qsc_tls_io_attach	qsc_tls_io_handshake
qsc_tls_io_handshake_ex	qsc_tls_io_send
qsc_tls_io_receive	qsc_tls_io_shutdown

tlskeyschedule.h | 26 identified exports

Public identifier A	Public identifier B
qsc_tls_keyschedule_state_initialize	qsc_tls_keyschedule_state_dispose
qsc_tls_keyschedule_hkdf_extract	qsc_tls_keyschedule_hkdf_expand
qsc_tls_keyschedule_hkdf_expand_label	qsc_tls_keyschedule_derive_secret_with_hash
qsc_tls_keyschedule_derive_secret_empty	qsc_tls_keyschedule_extract_early_secret
qsc_tls_keyschedule_extract_handshake_secret	qsc_tls_keyschedule_extract_master_secret
qsc_tls_keyschedule_derive_handshake_traffic_secrets	qsc_tls_keyschedule_derive_application_traffic_secrets
qsc_tls_keyschedule_derive_exporter_master_secret	qsc_tls_keyschedule_derive_resumption_master_secret
qsc_tls_keyschedule_derive_traffic_keys	qsc_tls_keyschedule_advance_traffic_secret
qsc_tls_keyschedule_compute_finished	qsc_tls_keyschedule_verify_finished
qsc_tls_keyschedule_build_certificate_verify_input	qsc_tls_keyschedule_suite_record_sizes
qsc_tls_keyschedule_suite_hash	qsc_tls_keyschedule_derive_resumption_psk
qsc_tls_keyschedule_derive_binder_key	qsc_tls_keyschedule_derive_client_early_traffic_secret
qsc_tls_keyschedule_derive_early_exporter_secret	qsc_tls_keyschedule_compute_psk_binder

tlsrecord.h | 9 identified exports

Public identifier A	Public identifier B
qsc_tls_record_state_initialize	qsc_tls_record_state_dispose
qsc_tls_record_state_install_keys	qsc_tls_record_state_get_sequence
qsc_tls_record_encode_plaintext	qsc_tls_record_decode_plaintext

Public identifier A	Public identifier B
qsc_tls_record_try_get_span_length	qsc_tls_record_encrypt
qsc_tls_record_decrypt	

tlsserver.h | 11 identified exports

Public identifier A	Public identifier B
qsc_tls_server_config_set_certificate_interface	qsc_tls_server_config_set_client_authorization
qsc_tls_server_authorize_client_certificate	qsc_tls_server_config_set_local_certificate
qsc_tls_server_config_add_certificate_identity	qsc_tls_server_config_set_sni_required
qsc_tls_server_initialize	qsc_tls_server_dispose
qsc_tls_server_process_record	qsc_tls_server_is_handshake_complete
qsc_tls_server_get_negotiated_cipher_suite	

tlssession.h | 3 identified exports

Public identifier A	Public identifier B
qsc_tls_session_ticket_encode	qsc_tls_session_ticket_decode
qsc_tls_session_ticket_dispose	

tlssigalgs.h | 13 identified exports

Public identifier A	Public identifier B
qsc_tls_signature_scheme_descriptor_get	qsc_tls_signature_scheme_is_supported
qsc_tls_signature_scheme_is_certificate_verify_capable	qsc_tls_signature_scheme_is_pq
qsc_tls_signature_scheme_is_mldsa	qsc_tls_signature_scheme_private_key_size
qsc_tls_signature_scheme_public_key_size	qsc_tls_signature_scheme_signature_size
qsc_tls_signature_scheme_validate_signature_length	qsc_tls_signature_scheme_x509_algorithm
qsc_tls_signature_scheme_matches_x509_algorithm	qsc_tls_signature_scheme_hash
qsc_tls_signature_scheme_name	

tlssignerdefault.h | 2 identified exports

Public identifier A	Public identifier B
qsc_tls_signer_default_sign	qsc_tls_signer_default_verify

tlssocket.h | 73 identified exports

Public identifier A	Public identifier B
qsc_tls_socket_result_clear	qsc_tls_socket_status_string
qsc_tls_socket_options_initialize_default	qsc_tls_socket_ticket_policy_initialize_default
qsc_tls_socket_context_initialize	qsc_tls_socket_context_dispose
qsc_tls_socket_context_set_default_client_policy	qsc_tls_socket_context_set_default_server_policy
qsc_tls_socket_context_set_mlkem_hybrid_policy	qsc_tls_socket_context_set_experimental_pqc_policy
qsc_tls_socket_context_set_strict_policy	qsc_tls_socket_context_set_development_policy
qsc_tls_socket_context_set_cipher_suites	qsc_tls_socket_context_set_named_groups
qsc_tls_socket_context_set_signature_schemes	qsc_tls_socket_context_load_trust_anchor_file
qsc_tls_socket_context_load_trust_anchor_bundle_file	qsc_tls_socket_context_load_crl_file
qsc_tls_socket_context_load_server_identity_files	qsc_tls_socket_context_load_client_identity_files
qsc_tls_socket_context_add_server_identity_files	qsc_tls_socket_context_set_sni_required
qsc_tls_socket_context_set_client_auth	qsc_tls_socket_context_set_client_authorization
qsc_tls_socket_context_set_socket_options	qsc_tls_socket_context_set_alpn_protocols
qsc_tls_socket_context_clear_alpn_protocols	qsc_tls_socket_context_set_log_callback
qsc_tls_socket_context_set_session_ticket_policy	qsc_tls_socket_context_set_psk_lookup_callback
qsc_tls_socket_session_ticket_is_valid	qsc_tls_socket_context_set_session_ticket

Public identifier A	Public identifier B
qsc_tls_socket_context_clear_session_ticket	qsc_tls_socket_connection_initialize
qsc_tls_socket_connection_dispose	qsc_tls_socket_client_connect_host
qsc_tls_socket_client_connect_host_ex	qsc_tls_socket_client_connect_ipv4
qsc_tls_socket_client_connect_ipv6	qsc_tls_socket_send
qsc_tls_socket_receive	qsc_tls_socket_shutdown
qsc_tls_socket_key_update	qsc_tls_socket_server_send_session_ticket
qsc_tls_socket_connection_set_socket_options	qsc_tls_socket_connection_set_log_callback
qsc_tls_socket_connection_cancel	qsc_tls_socket_get_peer_info
qsc_tls_socket_get_selected_alpn	qsc_tls_socket_connection_get_session_ticket
qsc_tls_socket_connection_clear_session_ticket	qsc_tls_socket_send_frame
qsc_tls_socket_receive_frame	qsc_tls_socket_is_connected
qsc_tls_socket_is_handshake_complete	qsc_tls_socket_negotiated_cipher_suite
qsc_tls_socket_negotiated_group	qsc_tls_socket_negotiated_signature_scheme
qsc_tls_socket_listener_initialize	qsc_tls_socket_listener_set_options
qsc_tls_socket_listener_set_socket_options	qsc_tls_socket_listener_bind
qsc_tls_socket_listener_accept	qsc_tls_socket_listener_close
qsc_tls_socket_server_initialize	qsc_tls_socket_server_configure
qsc_tls_socket_server_set_callbacks	qsc_tls_socket_server_set_log_callback
qsc_tls_socket_server_set_max_clients	qsc_tls_socket_server_start
qsc_tls_socket_server_start_concurrent	qsc_tls_socket_server_stop
qsc_tls_socket_server_dispose	

tlstranscript.h | 6 identified exports

Public identifier A	Public identifier B
qsc_tls_transcript_initialize	qsc_tls_transcript_dispose
qsc_tls_transcript_update	qsc_tls_transcript_snapshot
qsc_tls_transcript_replace_with_message_hash	qsc_tls_transcript_digest_size

transpose.h | 4 identified exports

Public identifier A	Public identifier B
qsc_transpose_bytes_to_native	qsc_transpose_hex_to_bin
qsc_transpose_native_to_bytes	qsc_transpose_string_to_scalar

winutils.h | 17 identified exports

Public identifier A	Public identifier B
qsc_winutils_file_get_attributes	qsc_winutils_file_set_attribute
qsc_winutils_network_statistics	qsc_winutils_process_list
qsc_winutils_process_token_elevate	qsc_winutils_process_terminate
qsc_winutils_registry_key_add	qsc_winutils_registry_key_delete
qsc_winutils_registry_key_list	qsc_winutils_run_executable
qsc_winutils_run_as_user	qsc_winutils_service_list
qsc_winutils_service_list_size	qsc_winutils_service_state
qsc_winutils_user_list	qsc_winutils_current_user
qsc_winutils_test	

x509aia.h | 1 identified exports

Public identifier A	Public identifier B
qsc_x509_aia_get_ocsp_url	

x509cert.h | 4 identified exports

Public identifier A	Public identifier B
qsc_x509_certificate_clear	qsc_x509_certificate_free
qsc_x509_certificate_decode_der	qsc_x509_certificate_find_extension

x509certwrite.h | 28 identified exports

Public identifier A	Public identifier B
qsc_x509_certificate_builder_initialize	qsc_x509_certificate_builder_clear
qsc_x509_certificate_builder_set_serial	qsc_x509_certificate_builder_set_issuer
qsc_x509_certificate_builder_set_subject	qsc_x509_certificate_builder_set_validity
qsc_x509_certificate_builder_set_spki	qsc_x509_certificate_builder_set_signature_algorithm
qsc_x509_certificate_builder_set_basic_constraints	qsc_x509_certificate_builder_set_key_usage
qsc_x509_certificate_builder_set_extended_key_usage	qsc_x509_certificate_builder_set_subject_key_identifier
qsc_x509_certificate_builder_set_authority_key_identifier	qsc_x509_certificate_builder_add_subject_alt_name_dns
qsc_x509_certificate_builder_add_subject_alt_name_ip	qsc_x509_certificate_builder_add_extension
qsc_x509_certificate_builder_encode_tbs_der	qsc_x509_certificate_builder_sign
qsc_x509_cert_issuance_validate_csr	qsc_x509_cert_issuance_filter_requested_extensions
qsc_x509_cert_issuance_apply_csr_extensions	qsc_x509_certificate_builder_set_issuer_from_certificate
qsc_x509_compute_subject_key_identifier	qsc_x509_compute_authority_key_identifier
qsc_x509_certificate_builder_apply_generated_identifiers	qsc_x509_certificate_builder_apply_profile
qsc_x509_certificate_builder_validate_profile	qsc_x509_certificate_encode_pem

x509crl.h | 9 identified exports

Public identifier A	Public identifier B
qsc_x509_crl_clear	qsc_x509_crl_decode_der
qsc_x509_crl_encode_der	qsc_x509_crl_is_current
qsc_x509_crl_check_algorithms	qsc_x509_crl_find_serial
qsc_x509_certificate_is_revoked_by_crl	qsc_x509_crl_is_revoked
qsc_x509_crl_verify	

x509crlwrite.h | 12 identified exports

Public identifier A	Public identifier B
qsc_x509_crl_builder_initialize	qsc_x509_crl_builder_clear
qsc_x509_crl_builder_set_issuer	qsc_x509_crl_builder_set_update_times
qsc_x509_crl_builder_set_signature_algorithm	qsc_x509_crl_builder_validate_issuer
qsc_x509_crl_builder_add_revoked_serial	qsc_x509_crl_builder_add_extension
qsc_x509_crl_builder_encode_tbs_der	qsc_x509_crl_builder_sign
qsc_x509_crl_der_encode_pem	qsc_x509_crl_encode_pem

x509csr.h | 28 identified exports

Public identifier A	Public identifier B
qsc_x509_csr_verify_with_spki	qsc_x509_csr_initialize
qsc_x509_csr_clear	qsc_x509_csr_set_subject
qsc_x509_csr_set_spki	qsc_x509_csr_set_signature_algorithm
qsc_x509_csr_set_extension_request	qsc_x509_csr_get_extension_request
qsc_x509_csr_copy_extension_request	qsc_x509_csr_encode_info_der
qsc_x509_csr_encode_der	qsc_x509_csr_sign
qsc_x509_csr_decode_der	qsc_x509_csr_decode_pem
qsc_x509_csr_find_extension	qsc_x509_csr_verify
qsc_x509_csr_verify_ex	qsc_x509_csr_encode_pem
qsc_x509_csr_add_attribute	qsc_x509_csr_get_attribute
qsc_x509_csr_set_subject_alt_name	qsc_x509_csr_add_san_dns
qsc_x509_csr_add_san_ip	qsc_x509_csr_set_extended_key_usage

Public identifier A	Public identifier B
qsc_x509_csr_set_subject_key_identifier	qsc_x509_csr_set_ml_dsa_signature_algorithm
qsc_x509_csr_set_ml_dsa_spki	qsc_x509_csr_set_ml_kem_spki

x509ext.h | 23 identified exports

Public identifier A	Public identifier B
qsc_x509_extension_initialize	qsc_x509_extensions_initialize
qsc_x509_extension_validate	qsc_x509_extensions_validate
qsc_x509_extension_decode	qsc_x509_extensions_decode
qsc_x509_ext_basic_constraints_decode	qsc_x509_ext_key_usage_decode
qsc_x509_ext_extended_key_usage_decode	qsc_x509_ext_subject_key_identifier_decode
qsc_x509_ext_authority_key_identifier_decode	qsc_x509_ext_basic_constraints_encode
qsc_x509_ext_key_usage_encode	qsc_x509_ext_extended_key_usage_encode
qsc_x509_ext_has_eku	qsc_x509_ext_subject_key_identifier_encode
qsc_x509_ext_authority_key_identifier_encode	qsc_x509_ext_subject_alt_name_decode
qsc_x509_ext_subject_alt_name_encode	qsc_x509_ext_issuer_alt_name_decode
qsc_x509_ext_issuer_alt_name_encode	qsc_x509_ext_subject_alt_name_add_dns
qsc_x509_ext_subject_alt_name_add_ip	

x509host.h | 5 identified exports

Public identifier A	Public identifier B
qsc_x509_dns_name_match	qsc_x509_certificate_match_dns_name
qsc_x509_certificate_match_ip_address	qsc_x509_certificate_match_hostname
qsc_x509_certificate_match_endpoint	

x509key.h | 11 identified exports

Public identifier A	Public identifier B
qsc_x509_private_key_initialize	qsc_x509_private_key_decode_sec1_der
qsc_x509_private_key_decode_sec1_pem	qsc_x509_private_key_decode_pkcs8_der
qsc_x509_private_key_decode_pkcs8_pem	qsc_x509_private_key_validate
qsc_x509_private_key_expected_private_size	qsc_x509_private_key_expected_public_size
qsc_x509_private_key_decode_pkcs8_der_ex	qsc_x509_private_key_decode_pkcs8_pem_ex
qsc_x509_certificate_key_match	

x509keywrite.h | 10 identified exports

Public identifier A	Public identifier B
qsc_x509_private_key_encode_sec1_der	qsc_x509_private_key_encode_sec1_pem
qsc_x509_private_key_encode_pkcs8_der	qsc_x509_private_key_encode_pkcs8_pem
qsc_x509_private_key_encode_pkcs8_der_ex	qsc_x509_private_key_encode_pkcs8_pem_ex
qsc_x509_private_key_encode_pkcs8_ml_dsa_der	qsc_x509_private_key_encode_pkcs8_ml_dsa_pem
qsc_x509_private_key_encode_pkcs8_ml_kem_der	qsc_x509_private_key_encode_pkcs8_ml_kem_pem

x509name.h | 8 identified exports

Public identifier A	Public identifier B
qsc_x509_name_clear	qsc_x509_name_attribute_type_from_oid
qsc_x509_name_attribute_short_name	qsc_x509_name_attribute_long_name
qsc_x509_name_parse	qsc_x509_name_find_first
qsc_x509_name_equals	qsc_x509_name_to_string

x509ocsp.h | 3 identified exports

Public identifier A	Public identifier B
qsc_x509_ocsp_parse_response	qsc_x509_ocsp_validate
qsc_x509_ocsp_verify_responder	

x509pem.h | 18 identified exports

Public identifier A	Public identifier B
qsc_x509_certificate_decode_pem	qsc_x509_chain_free
qsc_x509_store_free	qsc_x509_chain_decode_pem_bundle
qsc_x509_store_load_pem_bundle	qsc_x509_csr_decode_pem_from_bundle
qsc_x509_crl_decode_pem	qsc_x509_private_key_decode_pem_from_bundle
qsc_x509_private_key_decode_sec1_pem_from_bundle	qsc_x509_private_key_decode_pkcs8_pem_from_bundle
qsc_x509_private_key_decode_pkcs8_pem_ex_from_bundle	qsc_x509_pem_encode_private_key_pkcs8
qsc_x509_pem_encode_private_key_sec1	qsc_x509_pem_encode_private_key_pkcs8_from_key
qsc_x509_pem_encode_private_key_sec1_from_key	qsc_x509_pem_encode_certificate
qsc_x509_pem_encode_crl	qsc_x509_pem_encode_csr

x509pkcs12.h | 6 identified exports

Public identifier A	Public identifier B
qsc_x509_pkcs12_initialize	qsc_x509_pkcs12_clear
qsc_x509_pkcs12_parse	qsc_x509_pkcs12_encode_mac_data_der
qsc_x509_pkcs12_verify_mac_data	qsc_x509_pkcs12_decrypt_encrypted_private_key_info

x509rev.h | 3 identified exports

Public identifier A	Public identifier B
qsc_x509_revocation_options_initialize	qsc_x509_certificate_check_revocation_with_crl
qsc_x509_certificate_check_revocation	

x509revext.h | 3 identified exports

Public identifier A	Public identifier B
qsc_x509_crl_check_certificate_scope	qsc_x509_apply_delta_crl
qsc_x509_ocsp_stapled_verify	

x509sig.h | 9 identified exports

Public identifier A	Public identifier B
qsc_x509_signature_algorithm_decode	qsc_x509_signature_algorithm_equal
qsc_x509_signature_algorithm_is_ecdsa	qsc_x509_signature_algorithm_is_ml_dsa
qsc_x509_signature_algorithm_matches_spki	qsc_x509_signature_expected_size
qsc_x509_signature_value_decode_raw	qsc_x509_signature_value_decode_ecdsa
qsc_x509_signature_component_size	

x509sigver.h | 5 identified exports

Public identifier A	Public identifier B
qsc_x509_qsc_verify_state_initialize	qsc_x509_qsc_signature_verify
qsc_x509_qsc_crl_signature_verify	qsc_x509_qsc_csr_signature_verify
qsc_x509_qsc_verify_signed_data	

x509spki.h | 18 identified exports

Public identifier A	Public identifier B
qsc_x509_algorithm_identifier_decode	qsc_x509_algorithm_identifier_validate
qsc_x509_subject_public_key_info_decode	qsc_x509_spki_validate
qsc_x509_spki_is_uncompressed_ec_point	qsc_x509_named_curve_coordinate_size
qsc_x509_named_curve_public_key_size	qsc_x509_spki_get_ec_coordinates
qsc_x509_algorithm_identifier_initialize	qsc_x509_subject_public_key_info_initialize
qsc_x509_pqc_public_key_size	qsc_x509_algorithm_identifier_is_mldsa
qsc_x509_algorithm_identifier_is_mlkem	qsc_x509_algorithm_identifier_initialize_mldsa
qsc_x509_algorithm_identifier_initialize_mlkem	qsc_x509_spki_initialize_ec
qsc_x509_spki_initialize_ml_dsa	qsc_x509_spki_initialize_ml_kem

x509store.h | 8 identified exports

Public identifier A	Public identifier B
qsc_x509_store_initialize	qsc_x509_store_add_anchor
qsc_x509_store_find_anchor_for_certificate	qsc_x509_store_find_anchor_by_subject
qsc_x509_store_find_anchor_by_subject_key_identifier	qsc_x509_store_contains_anchor
qsc_x509_store_find_issuer	qsc_x509_chain_build

x509time.h | 7 identified exports

Public identifier A	Public identifier B
qsc_x509_time_decode	qsc_x509_time_parse_utctime
qsc_x509_time_parse_generalizedtime	qsc_x509_validity_decode
qsc_x509_time_compare	qsc_x509_time_is_valid
qsc_x509_validity_is_valid	

x509verify.h | 18 identified exports

Public identifier A	Public identifier B
qsc_x509_verify_options_initialize	qsc_x509_certificate_is_self_issued
qsc_x509_certificate_is_self_signed	qsc_x509_certificate_is_ca
qsc_x509_certificate_allows_server_auth	qsc_x509_certificate_allows_client_auth
qsc_x509_certificate_check_structure	qsc_x509_certificate_check_algorithms
qsc_x509_certificate_check_validity	qsc_x509_certificate_check_purpose
qsc_x509_certificate_check_hostname	qsc_x509_certificate_check_ip_address
qsc_x509_certificate_check_issuer	qsc_x509_certificate_verify
qsc_x509_chain_is_anchored	qsc_x509_chain_verify
qsc_x509_certificate_verify_ex	qsc_x509_chain_verify_ex

x509wrap.h | 74 identified exports

Public identifier A	Public identifier B
qsc_x509w_profile_initialize	qsc_x509w_profile_apply_preset
qsc_x509w_profile_set_tls_server_defaults	qsc_x509w_profile_set_tls_client_defaults
qsc_x509w_profile_set_ca_defaults	qsc_x509w_profile_set_strict_revocation_defaults
qsc_x509w_profile_set_development_defaults	qsc_x509w_result_initialize
qsc_x509w_trust_store_initialize	qsc_x509w_trust_store_clear
qsc_x509w_server_identity_initialize	qsc_x509w_server_identity_clear
qsc_x509w_deployment_config_initialize	qsc_x509w_tls_bridge_initialize
qsc_x509w_tls_local_certificate_initialize	qsc_x509w_current_utc_time
qsc_x509w_certificate_load_file	qsc_x509w_certificate_load_memory
qsc_x509w_certificate_chain_load_file	qsc_x509w_certificate_chain_load_memory
qsc_x509w_private_key_load_file	qsc_x509w_private_key_load_memory
qsc_x509w_crl_load_file	qsc_x509w_crl_load_memory
qsc_x509w_csr_load_file	qsc_x509w_csr_load_memory

Public identifier A	Public identifier B
qsc_x509w_trust_store_add_anchor	qsc_x509w_trust_store_add_anchor_bundle_memory
qsc_x509w_trust_store_add_anchor_bundle_file	qsc_x509w_trust_store_add_anchor_file
qsc_x509w_trust_store_add_anchor_memory	qsc_x509w_trust_store_add_crl
qsc_x509w_trust_store_add_crl_file	qsc_x509w_trust_store_add_crl_memory
qsc_x509w_server_identity_load_files	qsc_x509w_server_identity_load_configuration
qsc_x509w_trust_store_load_configuration	qsc_x509w_server_identity_get_chain
qsc_x509w_server_identity_validate	qsc_x509w_server_identity_verify
qsc_x509w_certificate_check_role	qsc_x509w_verify_peer_certificates
qsc_x509w_name_string	qsc_x509w_name_get_attribute_first
qsc_x509w_certificate_subject_string	qsc_x509w_certificate_issuer_string
qsc_x509w_certificate_subject_common_name	qsc_x509w_certificate_subject_dns_name_count
qsc_x509w_certificate_subject_dns_name	qsc_x509w_revocation_mode_string
qsc_x509w_locator_policy_string	qsc_x509w_revocation_source_string
qsc_x509w_availability_string	qsc_x509w_status_string
qsc_x509w_stage_string	qsc_x509w_verify_status_string
qsc_x509w_result_message	qsc_x509w_certificate_export_der
qsc_x509w_certificate_export_pem	qsc_x509w_private_key_export_pkcs8_der
qsc_x509w_private_key_export_pkcs8_pem	qsc_x509w_csr_export_der
qsc_x509w_csr_export_pem	qsc_x509w_csr_create
qsc_x509w_csr_add_dns_name	qsc_x509w_csr_sign_der
qsc_x509w_csr_sign_pem	qsc_x509w_csr_verify
qsc_x509w_certificate_issue_from_csr	qsc_x509w_tls_bridge_configure
qsc_x509w_tls_bridge_is_ready	qsc_x509w_tls_bridge_get_interface
qsc_x509w_tls_local_certificate_is_ready	qsc_x509w_tls_local_certificate_from_identity

x509write.h | 39 identified exports

Public identifier A	Public identifier B
qsc_x509_write_length	qsc_x509_write_tag
qsc_x509_write_integer	qsc_x509_write_boolean
qsc_x509_write_null	qsc_x509_write_oid
qsc_x509_write_octet_string	qsc_x509_write_bit_string
qsc_x509_write_utf8_string	qsc_x509_write_printable_string
qsc_x509_write_ia5_string	qsc_x509_write_utctime
qsc_x509_write_generalized_time	qsc_x509_write_sequence
qsc_x509_write_set	qsc_x509_write_explicit
qsc_x509_write_raw	qsc_x509_write_algorithm_identifier_for_signature
qsc_x509_write_algorithm_identifier_for_public_key	qsc_x509_write_spki_ec
qsc_x509_write_spki_ml_dsa	qsc_x509_write_spki_ml_kem
qsc_x509_write_algorithm_identifier	qsc_x509_write_name
qsc_x509_write_validity	qsc_x509_write_general_name
qsc_x509_write_spki	qsc_x509_write_extension
qsc_x509_write_extensions	qsc_x509_write_basic_constraints
qsc_x509_write_key_usage	qsc_x509_write_extended_key_usage
qsc_x509_write_subject_key_identifier	qsc_x509_write_authority_key_identifier
qsc_x509_write_subject_alt_name	qsc_x509_write_issuer_alt_name
qsc_x509_crl_builder_set_authority_key_identifier	qsc_x509_crl_builder_set_crl_number
qsc_x509_compute_authority_key_identifier_from_issuer	

APPENDIX E

Source Provenance and Chain of Custody

A cryptographic qualification is reproducible only if source, harness, platform and test-result identities can be tied together. The following fields appear in the supplied original consolidated records and identify the final examined implementation.

Identity property	Value
Source archive	QSC 2026-11-06-B.zip
Archive SHA-256	2e64df12c4956c4f58cac1aa468a0765f21a05b23dc4f86a257d48ed15dbf45c
Qualification repository	QRCS-CORP/QSC-ARM-Tests
Qualification branch	qualification/acvp-qsc-20261106
Qualification commit	07f32d8a18a91f7a9035994adcbecd234a09b5ac
Common source digest	0f66a603af66b0452b25e438483a15ecf9516fa267c7970ce6b6c7dde3430a5e
x64 consolidated JSON	qsc-acvp-offline-consolidated-20261106.zip
Native consolidated JSON	qsc-arm64-consolidated-final-20261106.zip
Source uniform (x64 / native)	YES / YES
Missing runner results (x64 / native)	0 / 0
Failed runner results (x64 / native)	0 / 0
Unexpected runner results (x64 / native)	0 / 0

Source comparison principle

The qualification branch commit identifies the executed harness context, while the archive digest identifies the library files restored into the runner working tree. Both values are required because the tested source snapshot is not necessarily identical to the library files that would otherwise be present in a Git checkout.

APPENDIX F

Historical Local Differential and Sanitizer Ledger

The following figures reproduce the separate native Linux x86-64 engineering report recorded in the QRCS program. That campaign used a different archived source baseline from the final hosted QSC-ARM-Tests qualification. The local assertion ledger is retained separately to avoid overstating continuity.

Test family	Passed assertions per compiler profile
SHA-2 / HMAC streaming	1935
SHA-3 / SHAKE streaming	5200
AES-GCM long-message differential	590
ECDSA negative input	213
AES-GCM AAD limits	52
KMAC independent comparisons	216
ML-KEM / OpenSSL interoperability	48
ML-DSA / OpenSSL interoperability	54
SLH-DSA SHAKE / OpenSSL interoperability	42
HKDF boundary comparisons	48
AES-CBC rejection cases	96
Manual symmetric self-test	1
TOTAL	8495

Historical compiler profile	Recorded assertions	Reported result
GCC scalar	8,495	PASS
GCC AVX2 / VAES	8,495	PASS
GCC AVX-512 / VAES	8,495	PASS
Clang scalar	8,495	PASS
Clang ASan / UBSan	8,495	PASS

Historical result accounting

Five profiles × 8,495 passing assertions equals 42,475 functional assertion executions. This figure is not added to the fifteen-runner marker total and does not establish current-revision coverage. ASan/UBSan leak detection was disabled; the two slow SLH-DSA 256s sanitizer checks were completed in a separately recorded resumed process.

APPENDIX G

Standards, Primary Evidence and Reference Sources

These references define the normative algorithms and identify the controlled evidence. References to standards are provided for technical scope and verification; applicable certification status is defined in the Document Control section.

[S1] QRCS controlled QSC source archive, `QSC 2026-11-06-B.zip`
Local source artifact; SHA-256 2e64df12c4956c4f58cac1aa468a0765f21a05b23dc4f86a257d48ed15dbf45c

[E1] QRCS-CORP/QSC-ARM-Tests, Windows/Linux x64 offline matrix
https://github.com/QRCS-CORP/QSC-ARM-Tests/actions/runs/42313414867

[E2] QRCS-CORP/QSC-ARM-Tests, ARM64/macOS offline matrix
https://github.com/QRCS-CORP/QSC-ARM-Tests/actions/runs/42313414773

[E3] Qualification commit 07f32d8... and branch
https://github.com/QRCS-CORP/QSC-ARM-Tests/commit/07f32d8a18a91f7a9035994adcbecd234a09b5ac

[L1] Historical local Linux x64 engineering qualification (November 6)
QRCS native Linux x86-64 differential and sanitizer assessment, November 6; separately identified source baseline.

[N1] NIST FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism
https://csrc.nist.gov/pubs/fips/203/final

[N2] NIST FIPS 204 — Module-Lattice-Based Digital Signature
https://csrc.nist.gov/pubs/fips/204/final

[N3] NIST FIPS 205 — Stateless Hash-Based Digital Signature
https://csrc.nist.gov/pubs/fips/205/final

[N4] NIST ACVP documentation and specifications
https://pages.nist.gov/ACVP/

[N5] NIST CMVP FIPS 140-3 standards and SP 800-140 series
https://csrc.nist.gov/Projects/cryptographic-module-validation-program/fips-140-3-standards

[N6] NIST SP 800-140C Rev. 2, Approved Security Functions
https://csrc.nist.gov/pubs/sp/800/140/c/r2/final

APPENDIX H

Terminology and Evidence Definitions

The following terms are defined in the narrow sense used throughout this dossier so that a reader does not confuse a codebase attribute with certification status.

Term	Definition
ACVP	Automated Cryptographic Validation Protocol, used for algorithm validation interaction with the designated server.
AFT	Algorithm Functional Test; a class of test-vector exercise that may be part of a broader ACVP requirement.
CAVP	Cryptographic Algorithm Validation Program for algorithms evaluated through approved testing and validation procedures.
CMVP	Cryptographic Module Validation Program for approved cryptographic modules.
FIPS 140-3	A cryptographic-module security standard whose validation process is distinct from primitive KATs.
Known-answer test (KAT)	Fixed input to expected output correctness test.
Differential test	Comparison against an independent implementation or expected behavior.
Interop test	A test of cryptographic output compatibility between independently implemented or integrated systems.
Native runner	Job executing on its intended operating system and CPU architecture rather than only cross-compiling.
Security class	The QSC compile/test-class identifier 1, 3 or 5 used in this campaign.
Source digest	Content-derived hash recorded for the runner source, separate from a Git revision ID.
Harness digest	Digest associated with the test-harness files/configuration for a job.
Legacy success marker	An expected success string counted by the legacy offline parser; not a certified vector set.
PASS_WITH_COVERAGE_GAPS	The actual consolidated offline result label, indicating that all defined jobs passed with recorded coverage exclusions.
Native consolidation	Final test of expected runner identities, statuses and uniform source provenance.
Export declaration	A lexically identified public function declaration following QSC_EXPORT_API.
Offline qualification	Testing without an authenticated NIST ACVP server session or issued certificate.
Module boundary	Specified cryptographic module components and services relevant to a formal evaluation.
Sensitive security parameter	Secret or other protected data requiring controlled lifecycle handling.
Source-pinned	Runner configured to use an identifiable source archive/digest and specified harness revision.

APPENDIX I

Reproduction and Reporting Protocol

This appendix documents how a qualified reviewer can examine the supplied record and establish exactly which results are being relied upon. It is intended as an audit and diligence checklist, not as a claim that this report has replayed every hosted test.

1. Obtain `QSC 2026-11-06-B.zip` and independently recompute the SHA-256. Reject an archive whose digest differs from the recorded 2e64df12… value.

2. Check out the qualification branch at commit 07f32d8a… and inspect the workflow files. Confirm how the controlled source archive is restored to the runner working tree before compilation.

3. Download both GitHub Actions run artifact sets and locate the native `consolidated.json` files. Compare the expected SHA values and `source_digest` across every runner.

4. Assert six unique x64 job identities and nine unique ARM/macOS identities, with classes 1, 3 and 5 on each platform/compiler group.

5. For each runner, verify `stages_pass`, `legacy_evidence_pass`, `source_match`, `class_match` and the suite PASS markers. Review unexpected and missing artifact sets and any failure diagnostics.

6. Retain PASS_WITH_COVERAGE_GAPS as the original offline acceptance result; reconcile its specified algorithm-scope fields against the technical qualification matrix and Document Control scope statement.

7. Retain the historical local Linux campaign as a distinct source-run dossier. Do not merge its 42,475 assertion count into the hosted fifteen-runner legacy-marker total.

8. For regulated procurement or formal certification initiatives, reconcile the required evaluation standard, documented cryptographic module boundary and issued records with the selected build and operating environment.

END OF TECHNICAL QUALIFICATION DOSSIER